Processor Peripherals

Prof. Claudio Passerone

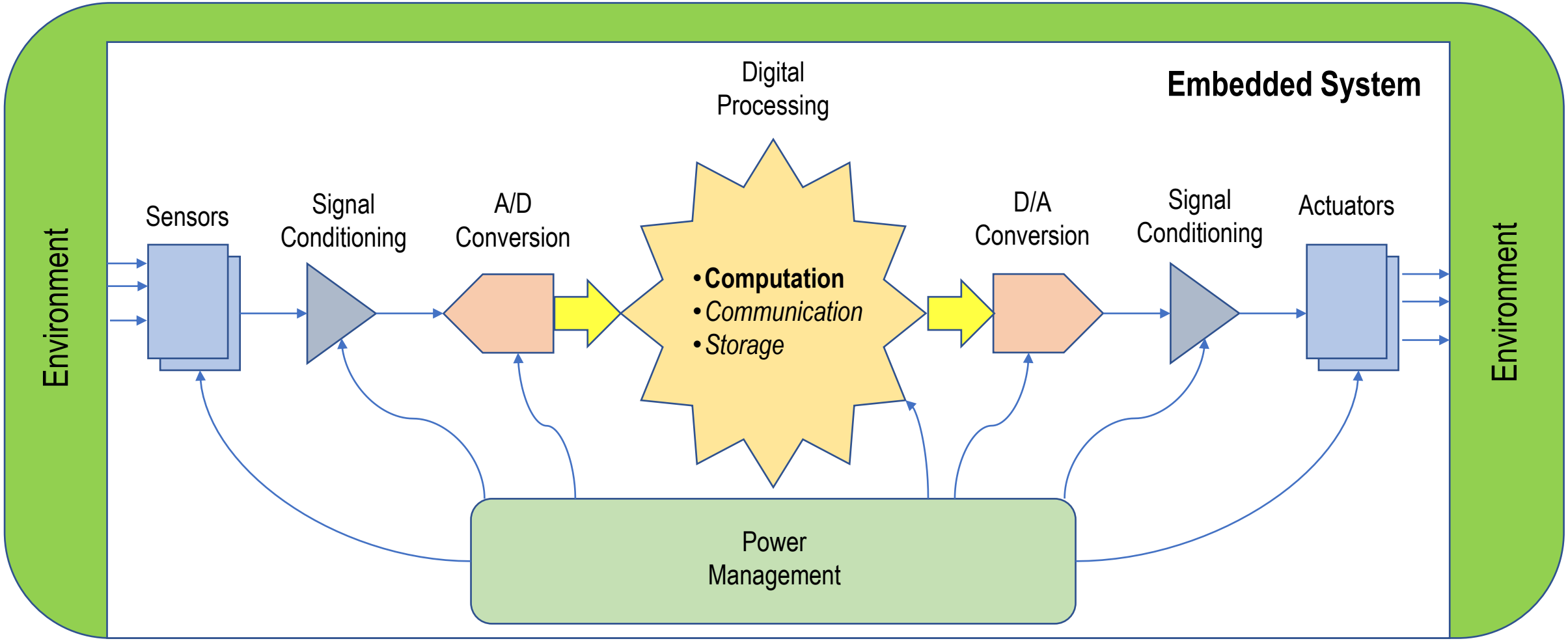
(Electronics for Embedded Systems)

Outline

- Introduction
  - Design process
  - Processor peripherals
- Anatomy of a peripheral
  - Register file and Bus Interface
  - Accessing peripheral registers
  - Processor – Peripheral interaction in a System on Chip
- Examples
  - GPIO, Timer, Counter, PWM, Memory Controller
  - Datasheets of microprocessors
- AMBA Bus

Introduction

- Typical architecture of an Embedded System



Introduction

- Design process: functional description
  - Initial specification is decomposed into interacting functional blocks
    - Specified using formal languages and models
      - Process networks
      - Data flow networks
      - Complex Finite State Machines
      - C code or other high level languages
    - Possibly simulated or executed to verify algorithms
    - No knowledge of the final implementation yet
  - Metrics (performance, costs, etc…) cannot be verified!

Introduction

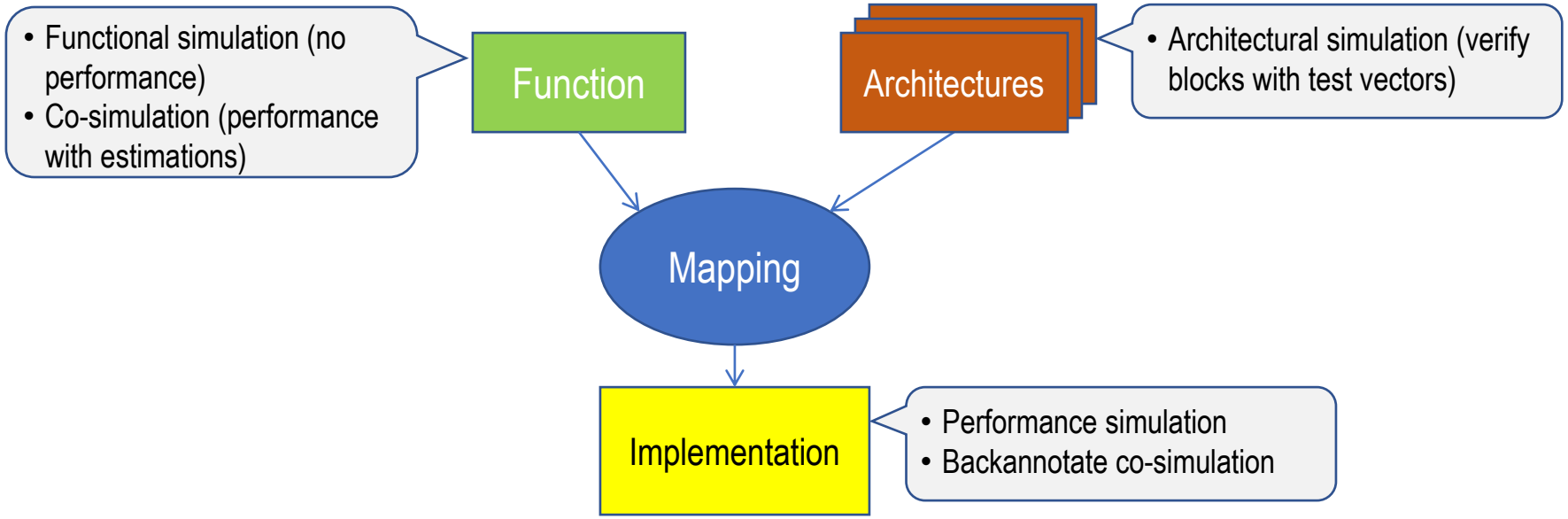
- Design process: architecture description
  - Set of devices that provide
    - Computation
      - Microprocessors, microcontrollers, DSP, FPGA, CPLD, dedicated co-processors (ASIC)
    - Storage
      - Memories, FIFOs
    - Communication
      - Bus, Communication protocols
  - Set of software (firmware) libraries that provide
    - Managing of processes, scheduling, data exchange, …
  - More than one architecture possible
    - Different kind and number of computation and storage devices
    - Different communication links

Introduction

- Design process: mapping
  - Assign an implementation to each functional block
    - Software on a microprocessor
    - Hardware on a programmable device
    - Hardware on an ASIC (Application Specific Integrated Circuit)
  - Assign an implementation to each interconnection
  - Metrics can now be evaluated!
    - Performance, costs, power, etc…
  - Very large design space – many different mappings
    - Impossible to explore it fully
    - Each mapping has its own strengths and weaknesses

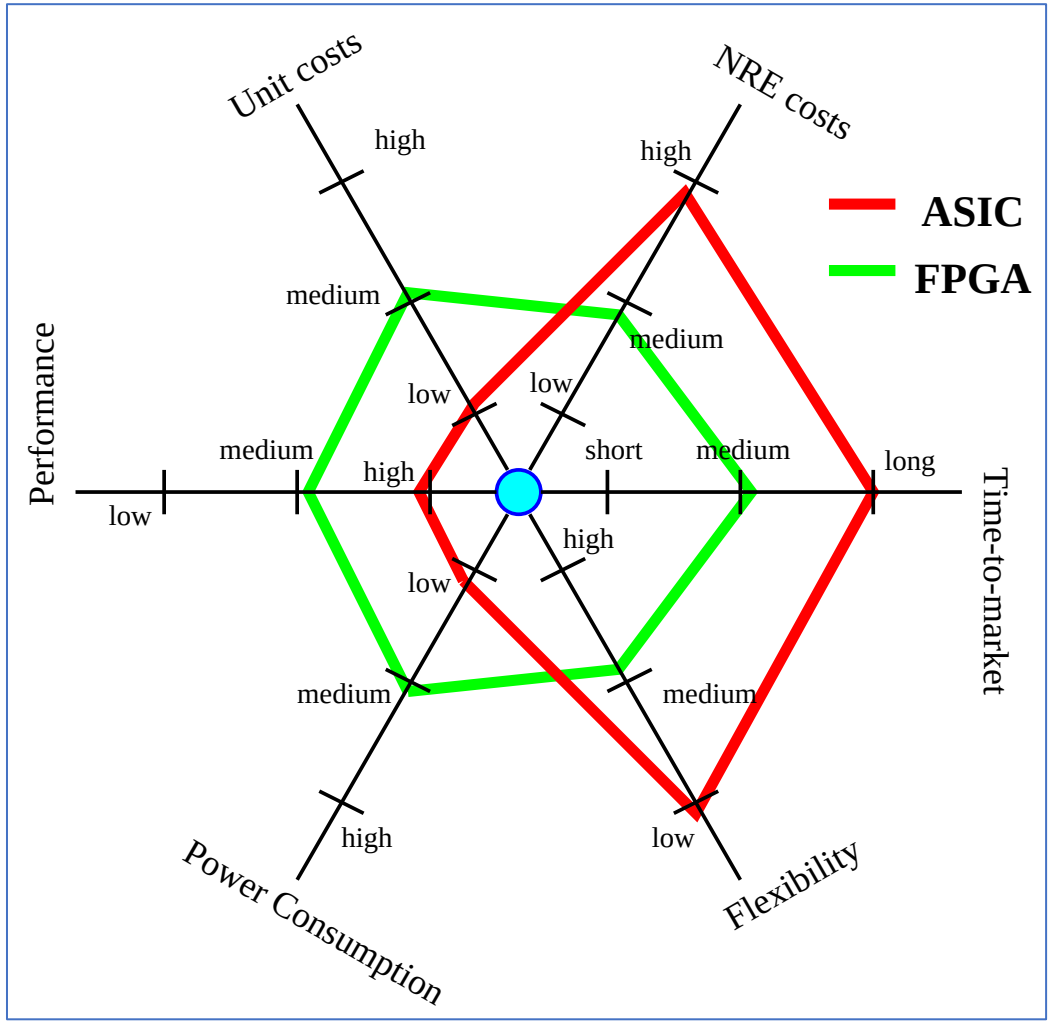
Introduction

- Function – Architecture Codesign (Y-chart)
  - Delay mapping decisions
  - Perform high level co-simulation using estimations



Introduction

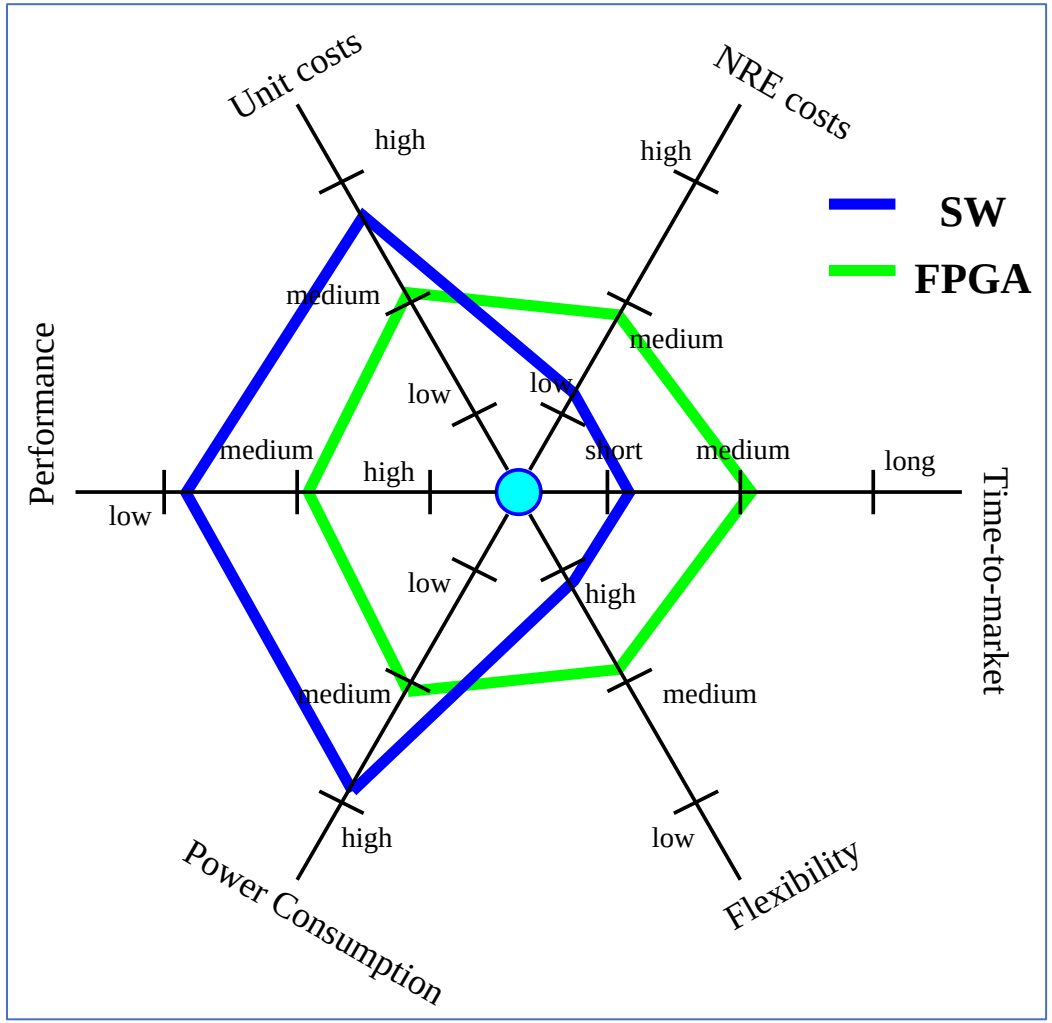
- Mapping to Hardware
  - Advantages (ASIC or FPGA)
    - Better performance and power consumption
    - Lower costs for high volumes
    - Sometimes it is the only option given the constraints
  - Disadvantages
    - High NRE costs
    - Long design cycle (especially for ASICs)
    - Difficult to debug
    - Low flexibility



Better if close to center

Introduction

- Mapping to Software
  - Advantages
    - Easy to develop, often cheap to implement
    - Very low NRE costs
    - Very high flexibility and reusability
  - Disadvantages
    - Low performance
    - High power consumption



Better if close to center

Introduction

- Mapping
  - Most functions are mapped to SW
    - A large part of the design is not critical (not real time)
      - User interfaces, non demanding applications
      - Error states and recovery
    - Modern processors provide good performance
      - Can handle some real time constraints
    - Easier and faster to implement
    - Cheaper to maintain and modify
  - A few functions are mapped to HW
    - Most critical parts of the design
      - Hard real time constraints
      - Power consumption
    - Harder to maintain and modify
    - High NRE costs

Processor peripherals

- Given an application domain, there are sets of functions that are shared by many designs
  - Examples:
    - Communication protocols
    - Memory controllers
    - Interrupt and DMA controllers
    - AD and DA converters
    - Camera and display controllers
    - Audio and video codecs
    - Timer and PWM drivers
    - …
  - Many of these functions would benefit from a HW implementation
    - But it would be too expensive to always implement them from scratch

Processor peripherals

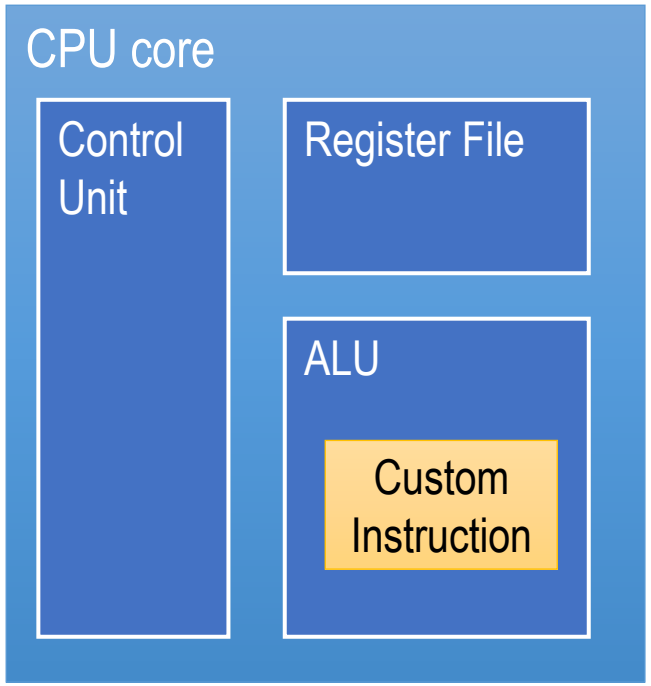
- Peripheral
  - Hardware module
  - Implements a specific function, common in an application domain
  - Often embedded in the same integrated circuit of a microprocessor
- All modern embedded processors include a certain number of peripherals
  - System On Chip (SoC)
    - One or more processor core
    - Many peripherals of different kind
    - Can implement an entire system
  - Differences: kind and number of peripherals
    - Depend on the size of the processor
    - Depend on the application domain of the processor
  - Differences: connection to the processor core

Processor peripherals

- Different connections to the core
  - Within the pipeline of the processor
  - Embedded on the same integrated circuit, with an On-Chip Bus
  - External circuit mapped in a shared memory
  - External circuit with communication protocol
- Different communication performance
  - Communication shall not be the bottleneck
    - Best communication depends on the amount of data that is exchanged between processor and peripheral and its ratio compared to the peripheral computation

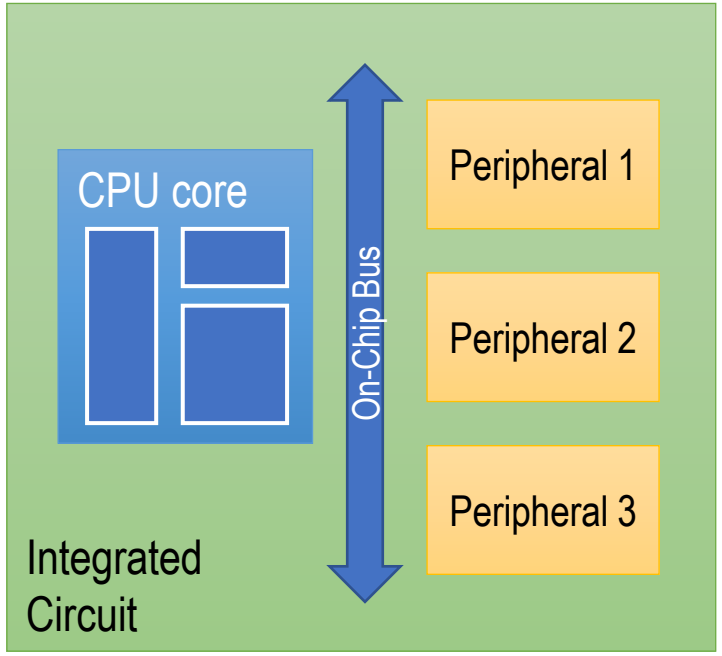
Processor peripherals

- Peripheral within the processor pipeline
  - Custom instruction (not really a peripheral)
    - Instructions dedicated to a specific application domain
      - MAC instruction on a DSP
    - Direct access to the processor register file
    - Small kernel that runs in the processor pipeline
    - Lowest communication overhead
      - But can only transfer a limited amount of data
    - For specific needs, the processor must be configurable
      - Soft Processors (Nios II)
      - Embedded FPGA (XiRisc)
      - Soft Processor targeting hardware (Tensilica Xtensa)



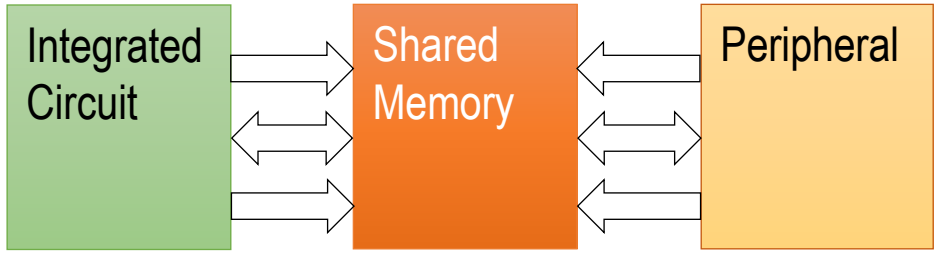
Processor peripherals

- Peripheral on the same chip, but outside the core
  - Embedded in the same integrated circuit (SoC)
  - Connected to an On-Chip Bus
    - Bus: shared set of interconnections
    - AMBA, Avalon, AXI, Network on Chip (NoC)
  - Accessed by the processor
    - I/O Mapped
    - Memory mapped
  - Low communication overhead
    - Data does not travel outside the integrated circuit
    - Set of peripheral registers to exchange data with the processor core
  - We will analyze in detail



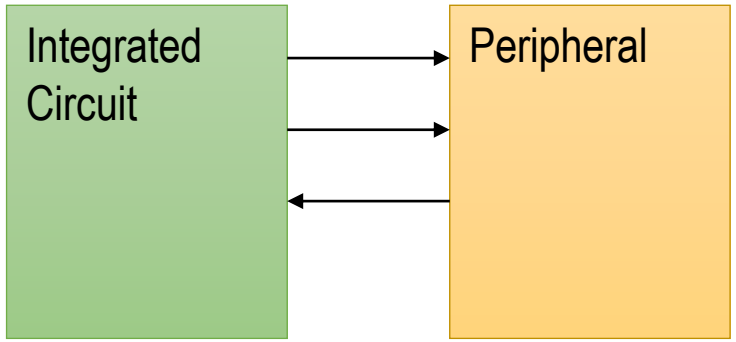
Processor peripherals

- Peripheral on external chip with shared memory
  - External integrated circuit or FPGA
    - Coprocessor (e.g., JPEG or MPEG codec)
    - Application specific digital circuit
  - Shared memory for communication
    - Usually large, on the same chip of the peripheral
    - Multiport
    - The processor core often needs a memory controller (embedded peripheral) to access the shared memory
  - High communication overhead
    - Peripheral usually implements a costly and complex function to make the communication overhead acceptable



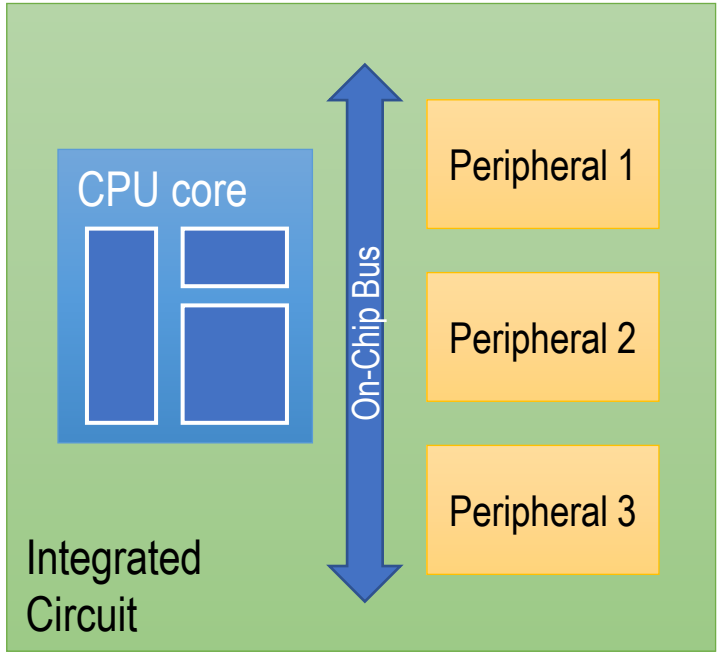
Processor peripherals

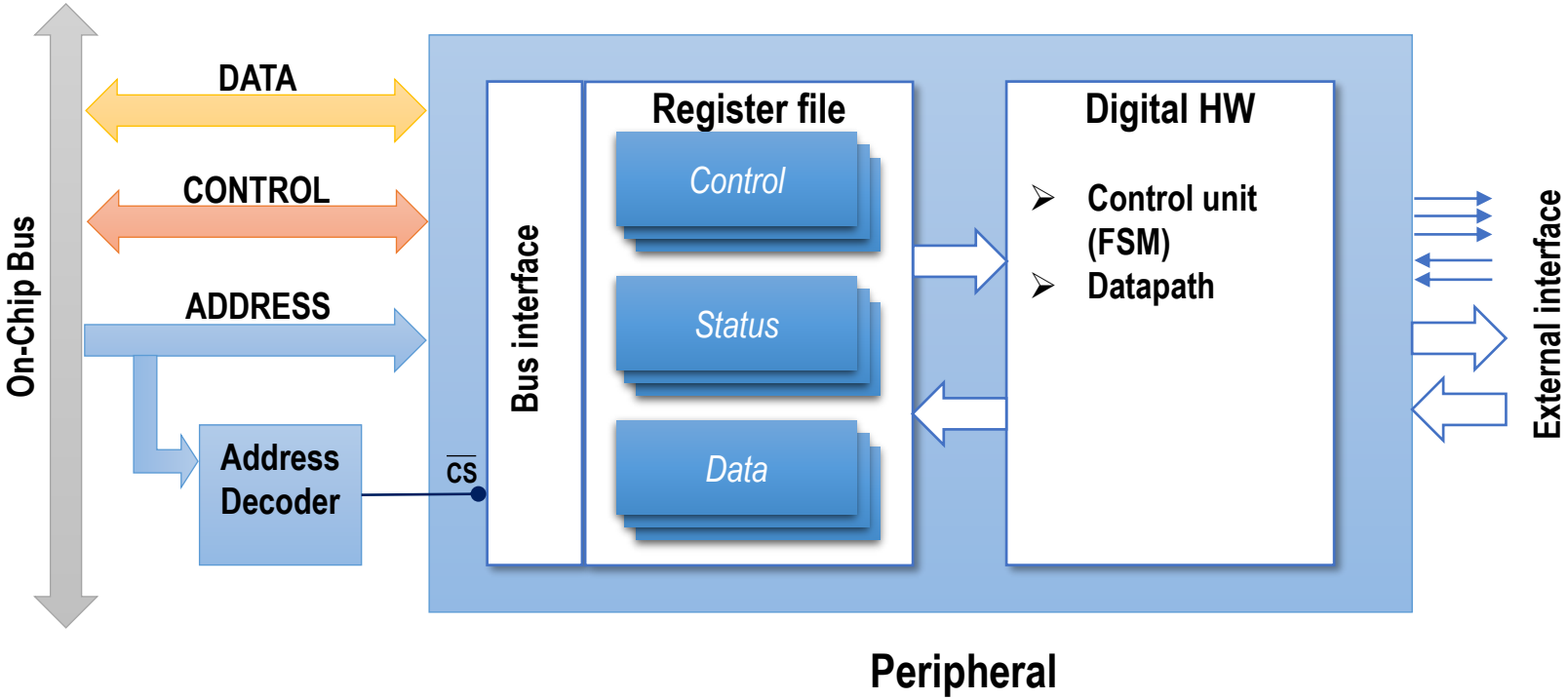
- Peripheral on external chip without shared memory
  - External integrated circuit
    - Audio codec
    - A/D and D/A converters in general
    - Ethernet and communication protocol adapters
  - Standard communication protocol
    - SPI, I^2C, I^2S, UART, USB
    - PCI, PCIExpress, SATA
    - Sometimes custom communication protocol
  - High communication overhead
    - Custom parallel protocol sometimes used to increase communication performance



Anatomy of a peripheral

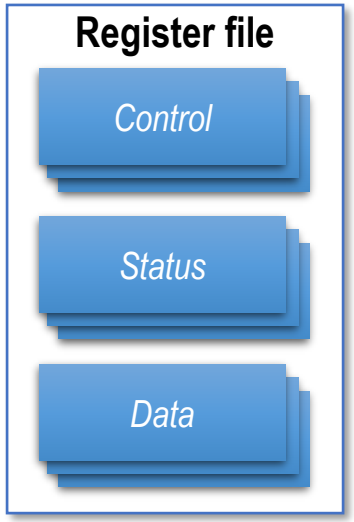
- Focus on embedded peripherals
  - Internal schematic diagram and connections





Anatomy of a peripheral

- Communication between processor core and peripheral only through the peripheral register file
- Peripheral register file contains three types of registers
  - Control registers
    - Used to configure the peripheral
    - Written by the processor, read by the peripheral
  - Status registers
    - Used to report the current state of the peripheral
    - Read by the processor, written by the peripheral
  - Data registers
    - Exchange of data
    - Read and written by both processor and peripheral
- The actual number of registers depends on the peripheral and its function



Anatomy of a peripheral

- Register contents and functions described in the datasheet
- Register types
  - Integers or C-like type
    - A divisor to set a clock frequency
    - A threshold for a counter/timer
    - An address in a memory controller
    - Data
    - …
  - Bit fields
    - Each bit or group of bits within a register has a different function
    - Processor core must use bit masking and shadow registers to correctly access them

SPI Baud Rate Register (SPI_BAUD)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Reset = 0x0000

Baud Rate[15:0]
SCLK/(2 × SPI_BAUD)

SPI Status Register (SPI_STAT)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Reset = 0x0001

TXCOL (Transmit Collision Error) - W1C
When set, corrupt data may have been transmitted

RXS (RX Data Buffer Status) - RO
0 - Empty
1 - Full

RBSY (Receive Error) - W1C
Set when data is received with receive buffer full

TXS (SPI_TDBR Data Buffer Status) - RO
0 - Empty
1 - Full

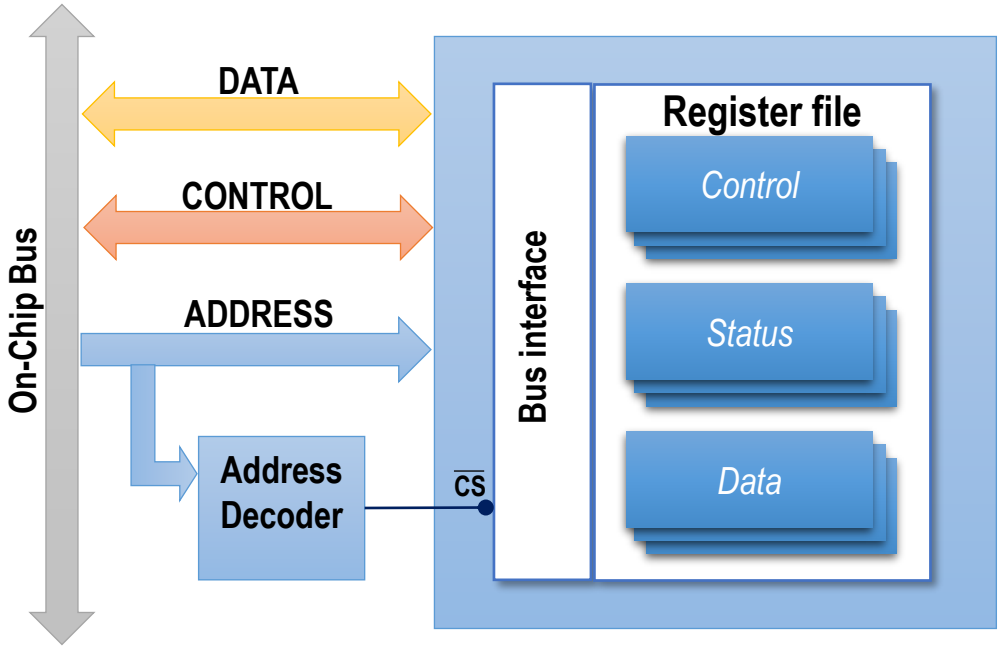
SPIF (SPI Finished) - RO
Set when SPI single word transfer complete

MODF (Mode Fault Error) - W1C
Set in a master device when some other device tries to become the master

TXE (Transmission Error) - W1C
Set when transmission occurred with no new data in SPI_TDBR

Anatomy of a peripheral

- Communication with processor through an On-Chip Bus
- The Bus Interface, always present, includes
  - Address Bus and Chip Select
    - Address decoder drives the Chip Select (CS) using most significant bits of the address and possibly some control lines (see I/O and memory mapping)
    - Least significant bits of the address select the register
  - Data Bus
    - Used to write/read information in the registers
  - Control Bus
    - Determines write or read
    - Contains Interrupt Request (IRQ) line
    - Other functions depending on bus type (arbitration, byte enables, bank selection, …)

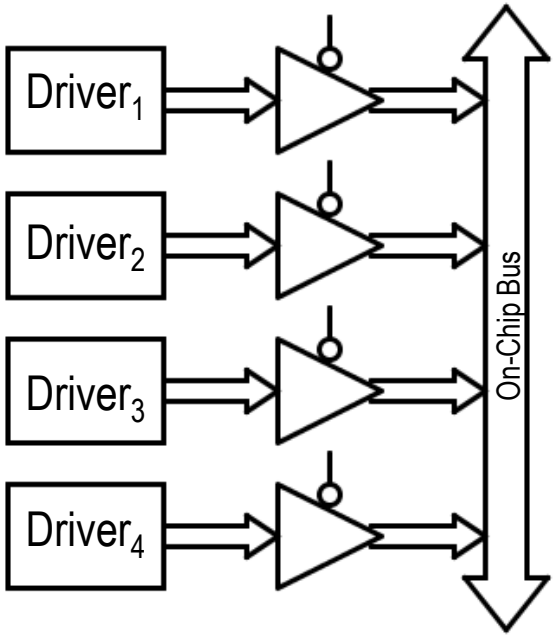


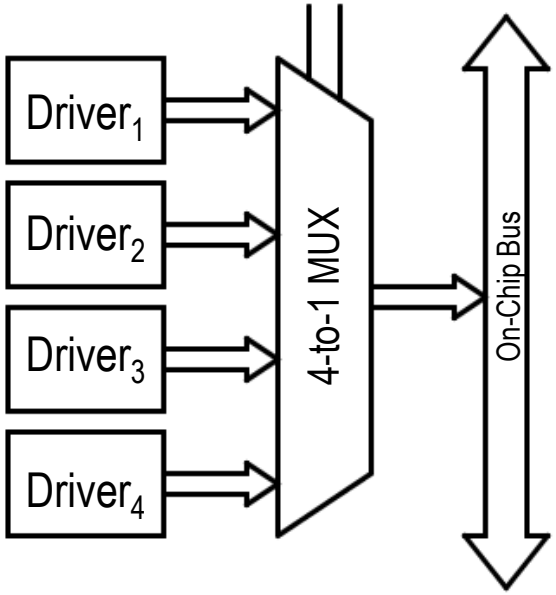
Anatomy of a peripheral

- The processor core is a master on the On-Chip Bus
  - A master can initiate a bus transaction to transfer data
    - A transaction includes the target address
  - The peripheral is a slave, it cannot generate transactions
- The processor core generates bus transactions to access the peripheral register file
  - The address decoder enables accessing the peripheral when necessary
    - It drives the Chip Select
      - Chip Select from the Address Decoder affects the bus interface only
      - The digital hardware implementing the internal behavior is always active, regardless of the status of the Chip Select
  - The peripheral bus interface selects the target register
    - Based on the content of the On-Chip Bus
  - Multiple bus transactions are required to access multiple registers

Anatomy of a peripheral

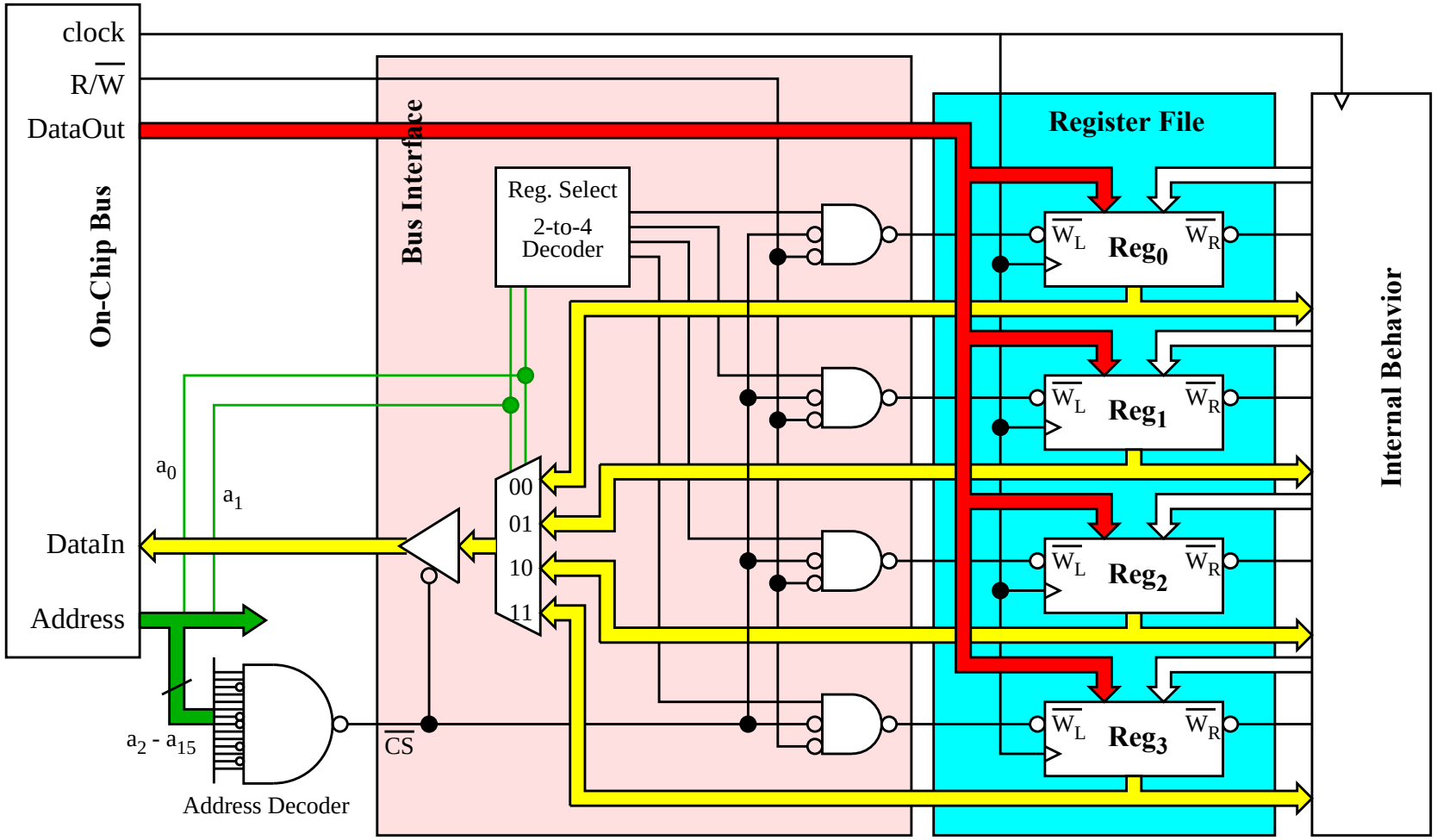
- The On-Chip Bus is a shared resource
  - Multiple blocks may drive the same wires concurrently
    - Risk of conflicts and possibly short circuits
  - Need to prevent concurrent driving of wires
    - Arbitration between masters
      - Only a single master can access the bus at any given instant of time
    - Selection of slaves
      - Using addresses and possibly other control signals
        - Address decoders
    - Different driving techniques
      - All drivers use tri-state output buffers
        - A single one is enabled, the others are in tri-state
      - A multiplexer drives the shared resource
        - A single driver output is selected
        - No need of tri-state outputs





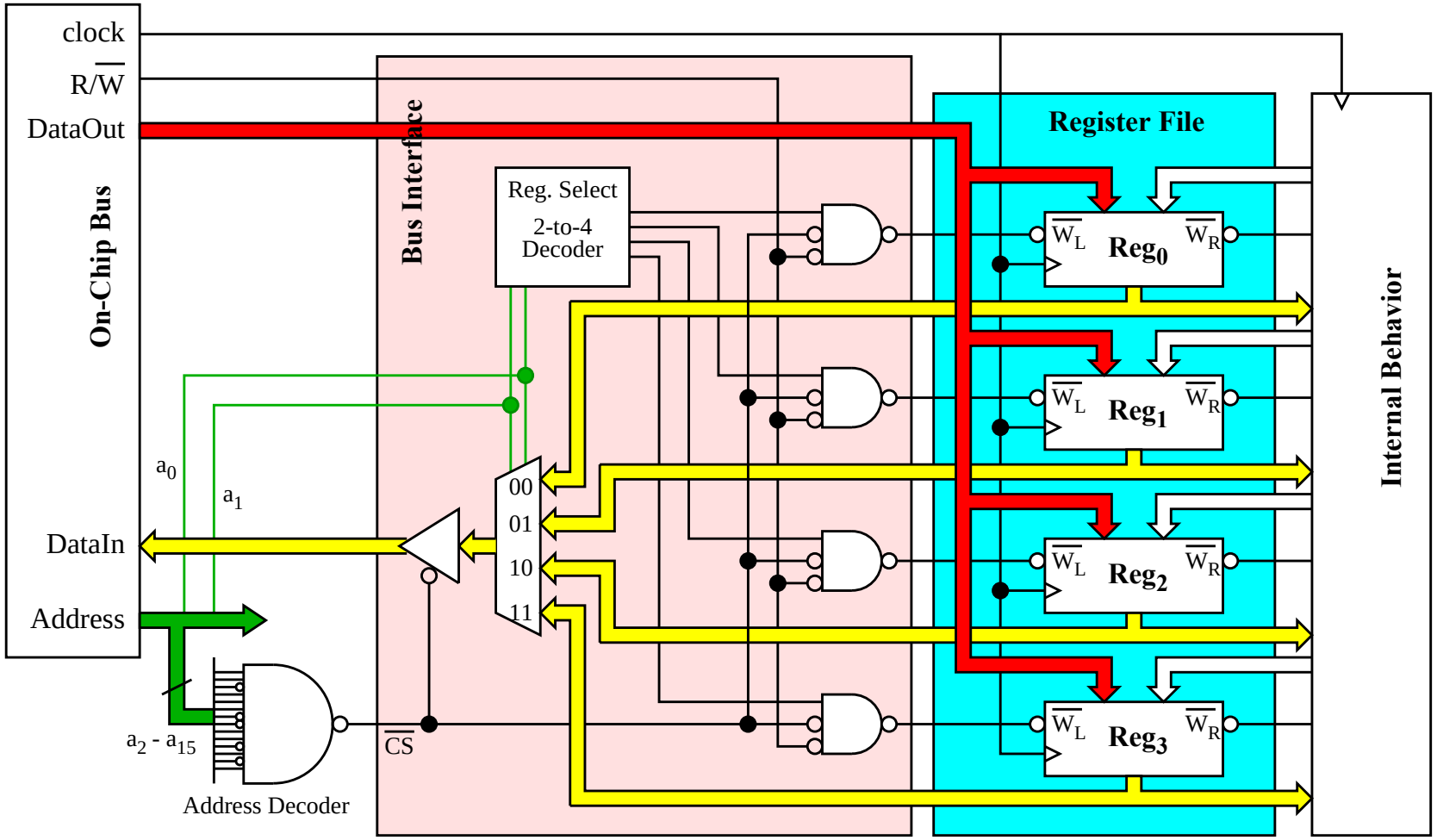
Anatomy of a peripheral

- Bus Interface example
  - Most significant bits of address
    - Select the peripheral
  - Least significant bits of address
    - Select a register
      - Write enable for writing
      - Multiplexer for reading
  - Write signal $R/\overline{W}$
    - When 0, it enables writing from the Bus Interface



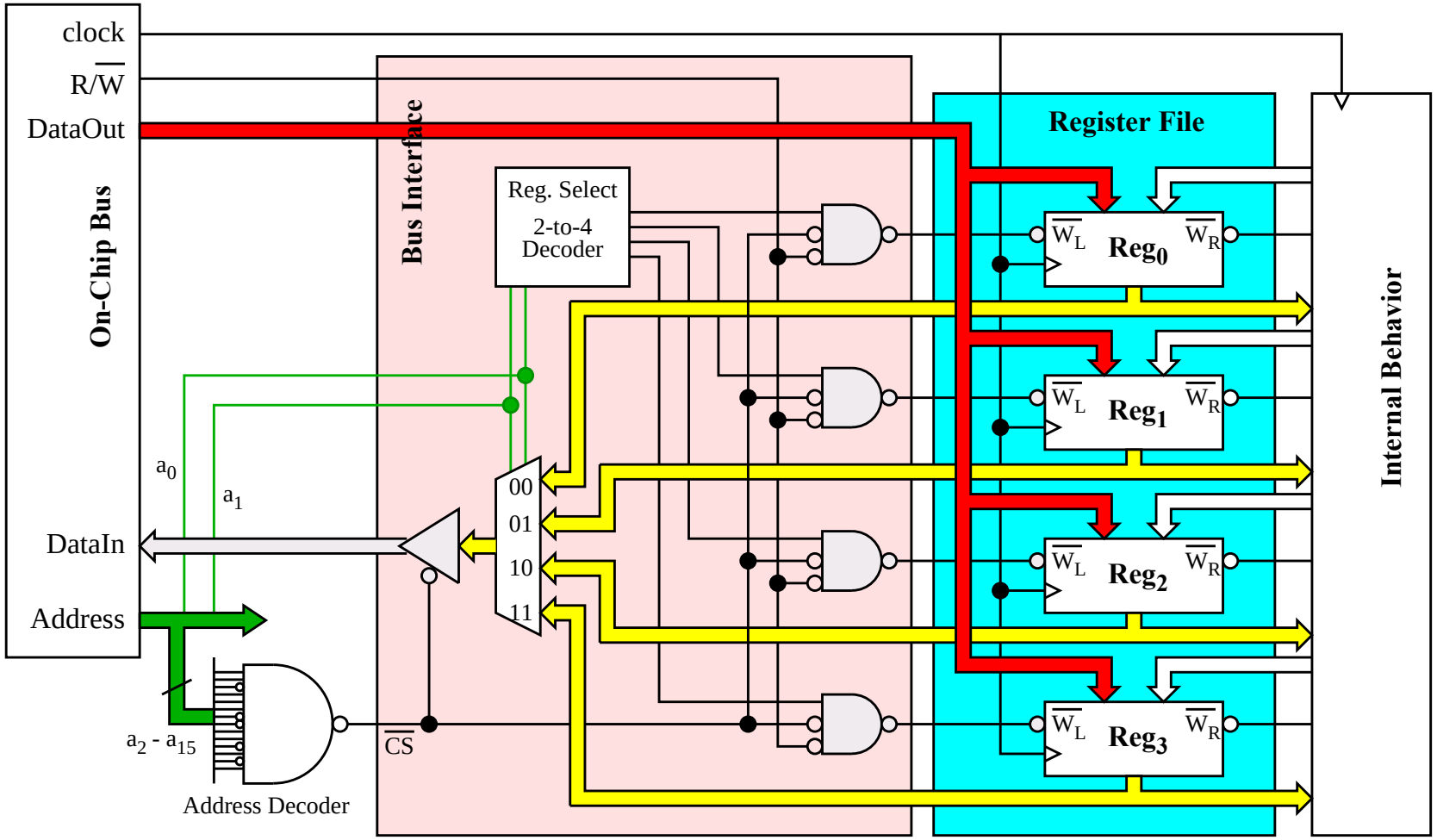
Anatomy of a peripheral

- Bus Interface example
  - Registers in the Register File are made of FFs
  - They can be read and written concurrently by the internal behavior of the peripheral
    - Signal $\overline{W_R}$ to enable writing (write from the Right-hand side)
  - They are read or written one at a time from the Bus Interface
    - Signal $\overline{W_L}$ to enable writing (write from the Left-hand side)



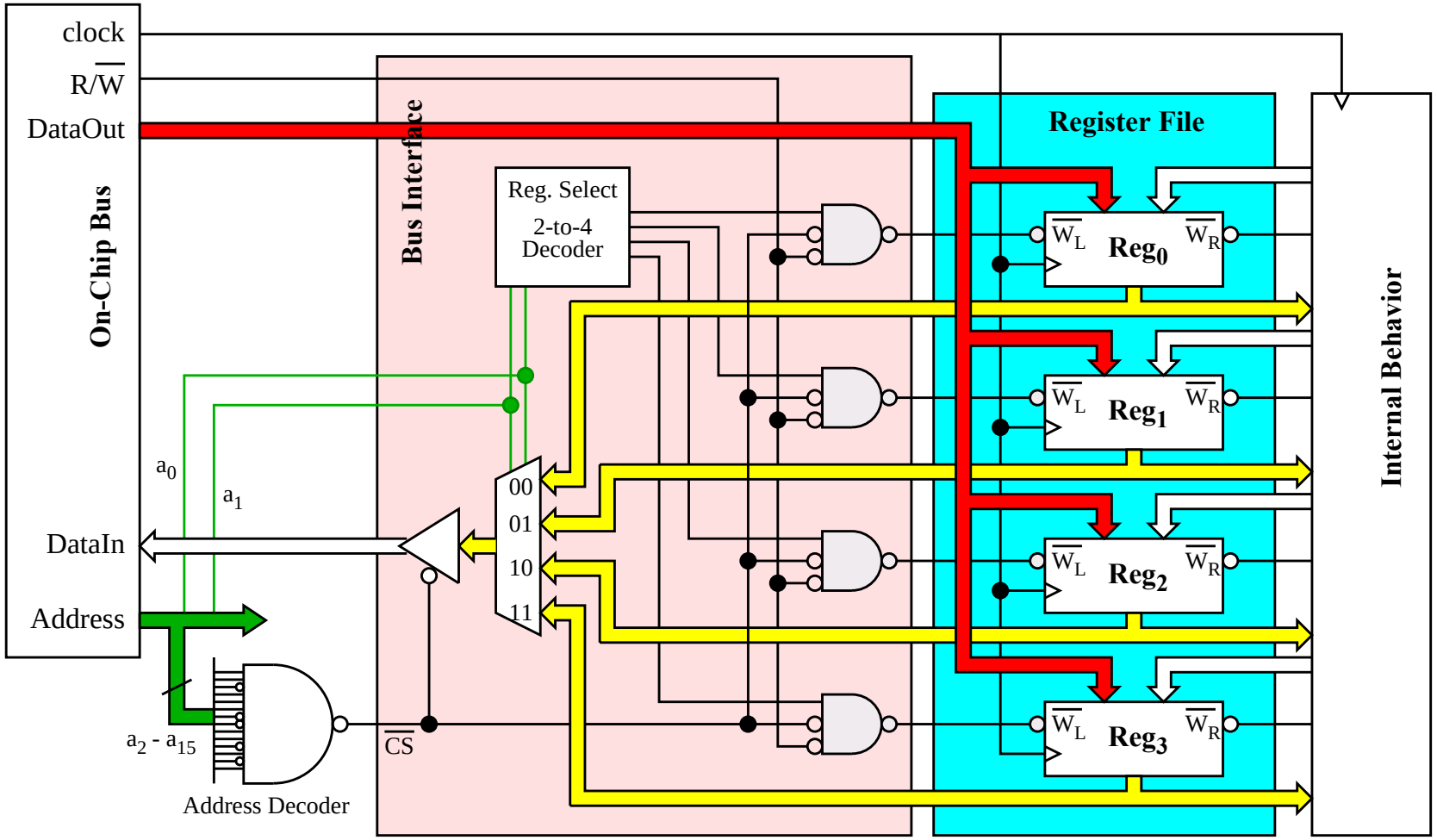
Anatomy of a peripheral

- Bus Interface example
  - Peripheral not selected
    - The output of the Address Decoder in not active
    - The output buffer driving the On-Chip Bus signal DataIn is in tristate
    - The $\overline{W_L}$ signals of the registers are inactive
      - The value on DataOut reaches the registers, but it is NOT written
  - No READ
  - No WRITE



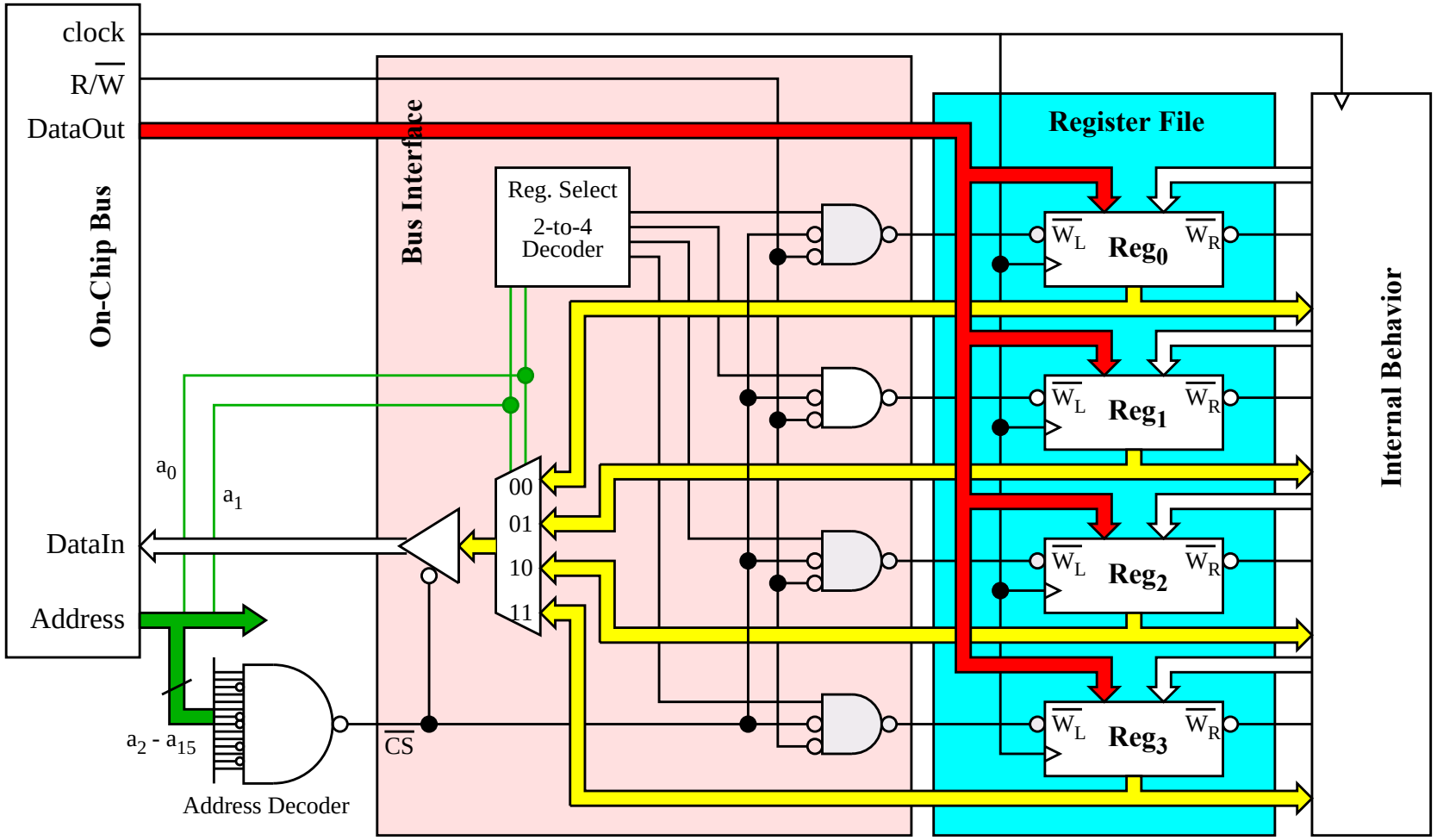
Anatomy of a peripheral

- Bus Interface example
  - Peripheral selected
  - Reading ($R/\overline{W} = 1$)
    - The output of the Address Decoder is active
    - Signals a_0 and a_1 drive the mux to select the output of a register
    - The output buffer driving the On-Chip Bus signal DataIn is active
      - The data is read by the processor
    - The $\overline{W_L}$ signals of the registers are inactive
      - The value on DataOut reaches the registers, but it is NOT written



Anatomy of a peripheral

- Bus Interface example
  - Peripheral selected
  - Writing ($R/\overline{W} = 0$)
    - The output of the Address Decoder is active
    - The $\overline{W_L}$ signal of a single register is active
      - Which one depends on the values of a_0 and a_1 through the 2-to-4 Decoder
      - The value on DataOut from the processor is written in the selected register



Anatomy of a peripheral

- Bus Interface example
  - Sample VHDL implementation
    - The 4 registers have different size
      - Extended with 0s when read
      - Written using only a portion of the data bus
    - Chip select signal is active high
    - Write signal is active low
    - Example adapted from Nios II peripherals

```vhdl
entity peripheral is
    port (
        -- Bus inputs:
        signal address:        in  std_logic_vector(1 downto 0);
        signal chipselect:     in  std_logic;
        signal clk:            in  std_logic;
        signal reset_n:        in  std_logic;
        signal write_n:        in  std_logic;
        signal writedata:      in  std_logic_vector(31 downto 0);
        -- Bus outputs:
        signal readdata:       out std_logic_vector(31 downto 0);
        -- Other peripheral inputs and outputs:
        -- ...
    );
end entity peripheral;
```

DataOut of On-Chip-Bus

DataIn of On-Chip-Bus

```vhdl
architecture rtl of peripheral is
    signal read_mux_out:            std_logic_vector(31 downto 0);
    signal Reg0:                    std_logic_vector(15 downto 0);
    signal Reg1:                    std_logic_vector(3 downto 0);
    signal Reg2:                    std_logic_vector(3 downto 0);
    signal Reg3:                    std_logic_vector(7 downto 0);
begin
    -- Select register based on offset
    with address select
        read_mux_out <= "0000000000000000"              & (Reg0)  when "00",
                        "000000000000000000000000000000" & (Reg1)  when "01",
                        "000000000000000000000000000000" & (Reg2)  when "10",
                        "00000000000000000000000"       & (Reg3)  when "11",
                        (others => '0')                           when others;
    -- Read operation on the Bus
    process (clk, reset_n)
    begin
        if (reset_n = '0') then
            readdata <= (others => 'Z');
        elsif (rising_edge(clk) and chipselect = '1' and write_n = '1') then
            readdata <= read_mux_out;
        end if;
    end process;
    -- Write operation on the Bus
    process (clk, reset_n)
    begin
        if (reset_n = '0') then
            Reg0 <= "0000000000000000";
            Reg1 <= "0000";
            Reg2 <= "0000";
            Reg3 <= "00000000";
        elsif (rising_edge(clk) and chipselect = '1' and write_n = '0') then
            case address is
                when "00" => Reg0 <= writedata(15 downto 0);
                when "01" => Reg1 <= writedata( 3 downto 0);
                when "10" => Reg2 <= writedata( 3 downto 0);
                when "11" => Reg3 <= writedata( 7 downto 0);
            end case;
        end if;
    end process;
    -- The peripheral implementation follows
    -- ...
end rtl;
```

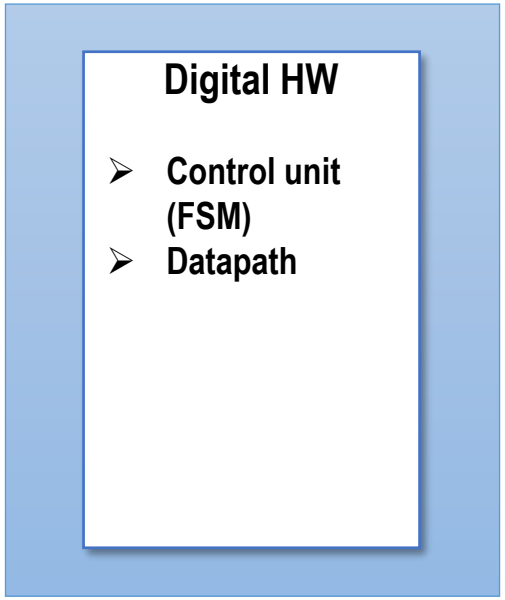
Output mux

Tristate output buffer

2-to-4 Decoder

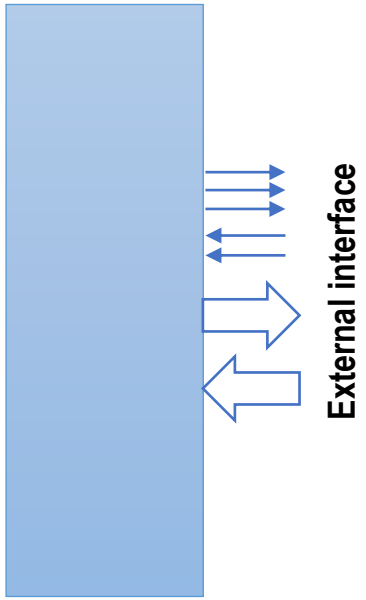
Anatomy of a peripheral

- Internal behavior
  - Typically implemented using digital hardware
    - Custom hardware
    - Complex Finite State Machine
      - Control Unit
      - Data Path
    - Direct access to the register file
      - Peripheral registers read and written in parallel
  - Uses the processor clock signal
    - Sometimes slower clock signals are also possible
  - May include some analog components
    - Analog Front End (AFE) for AD and DA converters



Anatomy of a peripheral

- External interface
  - Communication with the external world
    - Number and type of signals depend on the peripheral
    - Some signals might be connected to other on-chip peripherals (not common)
  - Connected to pins of the SoC
    - Most pins of a SoC belong to peripherals rather than the processor core
    - The SoC datasheet reports all external interfaces and their function
  - Not always present
    - Timer for instance does not typically need it

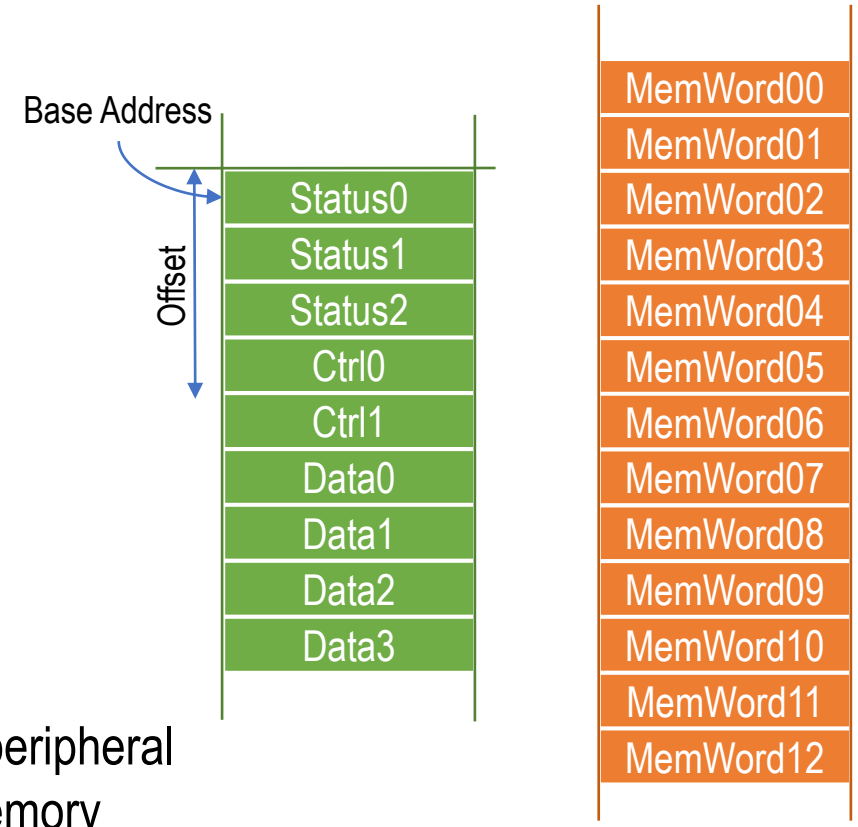


Anatomy of a peripheral

- A bus transaction is characterized by the address generated by the bus master (i.e. the processor core)
  - The Address Bus of the On-Chip Bus has a certain number of bits
    - 32 bits is a typical value, for a maximum addressable space of 4GB
  - The amount of memory in the system is generally less than the maximum addressable space
    - A system may have only 128MB out of 4GB of addressable space
  - The master can generate any address
    - Also addresses where there is no memory
    - This can be exploited to access the peripheral registers
- Peripheral registers mapping
  - Two main methods
    - I/O mapping
    - Memory mapping

Anatomy of a peripheral

- I/O Mapping
  - Registers in a peripheral are assigned a contiguous segment of addresses
    - Base Address + Offset
    - Base Address might need to be aligned to a power of 2
      - Makes address decoders easier
    - Offset from 0 to $n-1$, where n is the number of registers
      - Assuming pointer arithmetic like in C language
  - Peripheral addresses can overlap memory addresses
    - A peripheral address range cannot overlap the range of another peripheral
    - A memory address range cannot overlap the range of another memory
  - To distinguish between memory and peripheral (I/O) operation
    - A special signal of the On-Chip Bus is used
    - The addressable memory space for I/O operations is often just a subset of the available addressable space (only some address wires are used, rather than all of them as when targeting a memory)

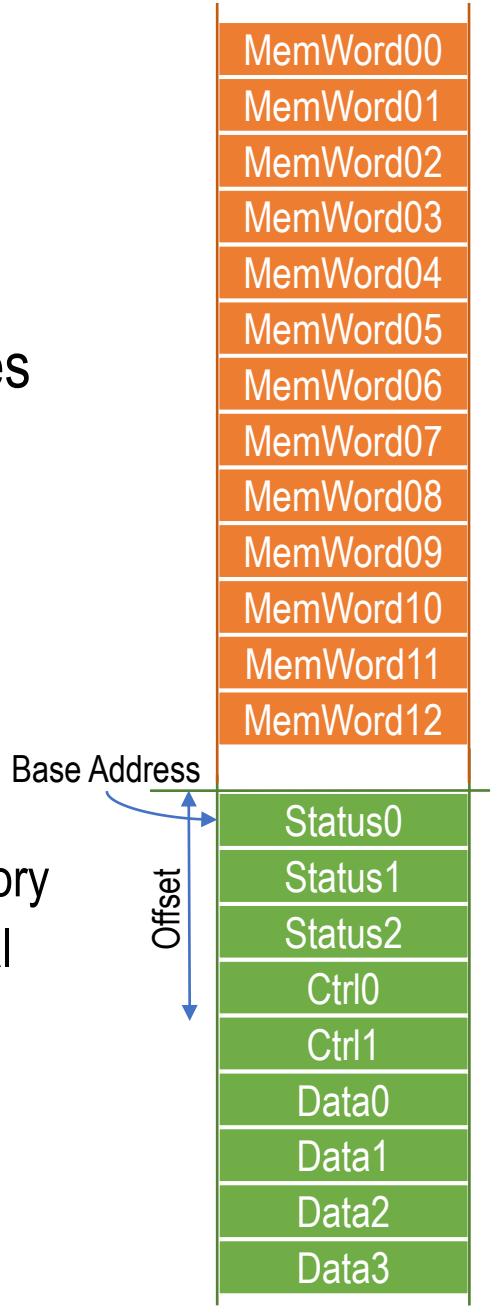


Anatomy of a peripheral

- I/O Mapping (continued)
  - Different assembler instructions used to access memory and I/O
  - Example: x86 architecture
    - A special signal called M/$\overline{\text{IO}}$ is present in the bus
      - When equal to 1, it is a memory operation
      - When equal to 0, it is a peripheral operation
    - Different instructions used for memory and peripheral accesses
      - MOV used to move data to and from memory (sets M/$\overline{\text{IO}}$ to 1)
        - MOV m_{add}, r_n
        - MOV r_n, m_{add}
      - IN and OUT used to access peripheral registers (sets M/$\overline{\text{IO}}$ to 0)
        - IN AX, DX
        - OUT DX, AX
          - Where AX is a processor core register that holds a pointer to the peripheral register

Anatomy of a peripheral

- Memory mapping
  - Registers in a peripheral are assigned a contiguous segment of addresses
    - Similar to I/O mapping
    - Base Address + Offset
    - Base Address might need to be aligned to a power of 2
    - Offset from 0 to $n-1$
  - Peripheral addresses should not overlap memory addresses
    → no need to distinguish
    - A peripheral address range cannot overlap the range of another peripheral or memory
    - A memory address range cannot overlap the range of another memory or peripheral
  - Peripheral addresses should not be cached, if a cache is present
    - Registers are modified by the peripheral, as well as by the processor
    - A special instruction might be used to bypass the cache (Nios II)
      - Use `volatile` in the C source and `-mno-cache-volatile` in gcc flags



Anatomy of a peripheral

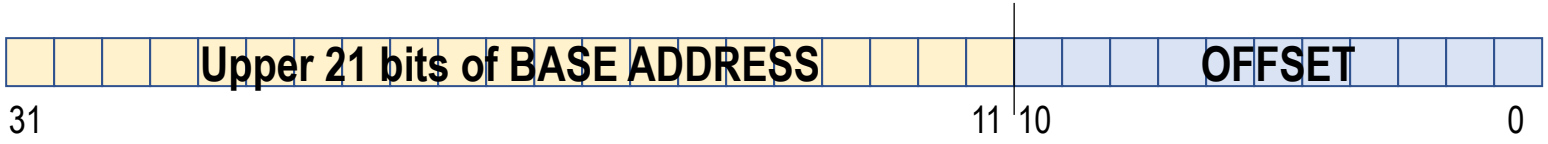
- Memory mapping (continued)
  - The same assembler instructions used to access memory and I/O
  - Example: ARM architecture
    - STR to copy a processor core register in a memory location or in a peripheral register
    - LDR to copy a memory location or a peripheral register into a processor core register
    - The address determines if it is a memory or an I/O operation
    - Examples
      - STR R0, [R5, R1] ; Store value of R0 into an address equal to the sum of R5 and R1
      - LDR R0, [R5, R1] ; Read value from an address equal to the sum of R5 and R1 and put it in R0
      - R5 contains an address
        - For peripheral access, it is the Base Address
      - R1 contains an offset
        - For peripheral access, it is the register index

Anatomy of a peripheral

- Deciding Base Addresses and Offsets
  - The hardware designer of the processor peripheral decides
    - How many registers to include in the peripheral
    - The function of each register
    - The Offset of each register
    - Decision independent of the mapping (I/O or memory mapping)
    - Implemented by properly designing the Bus Interface in the peripheral
  - The hardware designer of the System On Chip decides
    - The Base Address of each peripheral
    - Implemented by properly designing the Address Decoders in the On-Chip Bus
  - A software developer needs to know Base Addresses and Offsets for all peripherals
    - See Memory Map

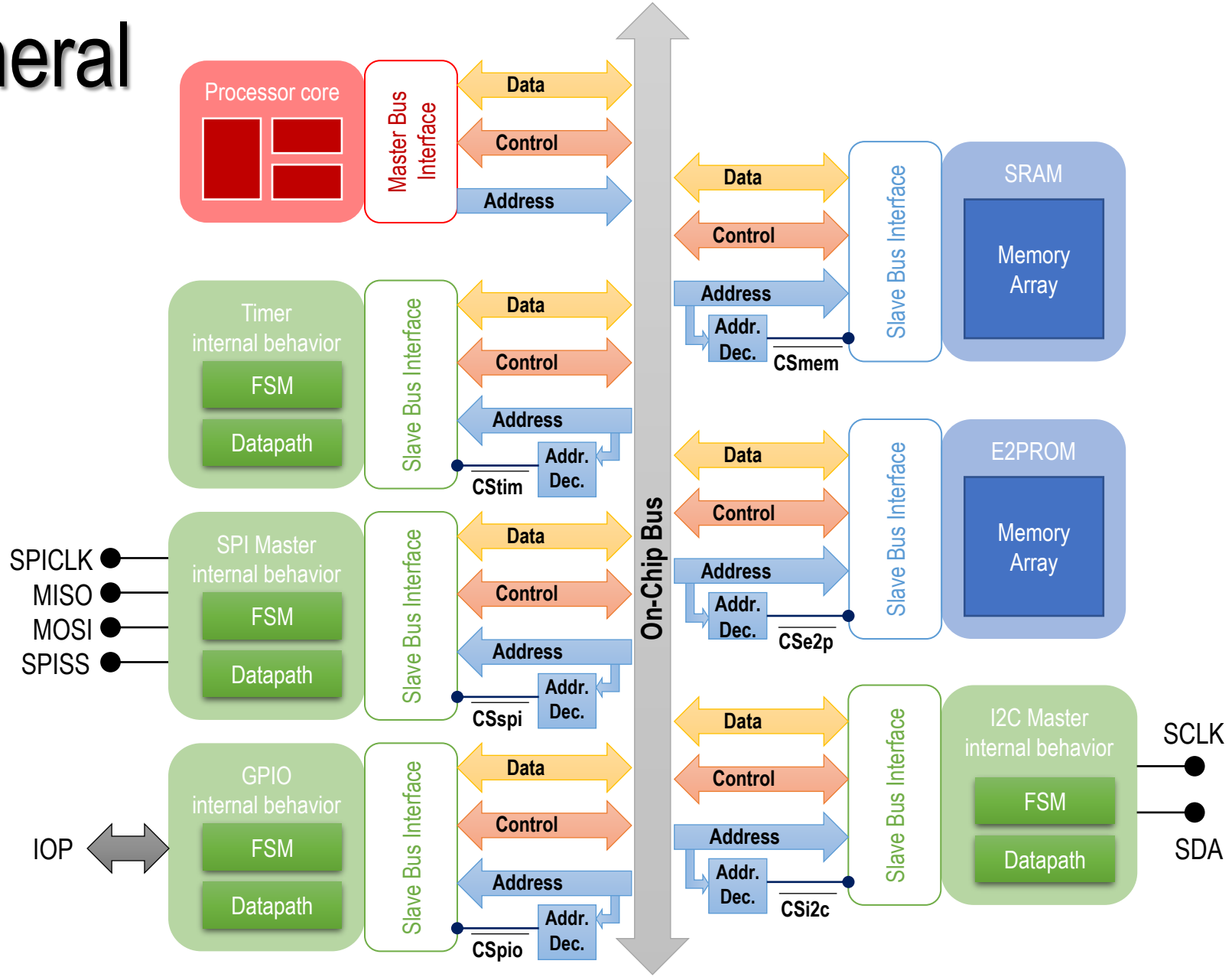
Anatomy of a peripheral

- Deciding Base Addresses and Offsets (continued)
  - Address alignment
    - The Base Address is always an integer multiple of some power of 2
      - Makes the address decoder for the Chip Select easier to implement
    - Example: address on 32 bits, Base Address multiple of 2KBytes (2^{11}), peripheral registers fit in 2KBytes
      - Least 11 significant bits of Base Address are 0
      - Offset is on the least significant 11 bits of the address
      - Address decoder compares 32-11=21 bits against a fixed value
        - Simple combinational logic circuit



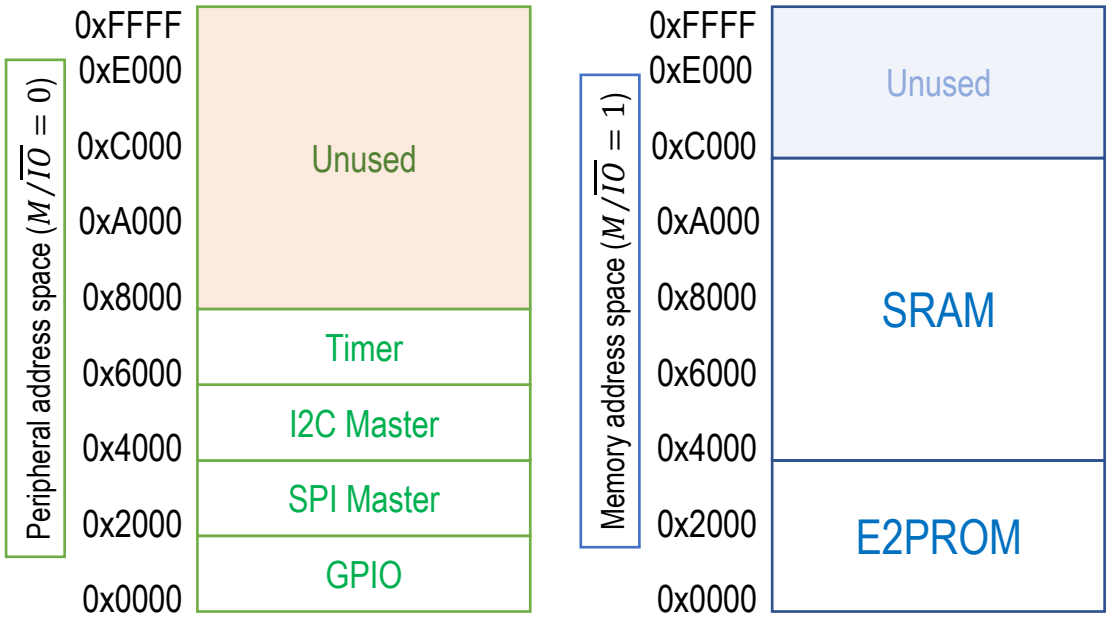
Anatomy of a peripheral

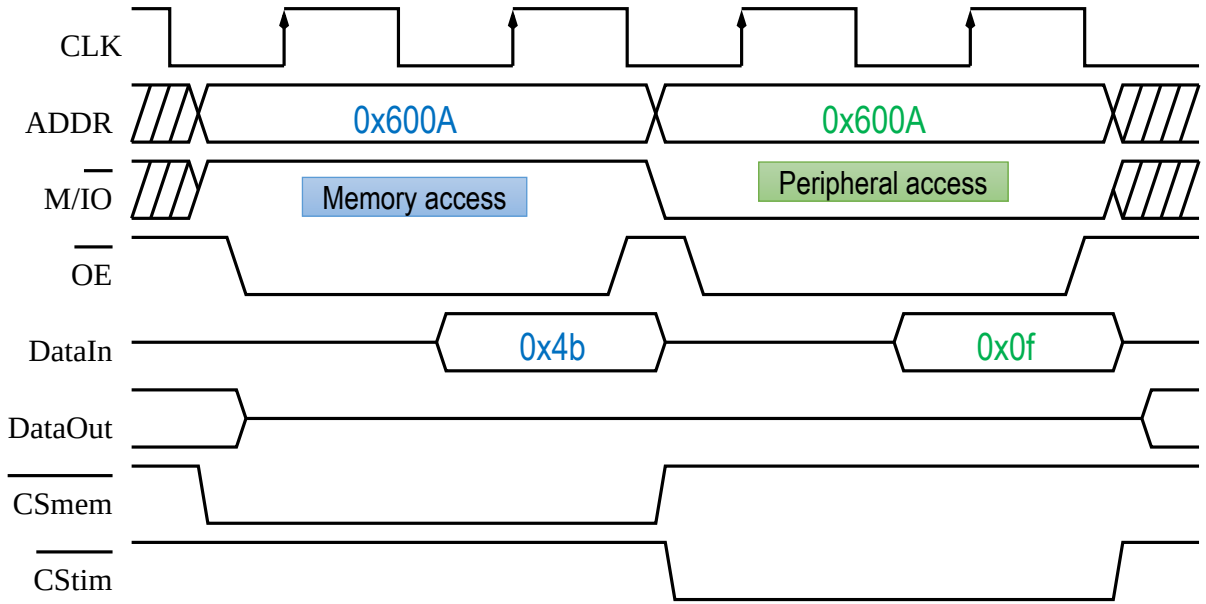
- Timing diagrams
  - SoC blocks
    - Processor core
    - Internal SRAM (32 Kbytes)
    - Internal E2PROM (16 Kbytes)
    - GPIO (0x0000)
    - SPI Master (0x2000)
    - I2C Master (0x4000)
    - Timer



Anatomy of a peripheral

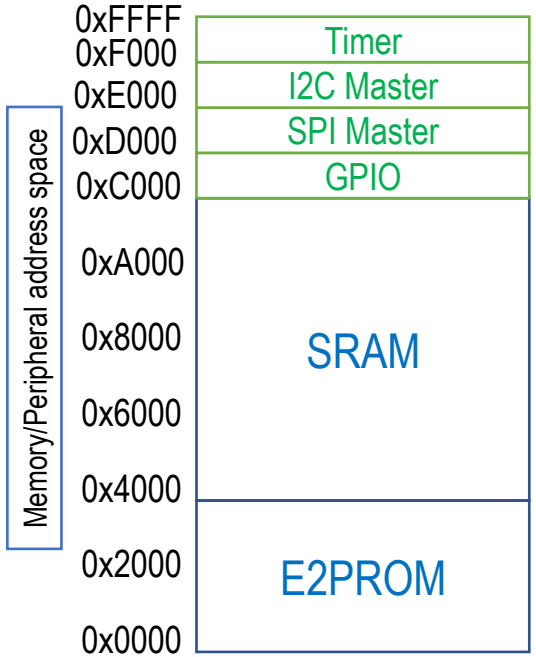
- I/O Mapping timing diagram
  - Same addresses for peripherals and memories
    - Memory and peripheral address spaces can overlap
    - Distinguish using $M/\overline{IO}$ signal
  - First access, memory
    - $M/\overline{IO} = 1$
    - Read location 0x200A of the SRAM
      - $0x600A - 0x4000 = 0x200A$
  - Second access, peripheral
    - $M/\overline{IO} = 0$
    - Read the 10th register of the Timer peripheral
      - $0x600A - 0x6000 = 0xA$

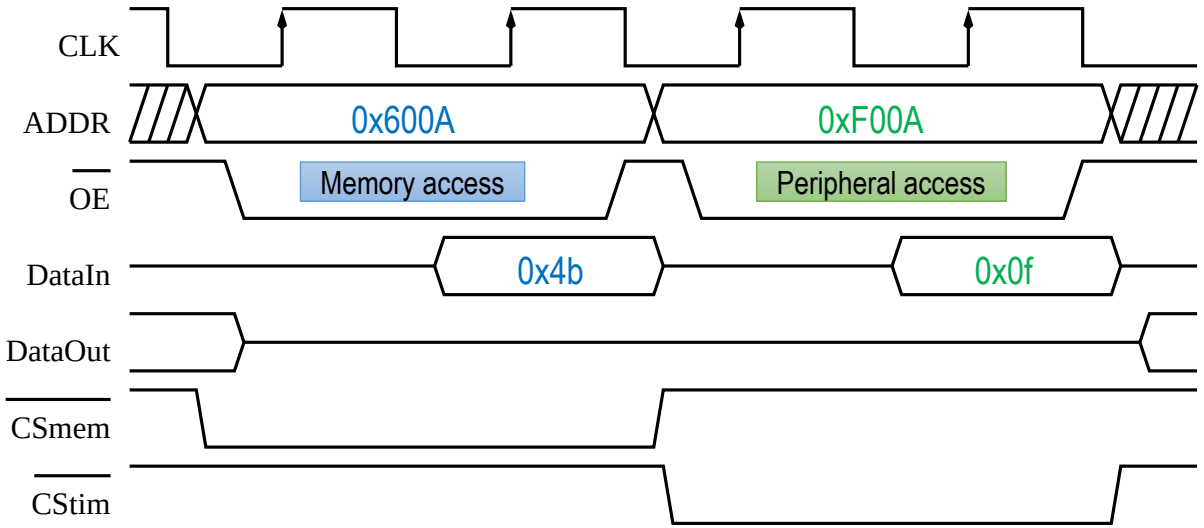




Anatomy of a peripheral

- Memory Mapping timing diagram
  - Different addresses for peripherals and memories
    - Memory and peripheral address spaces cannot overlap
  - First access, memory
    - Read location 0x200A of the SRAM
      - $0x600A - 0x4000 = 0x200A$
  - Second access, peripheral
    - Read the 10th register of the Timer peripheral
      - $0xF00A - 0xF000 = 0xA$





Anatomy of a peripheral

- Processor access to registers
  - Each register access is a load or store operation
  - Sequentially read or write peripheral registers
    - Similar to reading and writing a memory
      - Cannot access two or more registers concurrently
    - The peripheral hardware has parallel access
  - Example of read and write in memory mapping, using C code
    - Use -mno-cache-volatile in gcc flags

```
/* Write the value 5 in a peripheral register */
volatile int * p;
p = (int *) 0x8000f560;
*p = 5;
```

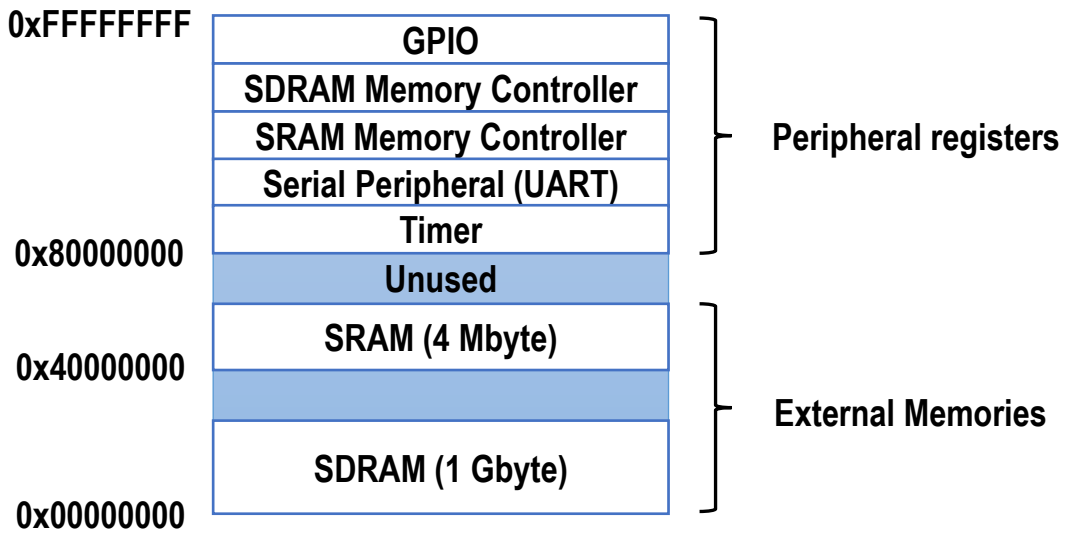
```
/* Read the value of a peripheral register */
volatile int * p;
int val;
p = (int *) 0x8000f560;
val = *p;
```

Anatomy of a peripheral

- Memory map
  - List of all address ranges for memories and peripherals in a processor addressable space
    - Base Address + Length
    - Base Address + End address
  - Memories
    - Usually at low addresses (i.e. starting from 0)
    - Large address segments (Mbytes to Gbytes)
  - Peripherals
    - Usually at higher addresses (in memory mapping)
    - Small address segments (Kbytes)

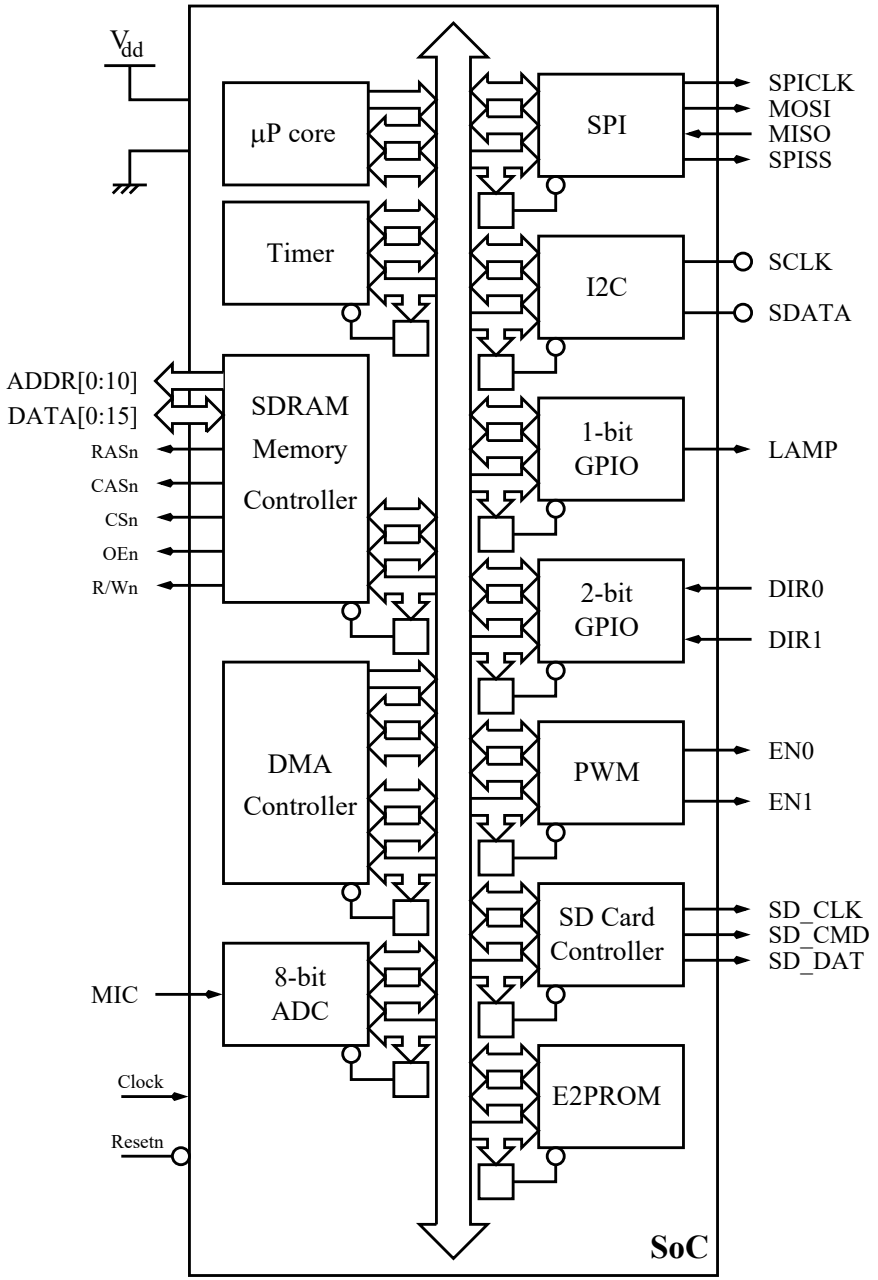
Anatomy of a peripheral

- Memory map (continued)
  - The processor addressable space is generally much larger than needed in most typical embedded systems
    - 32 address lines → 4 Gbytes
  - Large parts of the addressable space are left unused



System on Chip (SoC)

- Block diagram of a SoC
  - Processor core has no external pins on the SoC
    - Also DMA Controller, E²PROM and Timer
  - Processor core and DMA are the only masters on the On-Chip Bus
    - DMA is also a slave
      - Two separate ports on the bus
  - Slave interfaces include address decoders
    - They drive the peripheral Chip Select signals of the respective bus interfaces
  - Clock and Resetn go to all blocks in the system
    - Also V_{dd} and GND

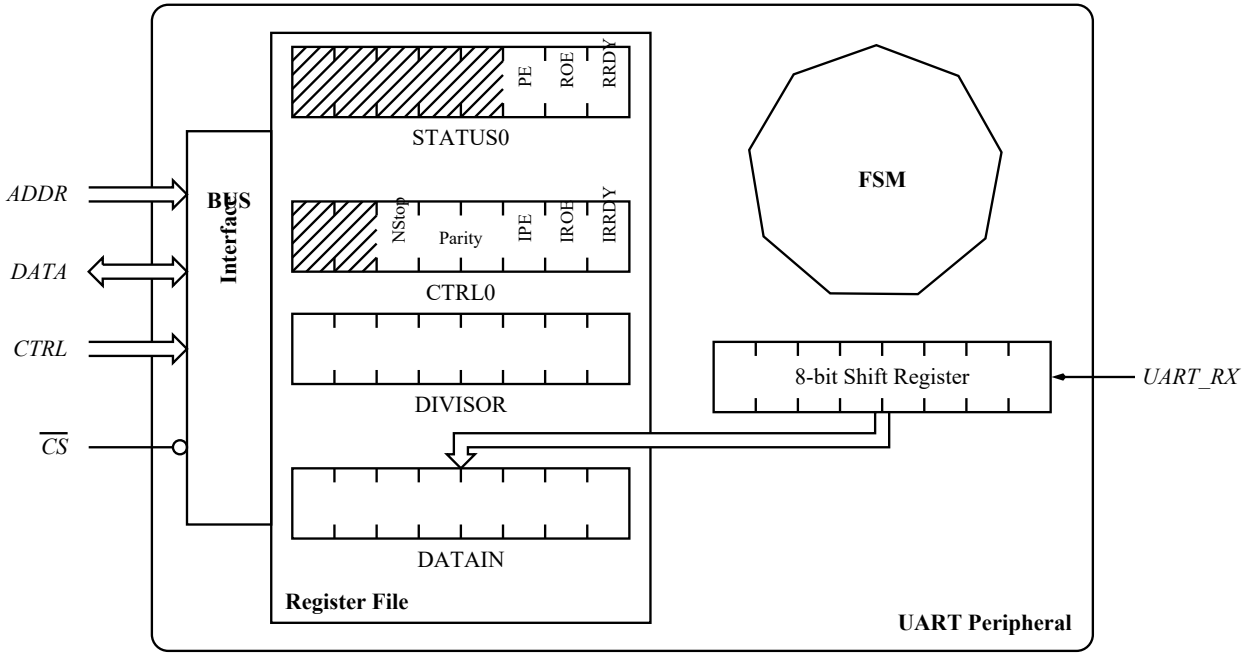


System on Chip (SoC)

- Communication architectures
  - Processor is a master on the On-Chip Bus, peripheral is a slave (except DMAs)
    - Master
      - Can generate addresses
      - Determines the kind of operation (read, write, …)
  - How can a peripheral request attention to the processor?
    - Polling (high latency, low performance)
      - Might loose some data
      - Not real time
    - Interrupt (medium latency, medium performance)
      - Need hardware in the processor
    - Direct Memory Access (small latency, high performance)
      - Need a special peripheral with master interface on the On-Chip Bus

System on Chip (SoC)

- Communication architectures (continued)
  - Example: UART peripheral receiving data
    - Status registers
      - STATUS0 (bits are cleared by writing a 0)
        - RRDY (bit 0): equal to 1 when data has been received, otherwise 0
        - ROE (bit 1): equal to 1 in case of data overrun
        - PE (bit 2): equal to 1 in case of parity error
    - Control registers
      - CTRL0
        - IRRDY (bit 0): interrupt enable for RRDY
        - IROE (bit 1): interrupt enable for ROE
        - IPE (bit 2): interrupt enable for PE
        - Parity (bit 3 and 4): N, O, E
        - NSTOP (bit 5): 1 or 2
      - DIVISOR
        - Sets the baud rate
    - Data registers
      - DATAIN: received data on 8 bits



System on Chip (SoC)

- Communication architectures (continued)
  - Polling example
    - At initialization the processor
      - Sets CTRL0 to 0 (disables all interrupts, no parity, 1 Stop bit)
      - Sets DIVISOR to an appropriate value
    - During normal operations the processor
      - Periodically reads STATUS0 and checks for RRDY to become 1
      - Possibly checks ROE and PE for errors
    - If RRDY=1 the processor
      - Reads DATAIN
      - Clears STATUS0
    - The program performs other actions while not polling STATUS0

System on Chip (SoC)

- Communication architectures (continued)
  - Polling example

```
/* Macro definitions and variable declarations */
#define UART_BASE 0x80001A80
#define UART_STATUS0_OFFSET 0
#define UART_CTRL0_OFFSET 4
#define UART_DIVISOR_OFFSET 8
#define UART_DATA_OFFSET 12
unsigned char buffer[1000], status;
unsigned volatile void *p;
```

```
/* Peripheral initialization */
p = UART_BASE;
*((int *) (p + UART_CTRL0_OFFSET)) = 0x00;
*((int *) (p + UART_DIVISOR_OFFSET)) = …;
```

```
/* Polling */
for (int i = 0; i < 1000; i++) {
   /* Wait for RRDY to become 1 */
   while ((status = *(p + UART_STATUS0_OFFSET)) & 0x01) == 0) {  };
   /* Read data and clear status register */
   buffer[i] = *(p + UART_DATA_OFFSET);
   *(p + UART_STATUS0_OFFSET) = 0x00;
}
```

System on Chip (SoC)

- Communication architectures (continued)
  - Interrupt example
    - At initialization the processor
      - Sets the bits in CTRL0 to enable the interrupts (some of them, or all of them)
      - Sets DIVISOR to an appropriate value
      - Registers an Interrupt Service Routine (ISR)
    - When a bit in STATUS0 for which the interrupt is enabled is set
      - An interrupt in generated
      - The ISR is called
    - The ISR reads STATUS0 and performs the needed action
      - If RRDY=1, reads DATAIN and clears STATUS0
      - If PE=1 or ROE=1, flags the error condition to user programs
    - When not in the ISR, the processor performs other tasks

System on Chip (SoC)

- Communication architectures (continued)
  - Interrupt example
    - Similar macros and initialization, but now enable interrupts

```
/* Peripheral and IRQ initialization */
p = UART_BASE;
*((int *) (p + UART_CTRL0_OFFSET)) = 0x01;
*((int *) (p + UART_DIVISOR_OFFSET)) = …;
register_isr(uart_isr, UART_IRQ);
```

```
/* User program */
unsigned char buffer[1000];
…
/* Read from UART, blocking
 * While waiting, other processes are free to run
 * on the uP
*/
uart_read(buffer, 1000);
…
```

```
/* Interrupt Service Routine */
#define FIFOSIZE 64
unsigned char fifo[FIFOSIZE];
int next = 0;
void uart_isr() {
   if (*(p + UART_STATUS0_OFFSET) & 0x01 == 1) {
      /* Read data in FIFO */
      fifo[next] = *(p + UART_DATA_OFFSET);
      /* Update FIFO pointer and clear status register */
      next = (next++) & (FIFOSIZE - 1);
      *(p + UART_STATUS0_OFFSET) = 0x00;
   } else {
      /* Handle errors */
   }
}
```

This is the top-half of the interrupt service routine. The top-half should run very quickly as interrupts are disabled during its execution. A bottom-half is also needed to copy the content of the FIFO to the user space buffer and enable the user program thread when the total number of requested item is received. The bottom-half is run later, with high priority but with interrupts enabled. The top-half may run several times before the bottom-half is executed.

System on Chip (SoC)

- Communication architectures (continued)
  - DMA example
    - At initialization the processor
      - Sets the bits in CTRL0 to enable the interrupts (some of them, or all of them)
      - Sets DIVISOR to an appropriate value
    - To perform a transfer of a block of data, the processor
      - Initializes the DMA controller
        - SRCADDR = UART_BASE + DATAIN_OFFSET, SRCINCR = 0
        - DESTADDR = buffer, DESTINCR = 1
        - NELEM = 1000
      - Starts the DMA transfer
    - When DMA finishes transferring all elements, it generates an interrupt
    - While the DMA transfers are occurring, the processor is free to perform any other task

System on Chip (SoC)

- Communication architectures (continued)
  - How many times the processor accesses the UART peripheral to transfer 1000 elements?
    - Polling: 5000 times
      - Assuming the peripheral is checked 5 times more frequently than necessary
      - Risk of loosing data, high latency
    - Interrupt: 1000 times
      - Once for each element, with lower latency than polling
    - DMA: 2 times
      - Once to initialize the DMA controller and UART peripheral
      - Once to serve the DMA final interrupt

Peripheral examples

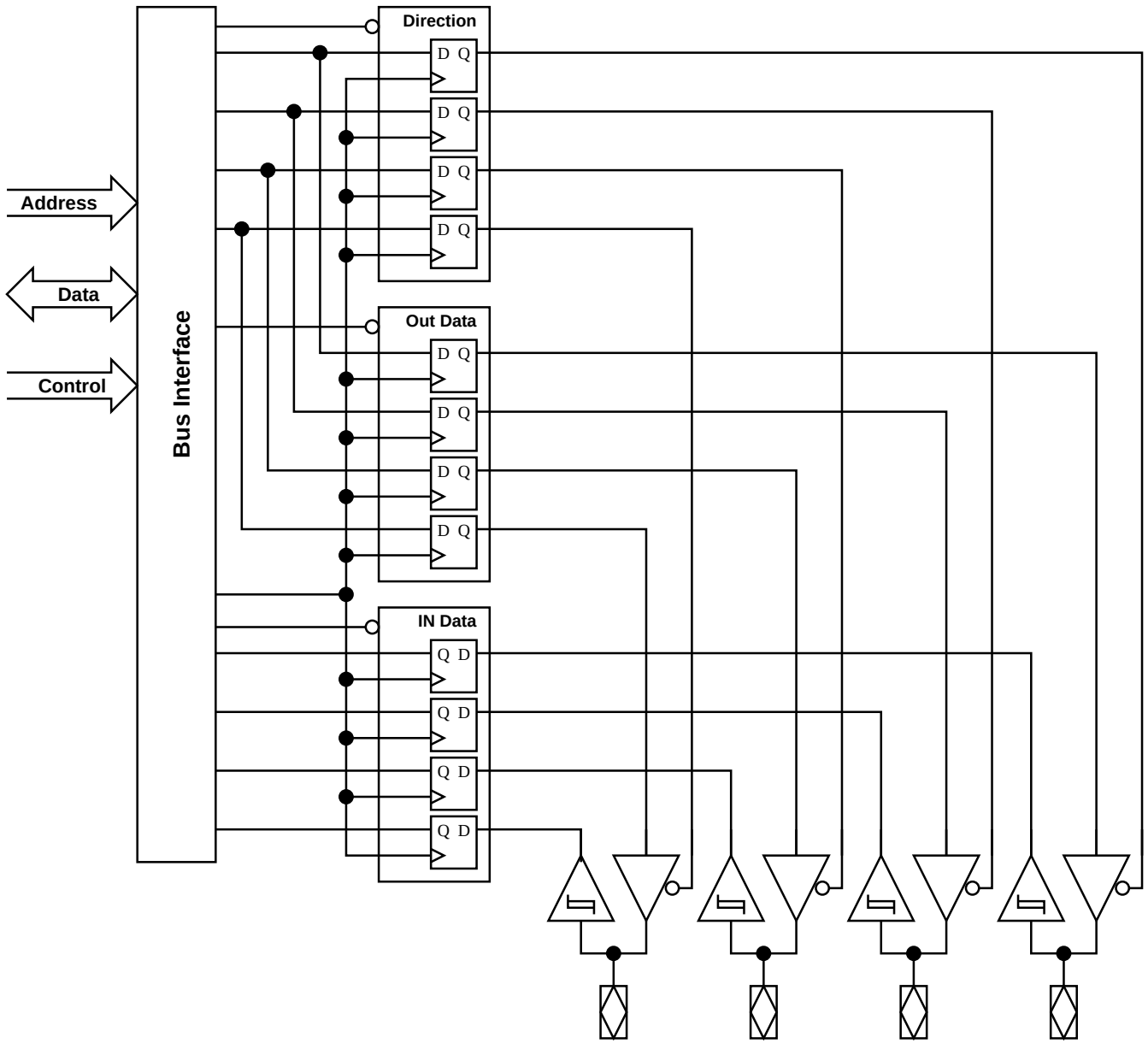
- GPIO (General Purpose Input Output), PIO (Parallel Input Output)
- Timer and Counter
  - With a clock source (measures time)
  - With a signal source (measures number of events)
  - Watchdog timer
- Pulse Width Modulator
  - With a low pass filter to obtain a DA converter
- UART
- SPI, I^2C
- DMA controller
- Memory controller

Peripheral examples

- GPIO (General Purpose Input Output)
  - Controls or checks the logic state of a set of pins
    - Used to switch on LEDs, control electronic switches (relays), interface the SoC to buttons and user-selectable switches, …
    - Found in all embedded processors, often sharing pins with other functions
    - Also knows as *Ports* (A port, B port, …)
  - Control registers
    - Direction (for bidirectional GPIOs)
    - Edge Detection
    - Interrupt generation
  - Status registers
    - Edge detected
  - Data registers
    - Pin value

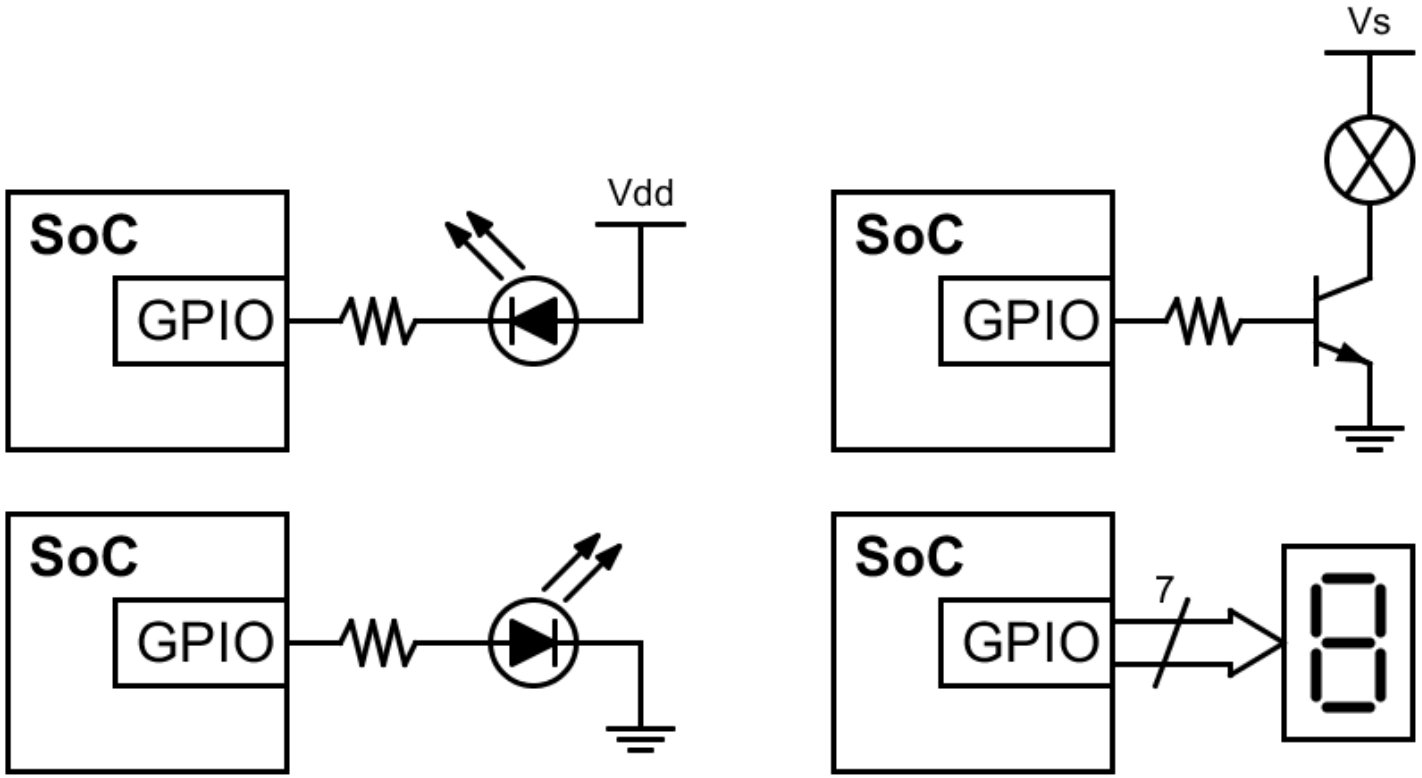
Peripheral examples

- GPIO (General Purpose Input Output)
  - 4-bit GPIO
  - Output buffers with enable
    - Signal from Out Data register
    - Enable driven by Direction register
      - 0: output
      - 1: input
  - Input buffers with hysteresis
  - Not shown
    - Edge detection
    - Drive strength control
    - Pull-up resistors
    - Protection
    - Interrupt generator



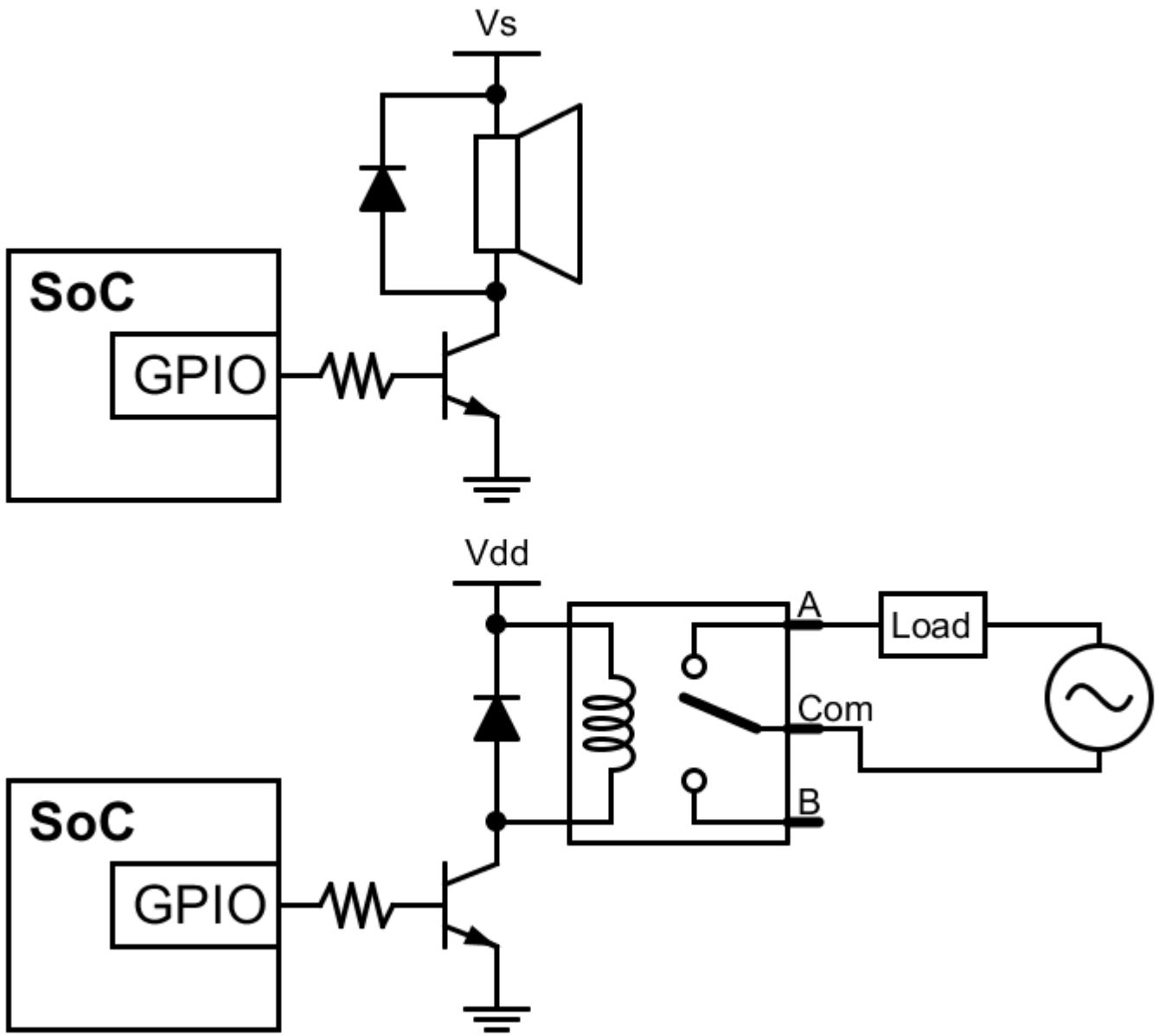
Peripheral examples

- GPIO (General Purpose Input Output)
  - Examples of output GPIOs
    - Driving an LED
      - Active low
      - Active high
    - Driving a lamp
      - Different voltage source
      - Use a transistor
    - Driving multiple LEDs
      - 7-Seg Display



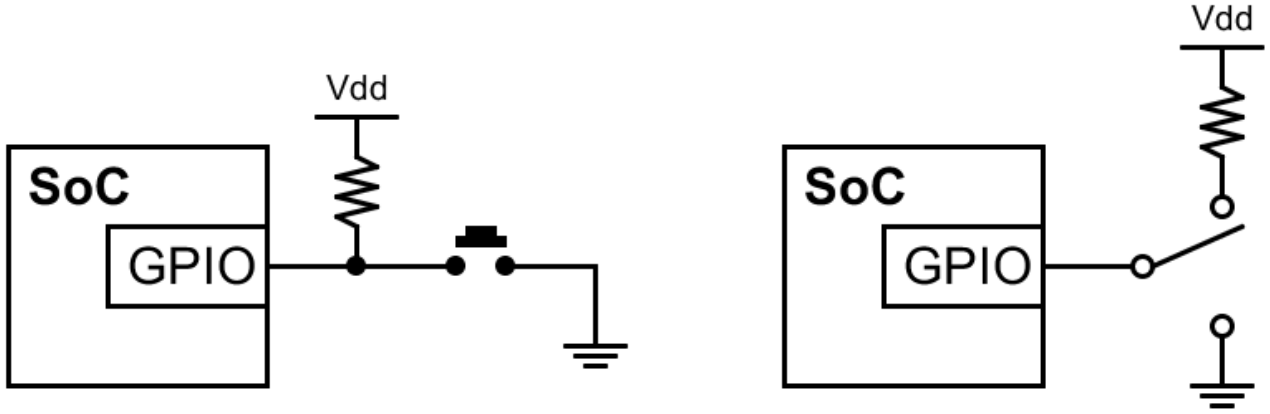
Peripheral examples

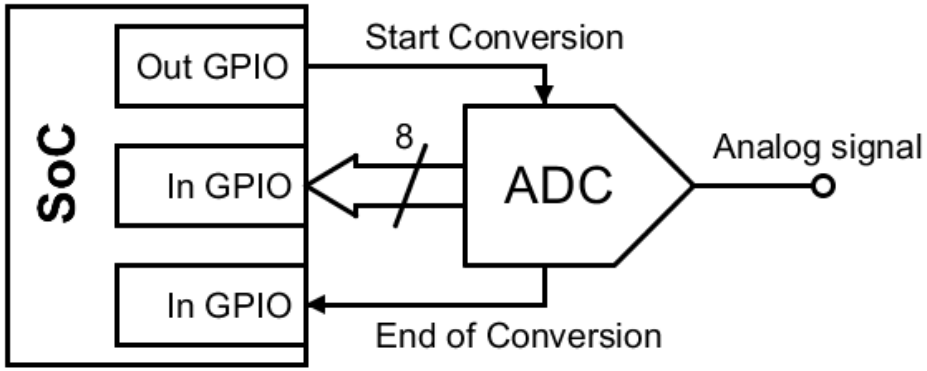
- GPIO (General Purpose Input Output)
  - Examples of output GPIOs
    - Driving a magnetic buzzer
      - Recirculating diode
        - Inductive load
      - For a piezo-electric buzzer use a resistance in place of the diode
    - Driving a relay
      - Recirculating diode
      - Alternate current voltage source on the load
        - Ex: 230 V, 50 Hz



Peripheral examples

- GPIO (General Purpose Input Output)
  - Examples of input GPIOs
    - Reading the status of a button
      - Normally high
      - Goes low when depressed
    - Reading the status of a switch
      - Two positions
    - Reading an Analog to Digital Converter
      - 1-bit output GPIO to start conversion
      - 1-bit input GPIO to detect the end of a conversion
      - 8-bit input GPIO to read the converted data



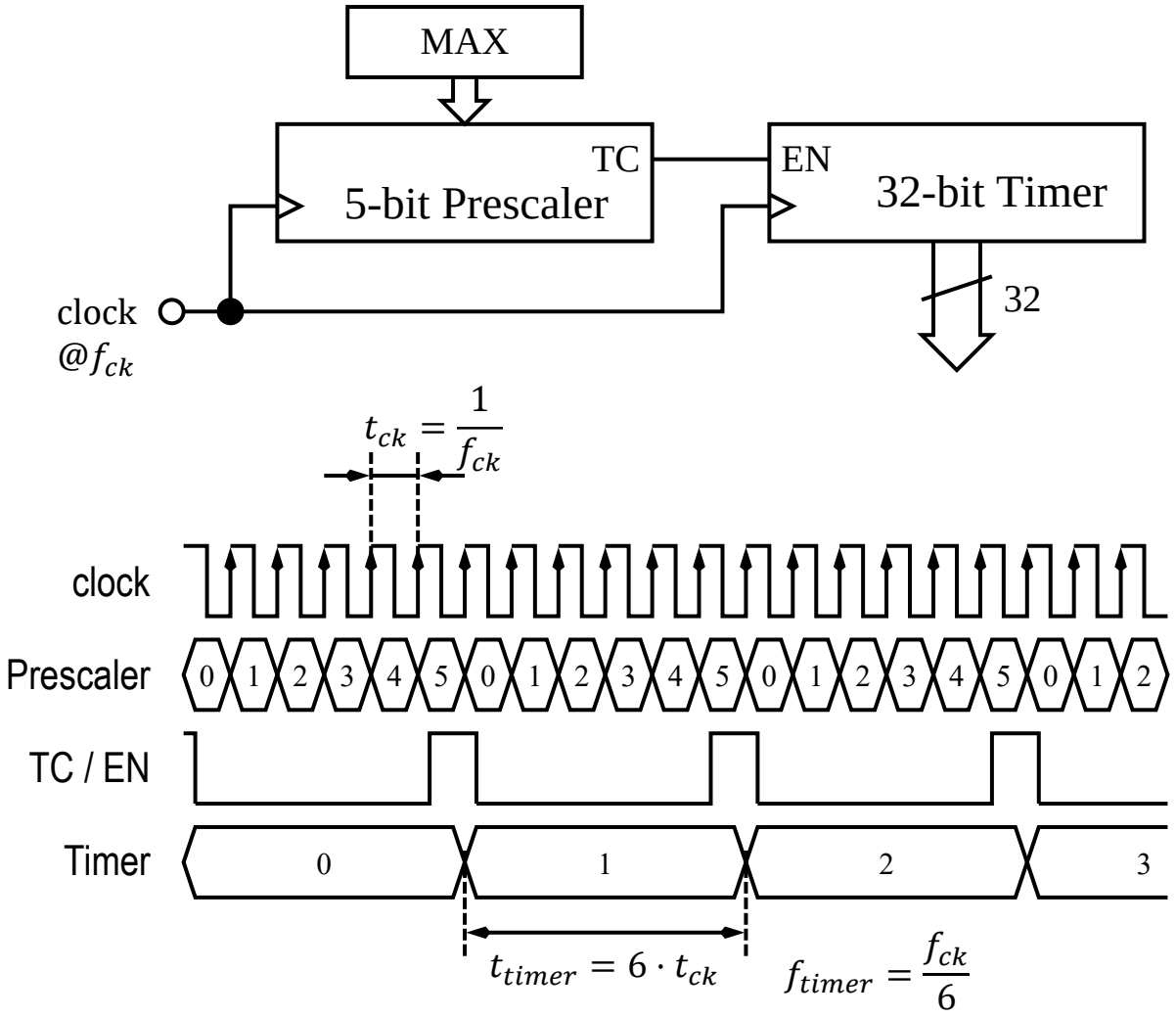


Peripheral examples

- Timer
  - Used to count clock ticks
    - Same processor clock, fed to a counter
    - Number of bits of the counter and clock frequency determine
      - The resolution (minimum measurable interval)
      - The range (maximum measurable interval)
      - $\text{mininterval} = {}^{1}/_{f_{ck}}, \quad f_{ck} = 20\ \text{MHz} \rightarrow \text{mininterval} = 50\ \text{ns}$
      - $\text{maxinterval} = {}^{2^n}/_{f_{ck}}, \quad n = 32, \quad f_{ck} = 20\ \text{MHz} \rightarrow \text{maxinterval} = 214.7\ \text{s}$
  - Control registers
    - Start, Stop, Clear, Up/Down, Restart automatically
    - Max value, Generate interrupt at max (or 0)
  - Status registers
    - Running/Idle, End reached
  - Data registers
    - Current counter snapshot

Peripheral examples

- Timer (continued)
  - How to adjust resolution and maximum measurable interval
    - Use a pre-scaler (hardware approach)
      - Divides the processor clock frequency by an arbitrary integer value
        - Not necessarily a power of 2
      - Terminal Count (TC) of pre-scaler drives the enable of the counter
      - Number of bits of pre-scaler determines the maximum divisor
      - Trade-off between resolution and maximum measurable interval



Peripheral examples

- Timer (continued)
  - How to adjust resolution and maximum measurable interval
    - Use a software program (software approach)
      - Use the timer for *fine* ticks
      - Accumulate *coarse* ticks on a program variable
        - Integer in C is often on 32 bit
        - Multiple variables (e.g. milliseconds, seconds, minutes, hours) to expand the measurable interval
      - May be more flexible than a hardware approach
      - Additional load on the processor to manage the variables

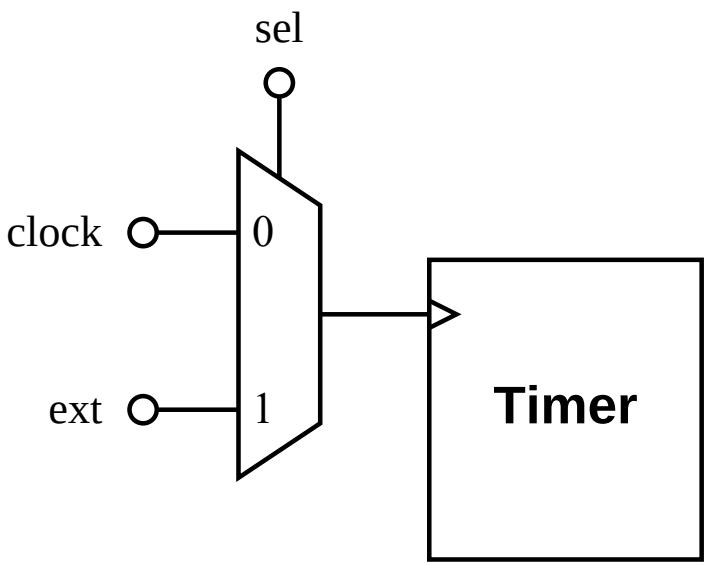
```c
/* Time variable */
int valms, vals, valm, valh;

/* 1ms ISR */
void timer1ms (void) {
   restart_timer();
   valms++;
   if (valms == 1000) {
      valms = 0;
      vals++;
      if (vals == 60) {
         vals = 0;
         valm++;
         if (valm == 60) {
            valm = 0;
            valh++;
         }
      }
   }
}
```

Peripheral examples

- Timer (continued)
  - Periodic interrupts
    - A timer can be used to periodically generate an interrupt
      - Extremely useful in many embedded systems
      - Used to meet real-time constraints
      - Used in embedded operating systems to control the scheduling of processes
    - The interrupt service routine should be quick
      - Prepare most of the work in other functions

Peripheral examples

- Counter
  - Similar to a timer, but counts events (not clock ticks)
    - Usually events are transitions on an external pin
    - Example
      - Number of turns of the wheel of a bicycle
      - If wheel radius is known, can compute the total distance
    - If a timer and a counter are combined
      - Compute speed (distance over time)
      - Compute frequency (number of events over time)
    - Peripheral registers similar to timer
  - Same peripheral as timer
    - Configure the input source

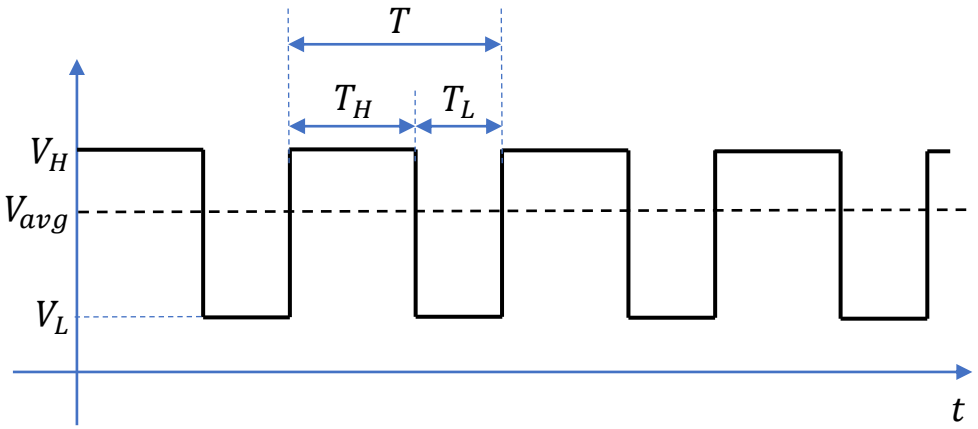


Peripheral examples

- Watchdog Timer
  - A down counter, initialized to some known value
    - Clock signal as input, measures time
    - Example: initialize to count 5 seconds
  - When it reaches 0, it generates a non-maskable reset to the system
    - Software shall reload the initial value before the deadline expires
  - Used to restart a system
    - In case it hangs up due to an error (bug)
    - In case it hangs up due to transient failure
    - Useful to recover when no user intervention is possible

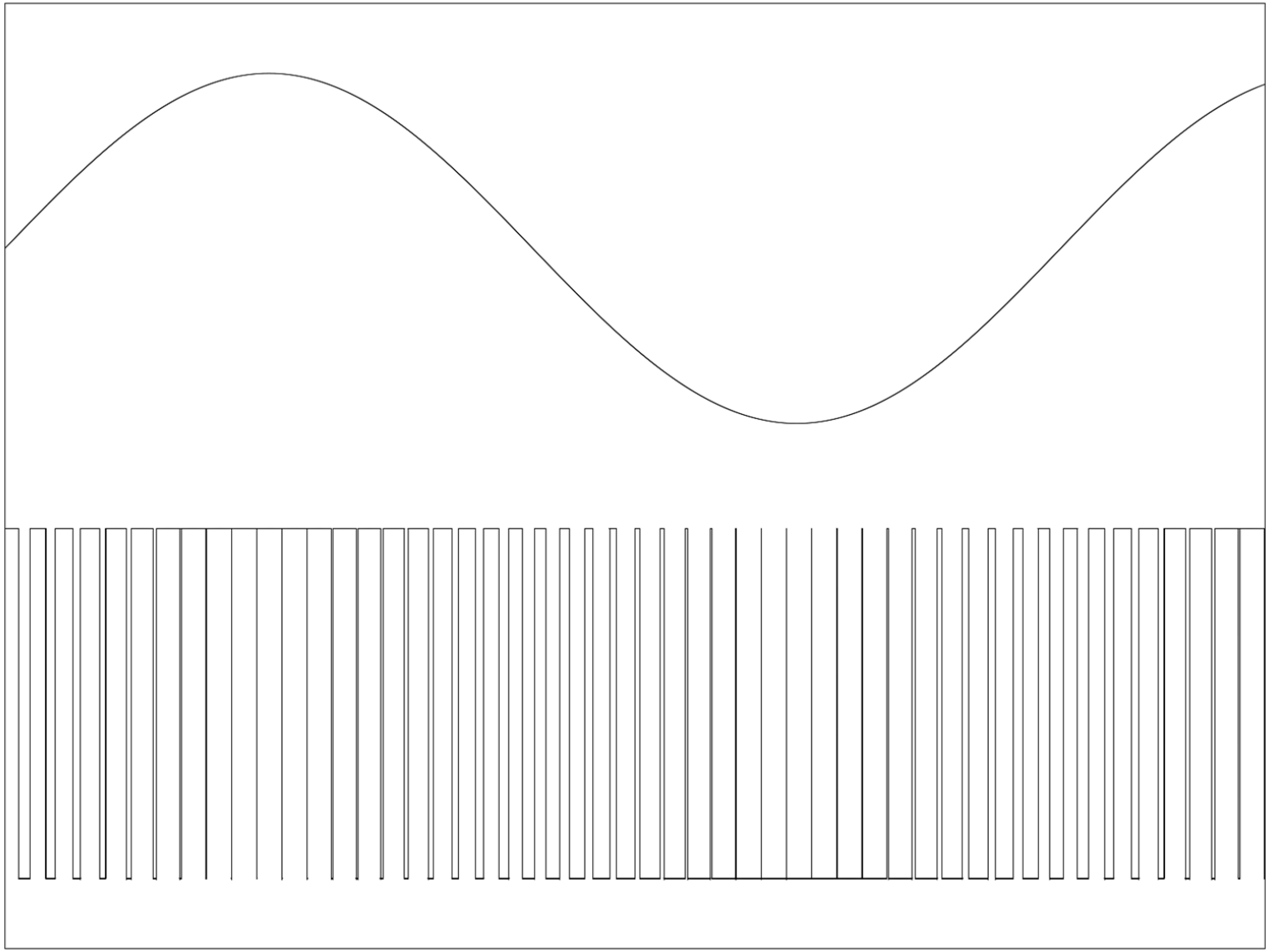
Peripheral examples

- Pulse Width Modulator (PWM)
  - The average value of a square wave depends on
    - The two voltage levels V_H and V_L
    - The duty cycle δ
  - Assume the square wave has frequency f
    - $f = \frac{1}{T},\ T = T_H + T_L,\ \delta = \frac{T_H}{T}$
    - $V_{avg} = \frac{T_H}{T} V_H + \frac{T_L}{T} V_L = \delta V_H + (1 - \delta) V_L$
    - If $V_L = 0$ → $V_{avg} = \delta V_H$
  - If δ changes slowly compared to $f = 1/T$
    - Use a filter to extract the waveform of the average value of the PWM square wave
    - The average value follows the modulation signal of the duty cycle δ



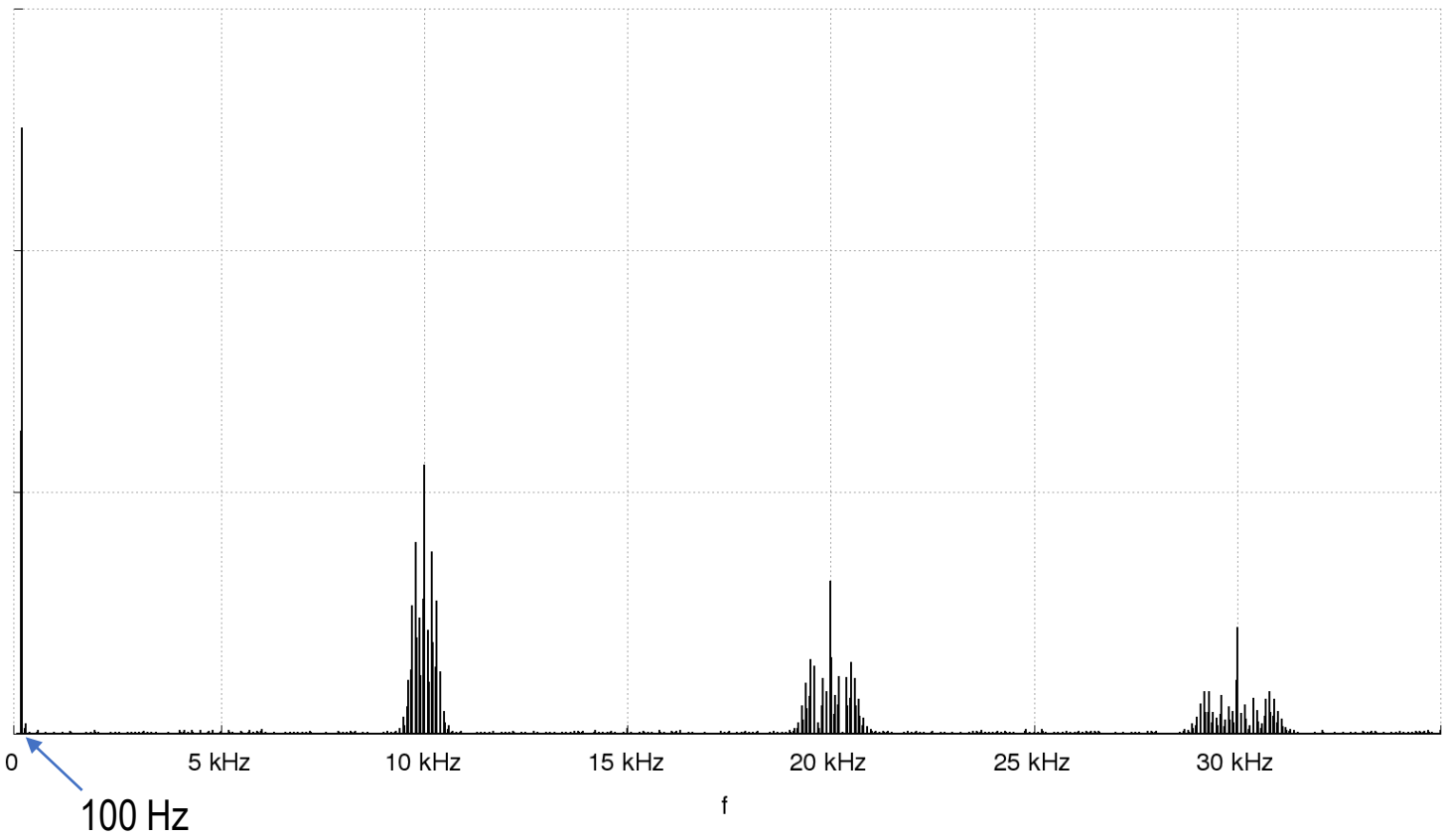
Peripheral examples

- Pulse Width Modulator (PWM)
  - Sin wave
    - $f_{PWM} = 42 f_{sin}$
      - Usually at least two decades to make the filter cheap
    - Filtering the bottom square wave gives the top sinusoid
      - Plus some remaining high frequency noise



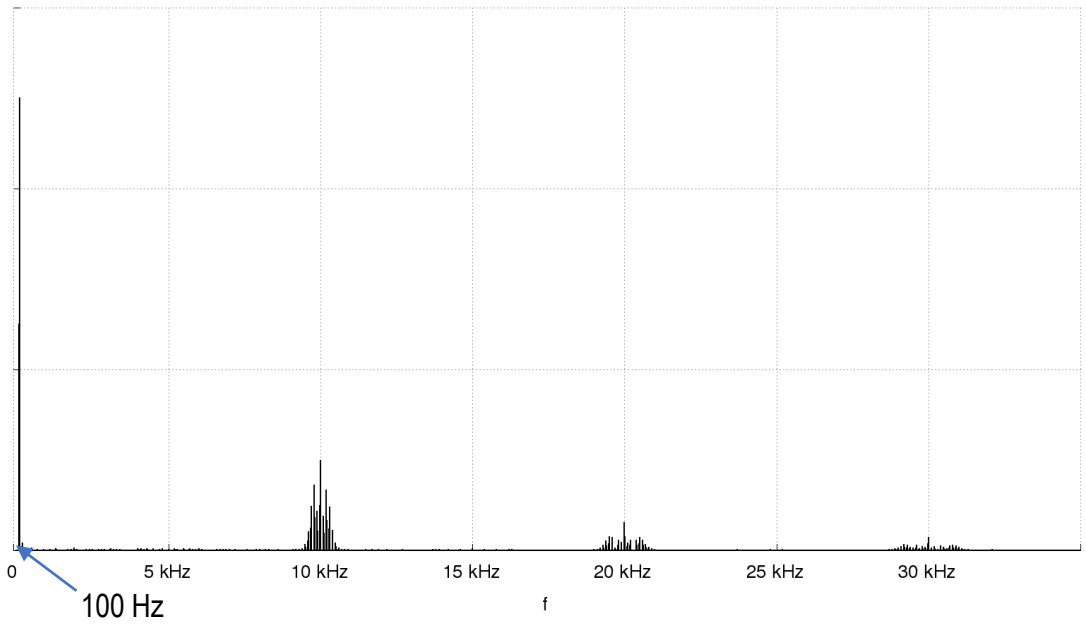
Peripheral examples

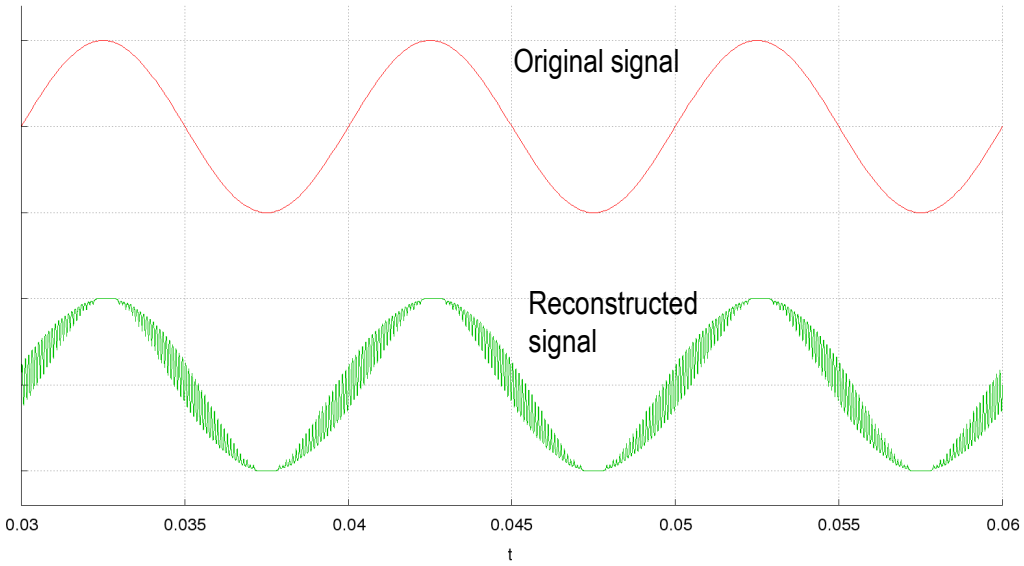
- Pulse Width Modulator (PWM)
  - Sin wave
    - $f_{sin} = 100\ \mathrm{Hz}$
    - $f_{PWM} = 100 f_{sin} = 10\ \mathrm{kHz}$
    - Spectrum of the PWM signal
      - Line at 100 Hz for the sin wave
        - Modulating signal
      - Contribution at 10 kHz
        - Fundamental frequency of the square wave
      - Contributions at 20 kHz, 30 kHz, …
        - Harmonics of the square wave



Peripheral examples

- Pulse Width Modulator (PWM)
  - Sin wave
    - $f_{sin} = 100\ \mathrm{Hz}$
    - $f_{PWM} = 100 f_{sin} = 10\ \mathrm{kHz}$
    - Spectrum of the PWM signal
      - Filtered with cut-off $f_C = 5$ kHz
      - Single pole (20dB/dec, 6dB/oct)
        - The component at $f = 10$ kHz is attenuated by 2
        - The component at $f = 20$ kHz is attenuated by 4
    - The reconstructed signal shows significant residual high frequency noise
      - The filter should attenuate more
        - Cut-off frequency can be less



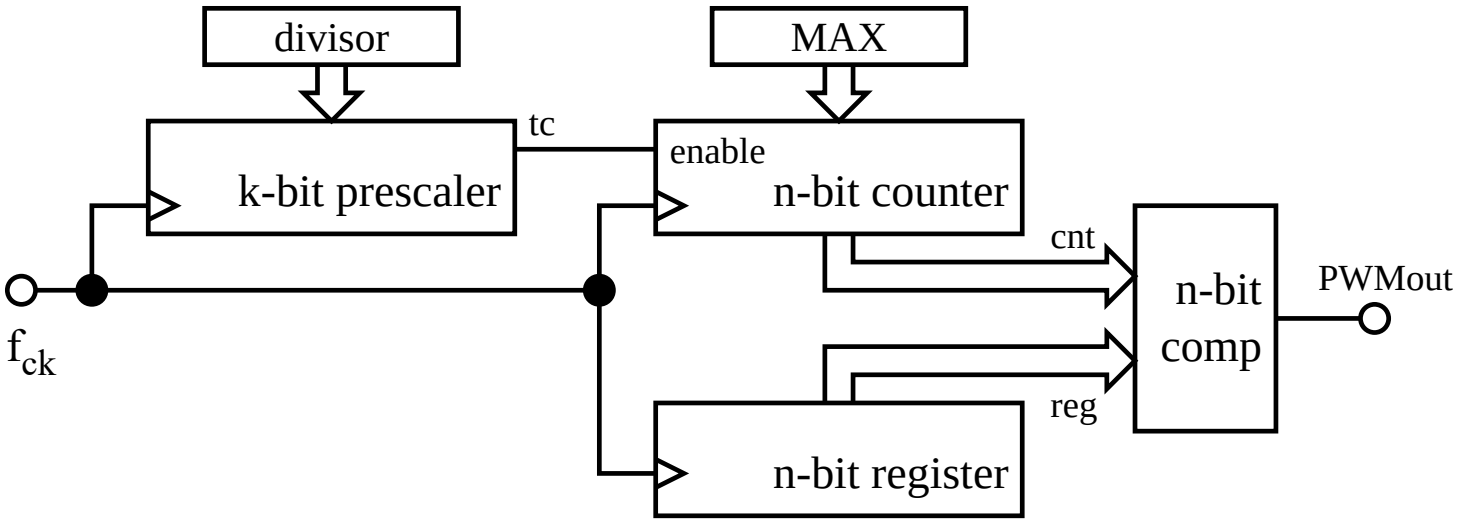


Peripheral examples

- Pulse Width Modulator (PWM)
  - A PWM peripheral generates a high frequency square wave, with a controllable duty cycle
    - Control registers
      - Square wave frequency
        - Divisor
        - MAX counter value
      - Start, Stop, Reset, Generate interrupt each cycle
    - Status registers
      - Current count value
      - Running/Idle
    - Data registers
      - Value of the duty cycle (register *reg*, see next slide)

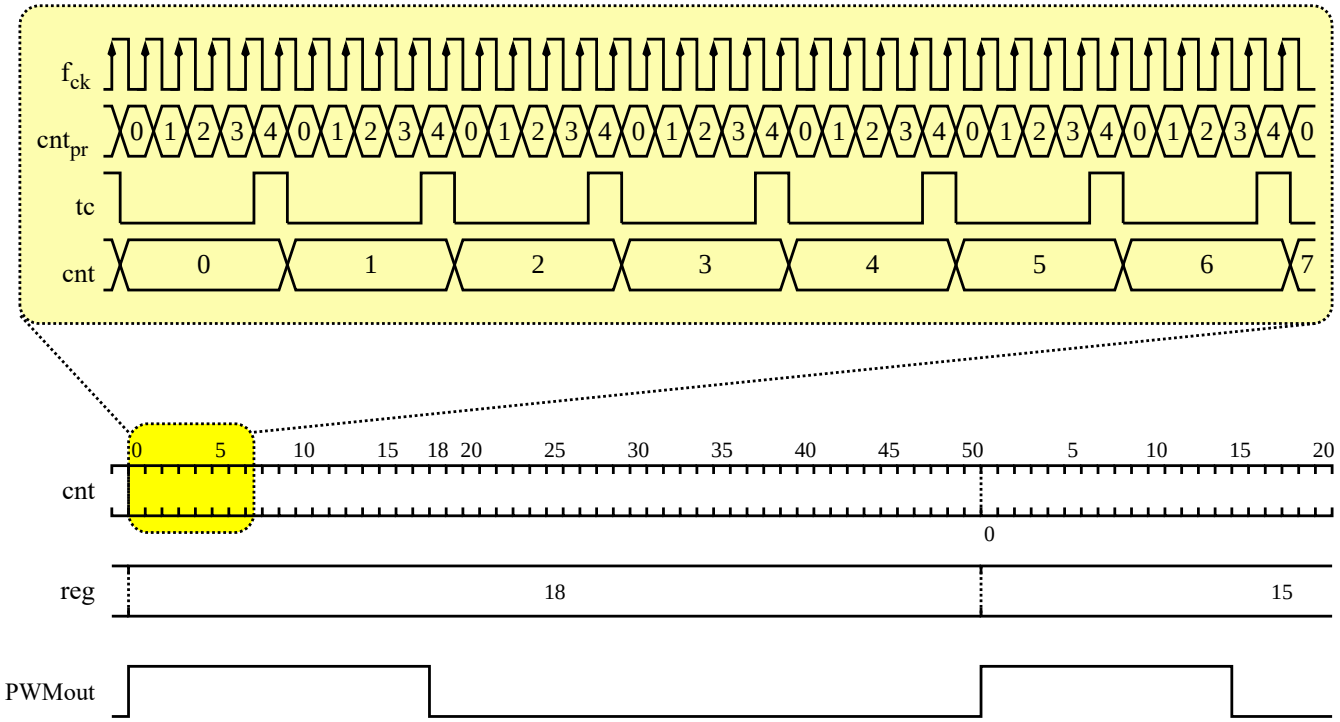
Peripheral examples

- Pulse Width Modulator (PWM)
  - Made of a counter, a register and a comparator
    - Counter counts up to a configurable MAX value
      - Then rolls back to 0 and keeps counting
      - A pre-scaler used to change the counter frequency
      - Scaling set by an integer divisor, starting from the processor clock frequency f_{ck}
    - Register holds a configurable value *reg* between 0 and MAX
      - When *cnt* < *reg* → PWMout = 1
      - When *cnt* ≥ *reg* → PWMout = 0
    - Duty cycle of the output waveform
      - $\delta = \frac{reg}{MAX+1}$



Peripheral examples

- Pulse Width Modulator (PWM)
  - Example
    - divisor = 4, MAX = 50, reg_1 = 18, reg_2 = 15, f_{ck} = 50MHz
    - Register content changed when the counter rolls back to 0
      - reg_1 = 18, reg_2 = 15
    - The *equivalent* input clock frequency to the counter is $f_{cnt} = f_{ck}/(\text{divisor} + 1)$
      - Obtained by enabling the counter only once every divisor+1 clock cycles
      - $f_{cnt} = 10\ \text{MHz}$
    - $\delta_1 = \frac{18}{51} = 35.29\%$
    - $\delta_2 = \frac{15}{51} = 29.41\%$
    - $f_{pwm} = \frac{f_{cnt}}{MAX+1} = 196\ \text{kHz}$

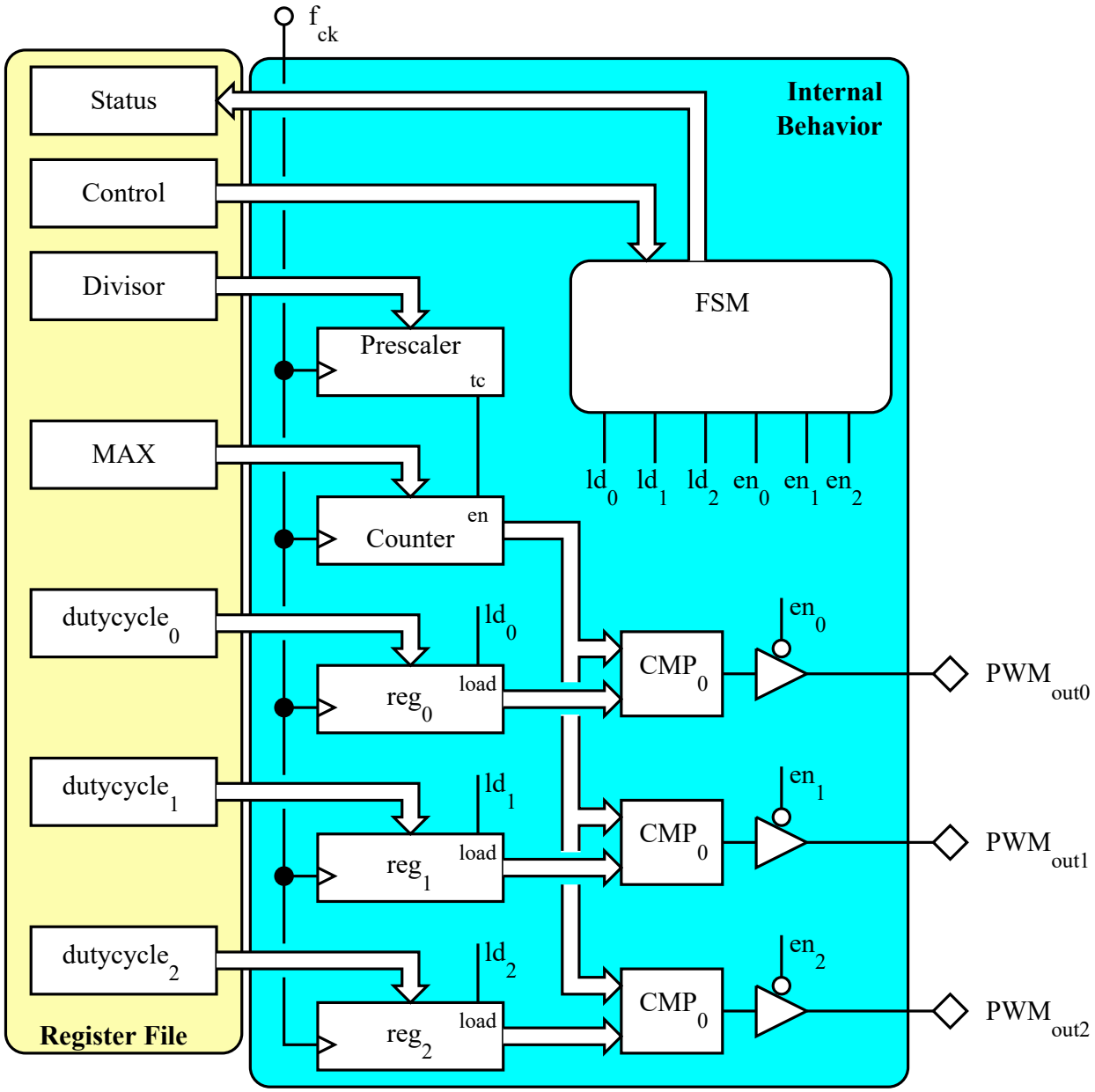


Peripheral examples

- Pulse Width Modulator (PWM)
  - Given a SoC with f_{ck}=50 MHz, and a PWM peripheral with 16 bit pre-scaler and 16 bit counter, generate a PWM signal with f_{pwm}=100kHz, and a duty cycle accuracy of 1%
    - Accuracy → MAX = 99
    - $DIVISOR = \frac{f_{ck}}{(MAX+1) * f_{pwm}} - 1 = \frac{50000000}{100*100000} - 1 = 4$
  - New requirement: duty cycle accuracy of 0.1%
    - MAX = 999
    - DIVISOR = -0.5 <u>Cannot be implemented!</u>
    - Maximum accuracy with f_{pwm}=100kHz is 0.2%
      - MAX = 499, divisor = 0

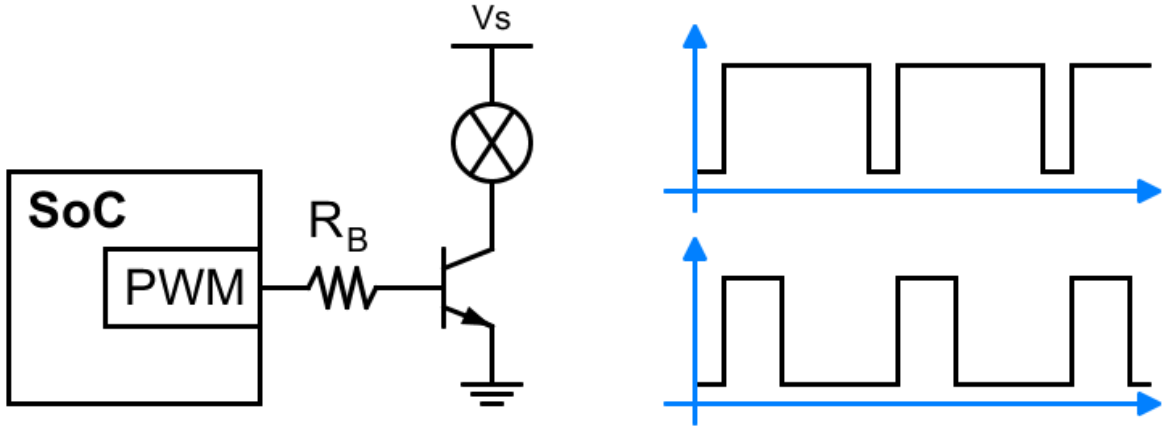
Peripheral examples

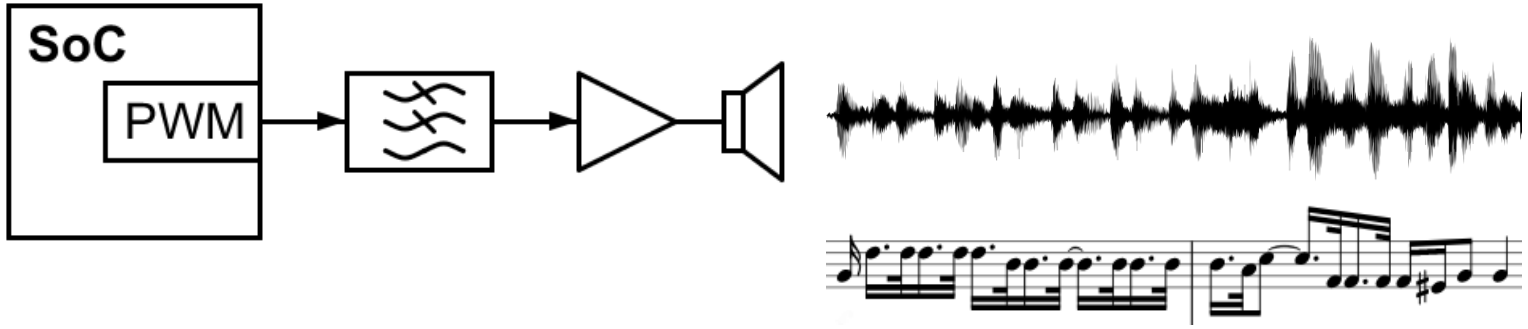
- Pulse Width Modulator (PWM)
  - A PWM peripheral may have
    - Single counter and single prescaler
    - Multiple registers and comparators
    - Multiple outputs
      - All outputs have the same switching frequency
      - The low frequency PWM_{out} waveforms differ between outputs
      - Each output can be individually enabled with a control register
      - The internal FSM reads the control register and enables output buffers
    - A single interrupt used to update all waveforms
      - Internal registers updated by the FSM only when the counter rolls back
      - Can update duty cycle registers in the register file at any time



Peripheral examples

- Pulse Width Modulator (PWM)
  - PWM examples
    - PWM driving a bulb
      - f_{PWM} several kHz
      - High duty cycle
        → bright bulb
      - Low duty cycle
        → faded bulb
    - PWM driving a loudspeaker
      - f_{PWM} several hundreds kHz
      - Modulating signal is a sound wave
        - $f_B = 10\ \mathrm{kHz}$
      - Low pass filter to remove high frequencies



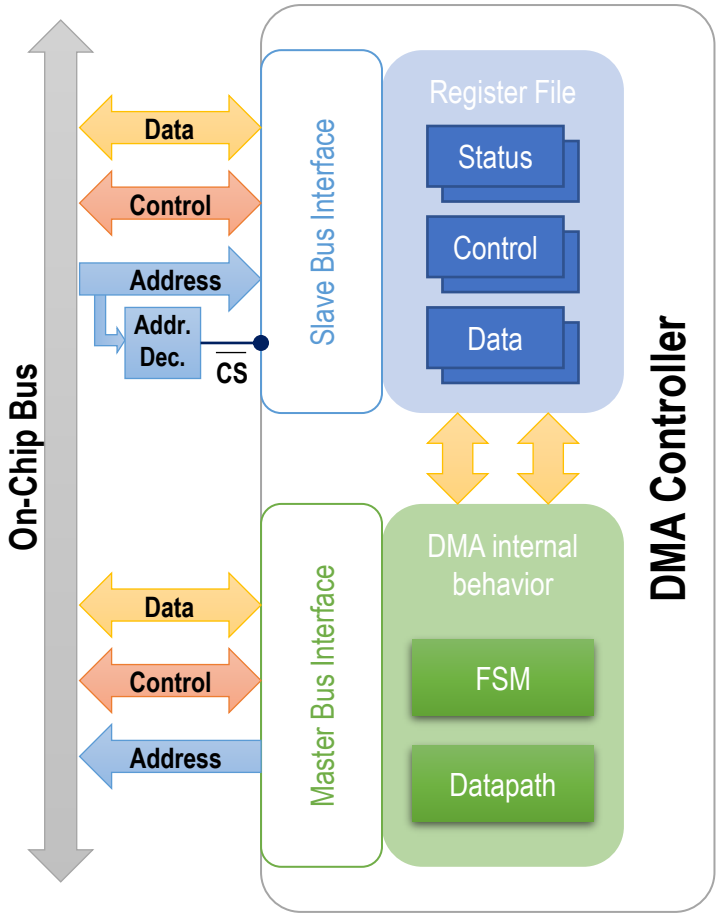


Peripheral examples

- DMA Controller
  - Performs data transfers independently of the microprocessor
    - From Memory to Memory
    - From Memory to Peripheral
    - From Peripheral to Memory
  - Control registers
    - Source Address (SRCADDR) and Source Increment (SRCINCR)
    - Destination Address (DESTADDR) and Destination Increment (DESTINCR)
    - Number of elements (NELEM)
    - Start, Stop, Enable Interrupts
  - Status registers
    - Running/Not running, Transfer completed
    - Number of elements already transferred

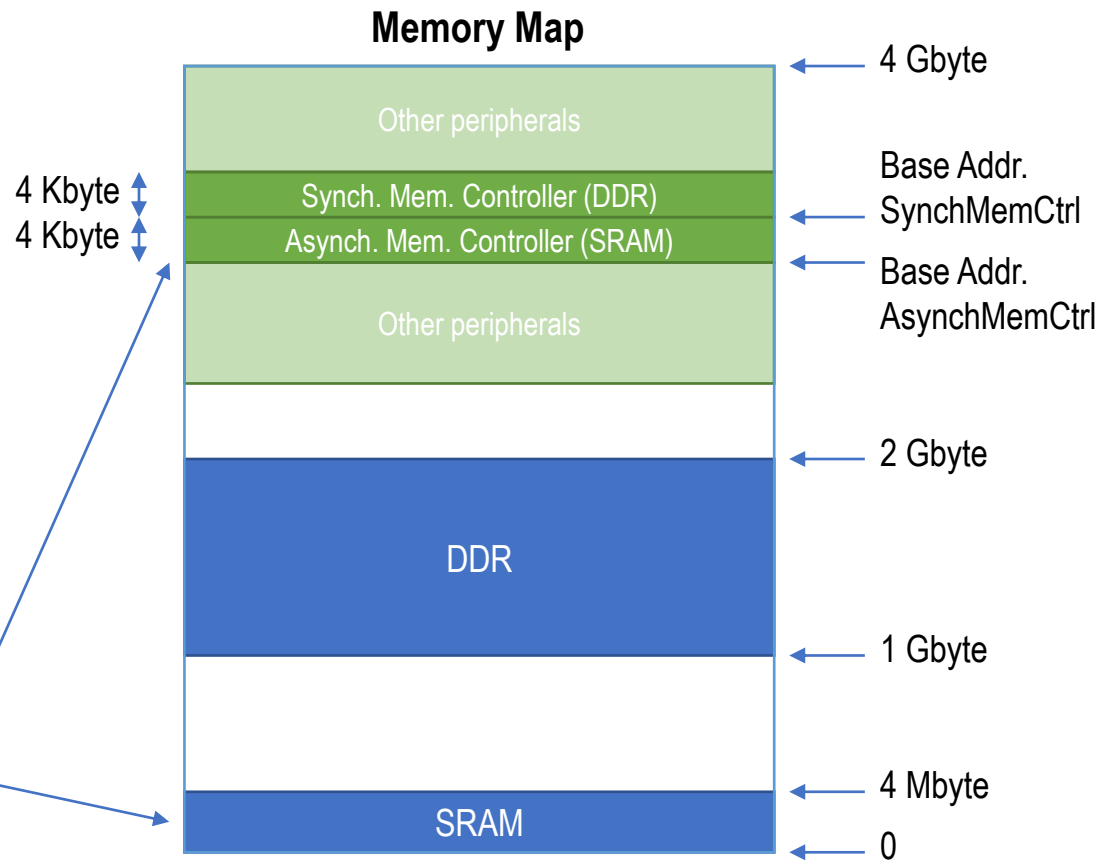
Peripheral examples

- DMA Controller
  - Slave Bus Interface to access the peripheral registers
    - Same as a standard peripheral
    - Used by the microprocessor core to
      - Configure the DMA controller
      - Check its status
  - Master Bus Interface to generate bus transactions
    - Used by the DMA controller internal behavior
    - Competes with other masters for the On-Chip Bus ownership
      - Need arbitration on the On-Chip Bus
      - A DMA often has higher priority than a microprocessor core



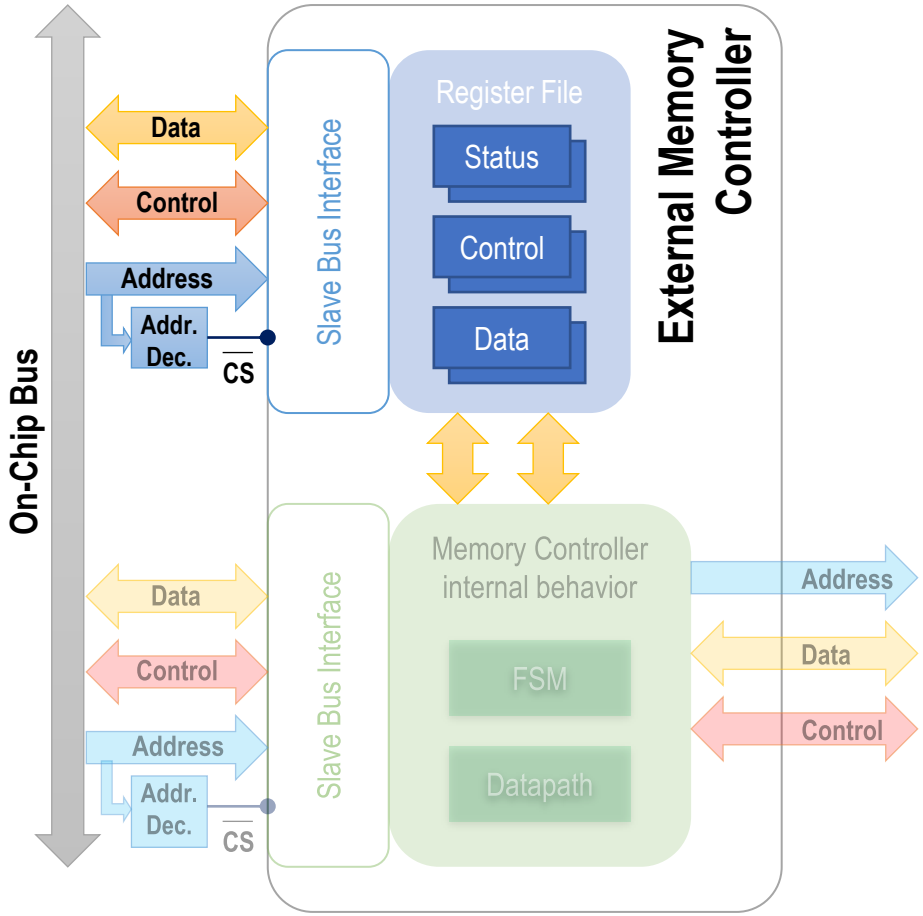
Peripheral examples

- External Memory Controller
  - Allows access to an external memory
    - Asynchronous memories
      - SRAM, E^{2}PROM, Flash
    - Synchronous memories
      - SDRAM, DDR
  - Generates the proper access protocol
    - Different depending on the memory kind
      - See timing diagrams in the memory datasheets
    - Need multiple controllers for different memory kinds
  - Responds to two distinct sets of addresses
    - The address range assigned to the peripheral registers
    - The address range assigned to the external memory



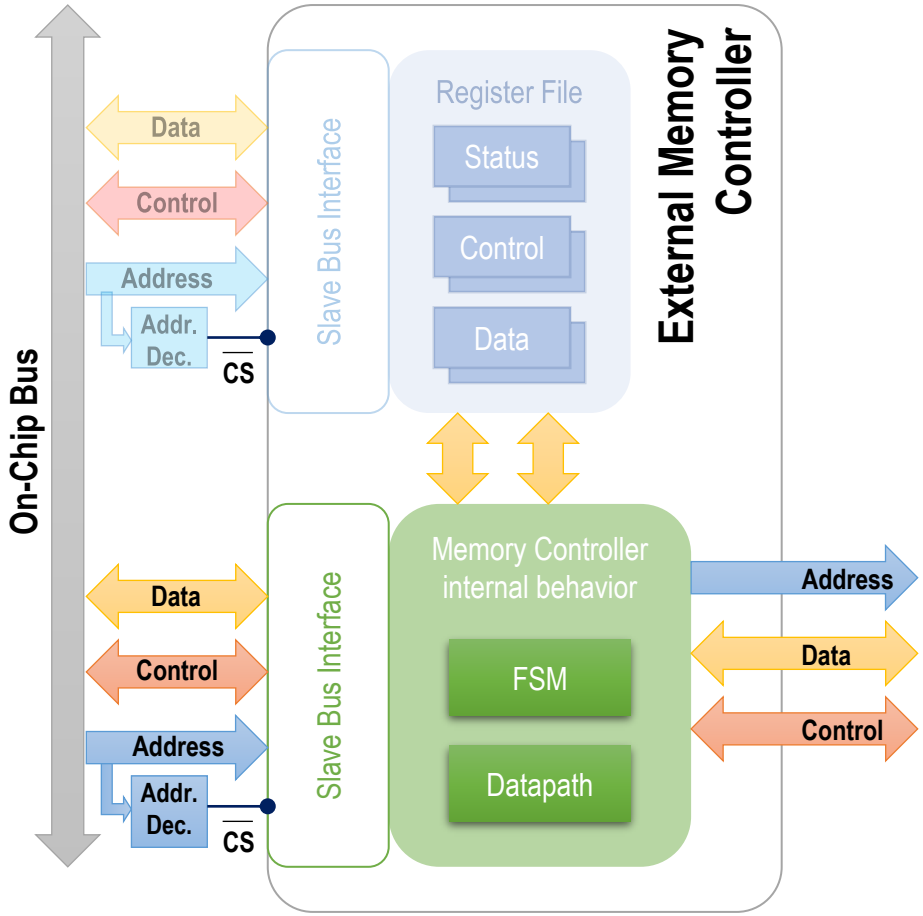
Peripheral examples

- External Memory Controller
  - To configure or read the state of the memory controller
    - The microprocessor core accesses the memory controller peripheral registers starting at the base address
      - Identical to a standard peripheral access
    - Control registers
      - Type of memory to be driven
      - Type of access (Page Mode, EDO, …)
      - Number of banks
      - Word size
      - Burst size
      - Timing parameters (latency)



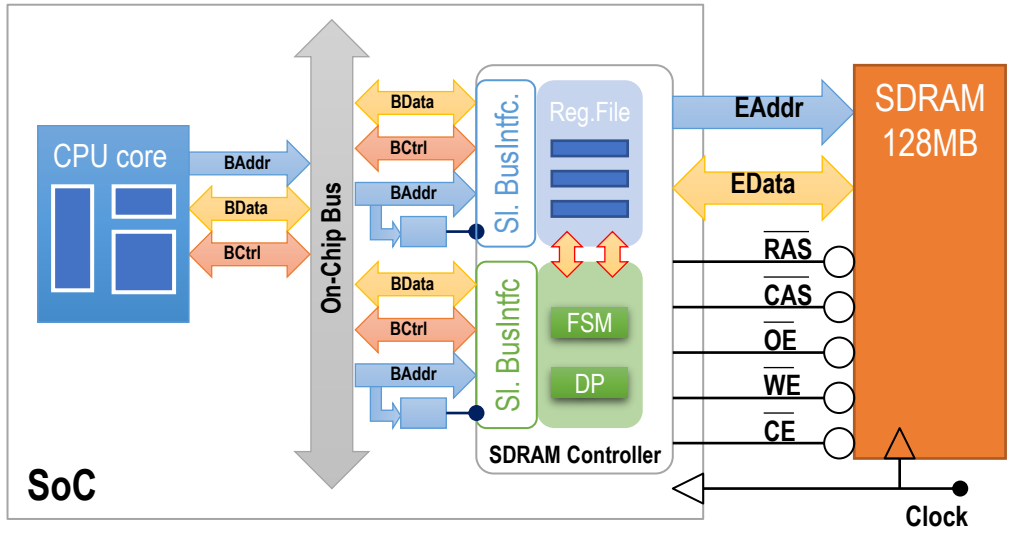
Peripheral examples

- External Memory Controller
  - To access the external memory
    - The microprocessor core generates addresses assigned to the memory
      - The transaction is initiated on the On-Chip Bus
    - The external memory controller detects the transactions
      - It generates the proper external signals to access the memory
        - Memory protocol
        - Address translations
      - It reports on the On-Chip Bus the results
    - The microprocessor core only sees a standard On-Chip Bus transaction
      - Possibly with some delay



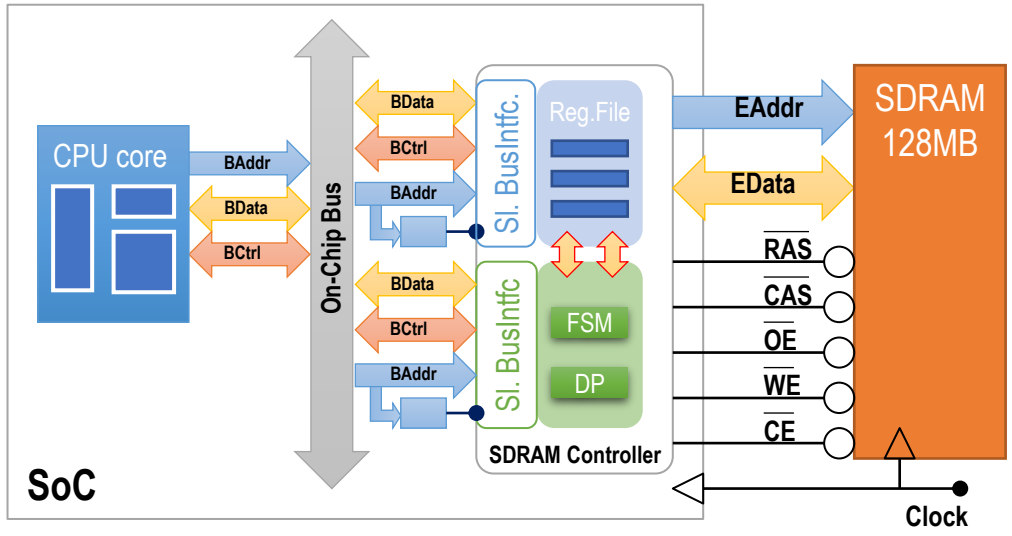
Peripheral examples

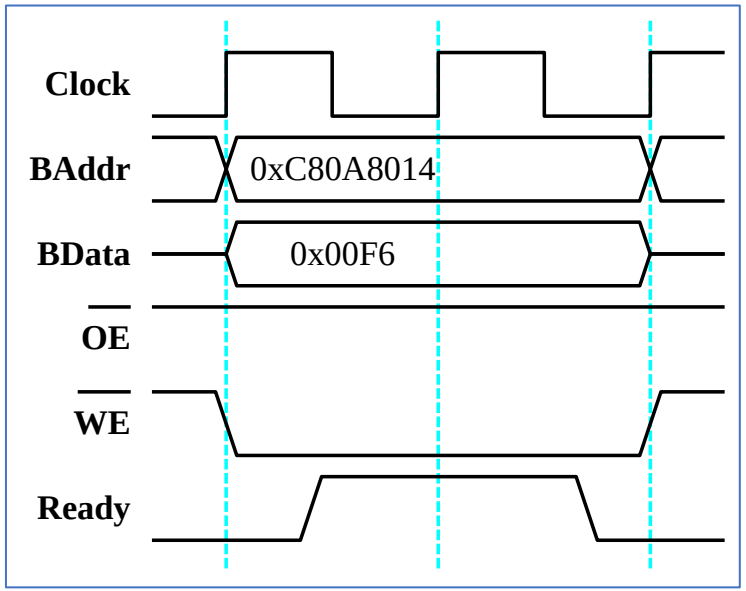
- External Memory Controller example
  - On-Chip Bus signal sizes
    - BAddr on 32 bits
    - BData on 16 bits
  - SDRAM Controller
    - Mapped at SDCBase = 0xC80A8000
    - Range: 0xC80A8000 – 0xC80B0000 (4 Kbyte)
  - External memory 128 Mbyte (1 Gbit)
    - Mapped at MEMBase = 0x20000000 (512 Mbyte)
    - Range: 0x20000000 – 0x28000000
    - EAddr on 15 bits
      - Row address on 15 bits, column address on 12 bits
    - EData on 8 bits



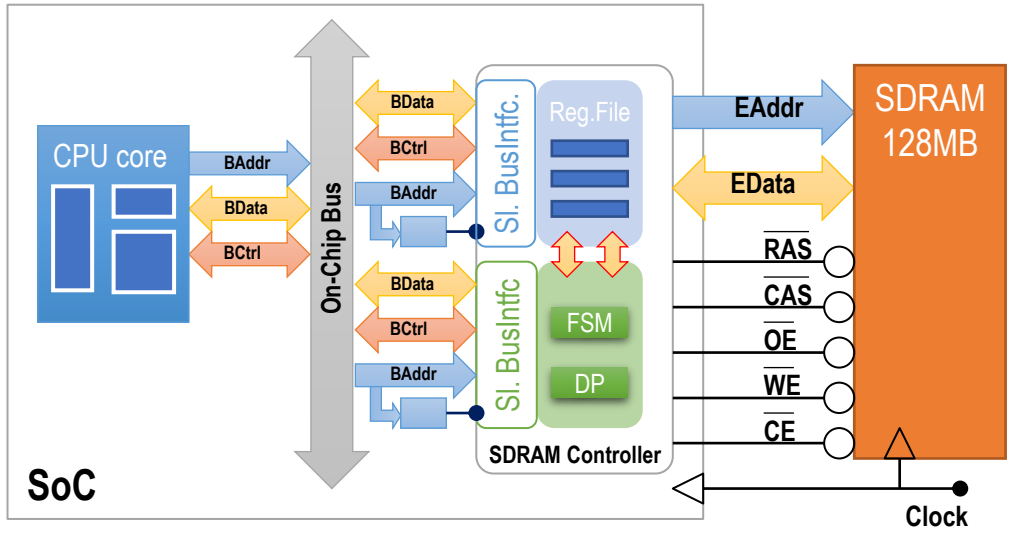
Peripheral examples

- External Memory Controller example
  - First transaction
    - CPU Core writes the value 0xF6 on the 11th register of the SDRAM Controller peripheral
      - Peripheral address: 0xC80A8014
        - Base: 0xC80A8000, Offset: 0x00000014
      - Value to write: 0x00F6
    - Transaction on the On-Chip Bus
      - The Slave Bus Interface for the Register File is activated
      - In this example the write takes 2 clock cycles
      - Ready signal generated by the slave
        - See AMBA bus, for instance
    - Nothing happens on the external memory interface





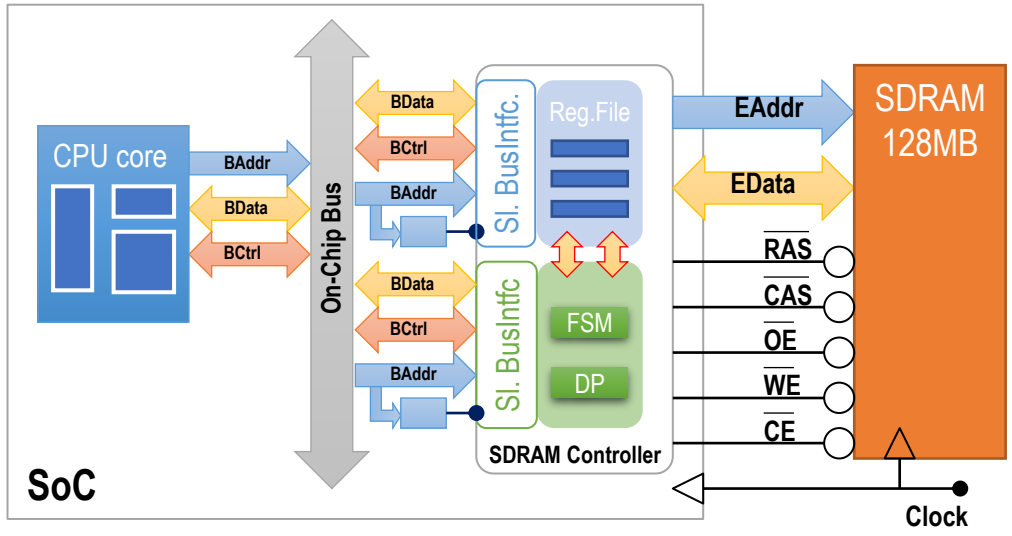
Peripheral examples

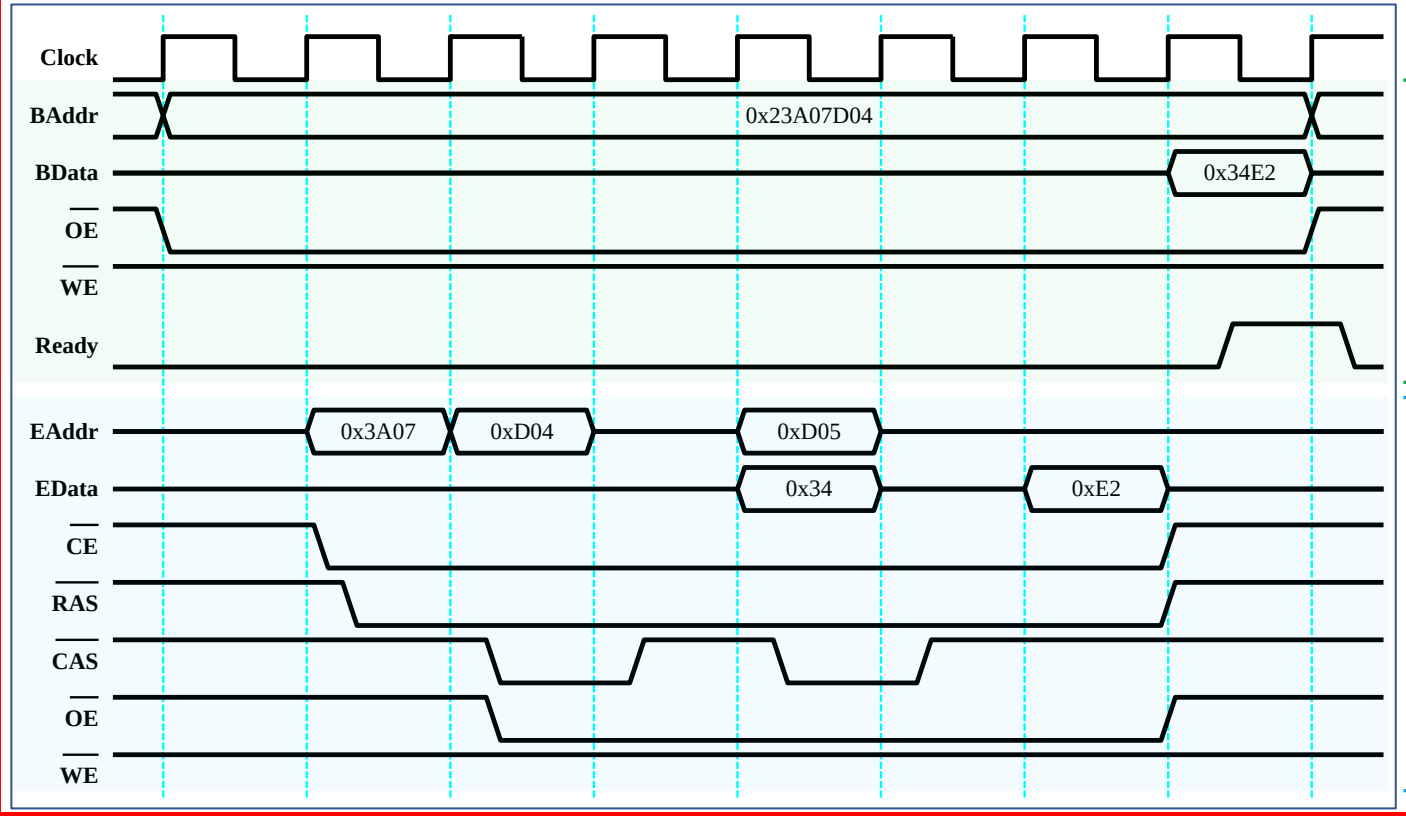


- External Memory Controller example
  - Second transaction
    - CPU Core reads the value 0x34E2 from address 0x23A07D04
      - Base: 0x20000000, Offset: 0x03A07D04
      - The transaction is initiated on the On-Chip Bus
      - The address belongs to the external SDRAM
      - The SDRAM Controller internal behavior detects the transaction
    - The SDRAM Controller generates external signals to access the SDRAM
      - Two read accesses are needed
        - To compose a 16 bit data from two 8 bit reads
      - Row Address: 0x3A07, Column Address: 0xD04, Data: 0x34
      - Row Address: 0x3A07, Column Address: 0xD05, Data: 0xE2
    - The transaction on the On-Chip Bus is then terminated

Peripheral examples

- External Memory Controller example
  - Second transaction timing diagram





On-Chip Bus

The transaction may be split to allow other operations while waiting

External memory signals

Using EDO access mechanism

Peripheral examples

- Datasheet examples
  - Samsung Exynos 4210
  - Texas Instruments MSP 430
  - Analog Devices Blackfin ADSP-BF51x
  - Freescale i.MX28
  - Altera Nios II

Peripheral examples

- Samsung Exynos 4210

Page 28 of the datasheet

Base Address	Limit Address	Size	Description
0x0000_0000	0x0001_0000	64 KB	iROM
0x0200_0000	0x0201_0000	64 KB	iROM (mirror of 0x0 – 0x10000)
0x0202_0000	0x0204_0000	128 KB	iRAM
0x0300_0000	0x0303_0000	192 KB	SRP-RAM or general purpose.
0x0381_0000	0x0383_0000	–	AudioSS's SFR region
0x0400_0000	0x0402_0000	128 KB	SROMC's Bank0
0x0500_0000	0x0502_0000	128 KB	SROMC's Bank1
0x0600_0000	0x0602_0000	128 KB	SROMC's Bank2
0x0700_0000	0x0702_0000	128 KB	SROMC's Bank3
0x0800_0000	0x0C00_0000	64 MB	PCI-e memory
0x0C00_0000	0x0C40_0000	–	OneNANDC's channel 0/1
0x0C60_0000	0x0CD0_0000	–	OneNANDC's SFR region
0x0CE0_0000	0x0D00_0000	–	NFCON's SFR region
0x1000_0000	0x1400_0000	–	SFR region
0x4000_0000	0xA000_0000	1.5 GB	DMC-0's memory
0xA000_0000	0x0000_0000	1.5 GB	DMC-1's memory

External NOR Flash, PROM, SRAM (512 Kbyte)

Special Function Register (64 Mbyte)

External DRAM (3 Gbyte)

Peripheral examples

- Samsung Exynos 4210
  - Each peripheral has 64 Kbytes of space assigned for the register file
  - Multiple identical peripherals with separate register files
    - Can be configured differently
    - They are independent

Page 29-32 of the datasheet

Base Address	IP
0x1387_0000	I2C1
0x1388_0000	I2C2
0x1389_0000	I2C3
0x138A_0000	I2C4
0x138B_0000	I2C5
0x138C_0000	I2C6
0x138D_0000	I2C7
0x138E_0000	I2CHDMI
0x1391_0000	TSADC
0x1392_0000	SPI0
0x1393_0000	SPI1
0x1394_0000	SPI2
0x1395_0000	–
0x1396_0000	I2S1
0x1397_0000	I2S2
0x1398_0000	PCM1
0x1399_0000	PCM2
0x139A_0000	AC97
0x139B_0000	SPDIF
0x139C_0000	–
0x139D_0000	PWMTimer

Peripheral examples

- Samsung Exynos 4210
  - GPIO (page 36 of the datasheet)
    - Multiple function for each pin of the integrated circuit (register map summary, pages 39-53)
      - Function configured using the GPIO control registers (page 55 for GPA0)
    - Data pin for reading and writing when configured as GPIO (page 56 for GPA0)
  - UART (page 511 of the datasheet, register map summary page 523)
    - Control registers (pages 524-529)
      - UART configuration (page 524)
      - Interrupt configuration (page 525-526)
      - Divisor (page 535, page 536 explains BAUD RATE computation)
    - Status registers (pages 530-534)
      - Error conditions (page 532)
      - Buffer status (pages 530-531, 533 for FIFO operations)
    - Data registers (page 535)
      - Data to be sent and data received

Peripheral examples

- Samsung Exynos 4210
  - I2C (page 539)
    - Flowcharts of operation in different modes (Master TX, Master RX, Slave RX, Slave TX)
      - (pages 546-549)

14.4.1 Register Map Summary

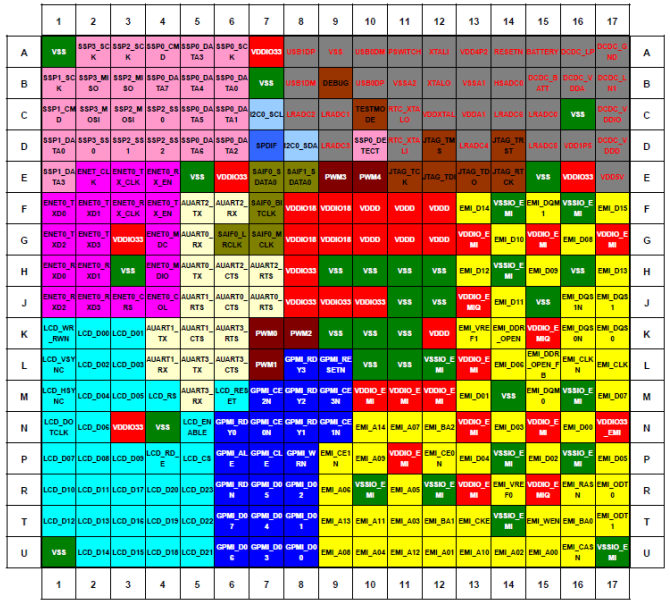
- Base Address: 0x1386_0000, 0x1387_0000, 0x1388_0000, 0x1389_0000, 0x138A_0000, 0x138B_0000, 0x138C_0000, 0x138D_0000, 0x138E_0000

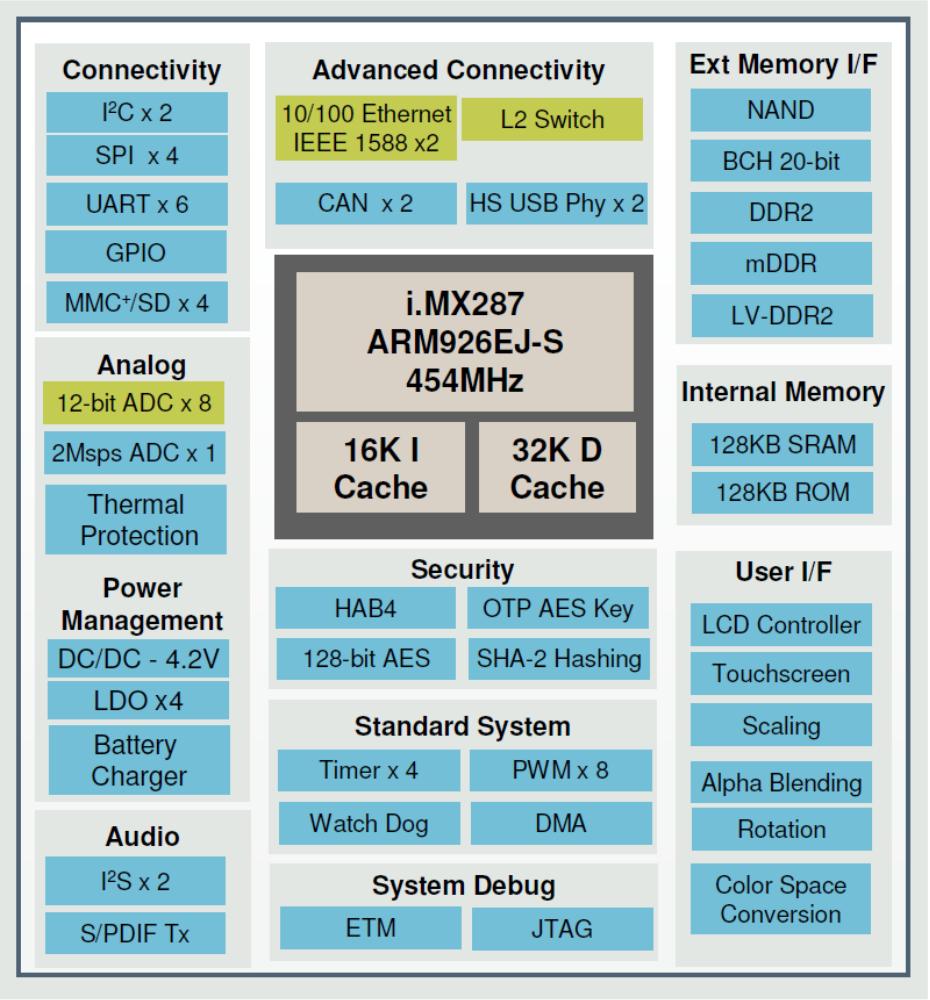
Page 550 of the datasheet

Register	Offset	Description	Reset Value
I2CCONn	0x0000	Specifies the I^2C-Bus Interface0 control register	0x0X
I2CSTATn	0x0004	Specifies the I^2C-Bus Interface0 control/status register	0x00
I2CADDn	0x0008	Specifies the I^2C-Bus Interface0 address register	0xXX
I2CDSn	0x000C	Specifies the I^2C-Bus Interface0 transmit/receive data shift register	0xXX
I2CLCn	0x0010	Specifies the I^2C-Bus Interface0 multi-master line control register	0x00

Peripheral examples

- Freescale i.MX28
  - Presentation
    - Family of processors, same architecture, different features (pages 3 and 7 of presentation)
    - Block diagrams (pages 8-11 of presentation)
  - Memory map (pages 125-127 of datasheet)
  - Pin description and Ball grid figure
    - Pages 640-649 of the datasheet





Peripheral examples

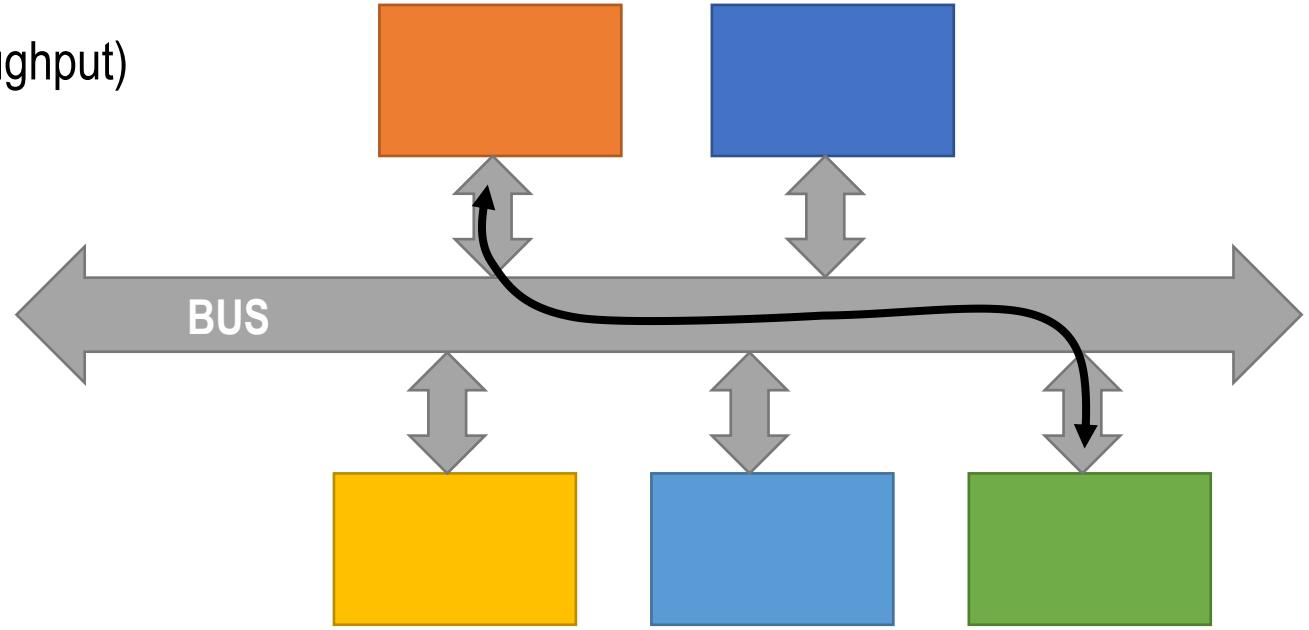
- Freescale i.MX28
  - I^2C peripheral (page 1729)
    - I^2C Bus protocol (page 1732-1734)
    - I^2C example of EEPROM reading and writing (pages 1735-1736)
    - I^2C Registers (map on page 1752, detailed descriptions pages 1753-1774)
  - PWM peripheral (page 1775)
    - PWM block diagram (pages 1776-1777)
    - PWM registers (active and inactive page 1785, states and period page 1786)
  - Synchronous Serial Port (SSP, used for SPI, page 1311)
    - SD/SDIO/MMC mode (pages 1321-1329)
    - Register map summary, page 1330
  - External Memory Interface (EMI, used for external DDR, page 1061)

Peripheral examples

- Texas Instruments MSP430
  - DMA Controller (page 287)
    - Addressing modes (page 290)
    - DMA Flow Chart (single page 292, burst page 294)
    - DMA Registers (map page 302)
      - SA, DA, SZ (page 305-306), Control for Increment, Decrement or Unchanged (page 304)
  - Timer (page 355)
    - Timer operation
    - Capture/Compare (page 362-363)
    - Timer registers (page 369)
- Altera Nios II
  - GPIO
  - UART

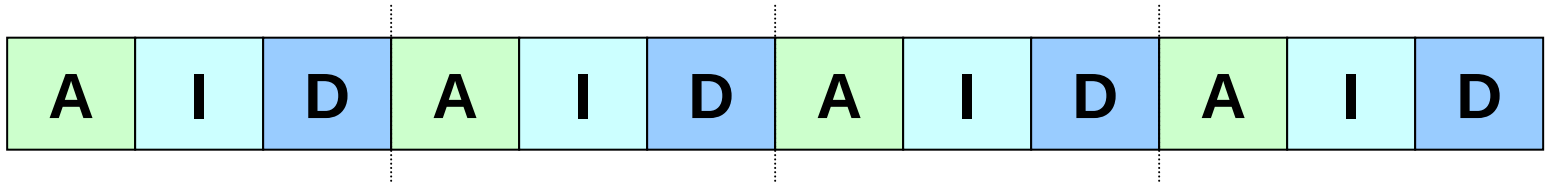
AMBA Bus

- Introduction
  - A BUS in general is a single interconnection to connect many components together
    - Reduced costs, more flexible compared to multiple point-to-point interconnections
    - Slower because shared
      - Used in mutual exclusion
  - Transfers a single data at a time
    - Number of transfers per second (throughput)
    - Size of each transfer
  - Usually divided in sub-busses
    - Address Bus
    - Data Bus
    - Control Bus



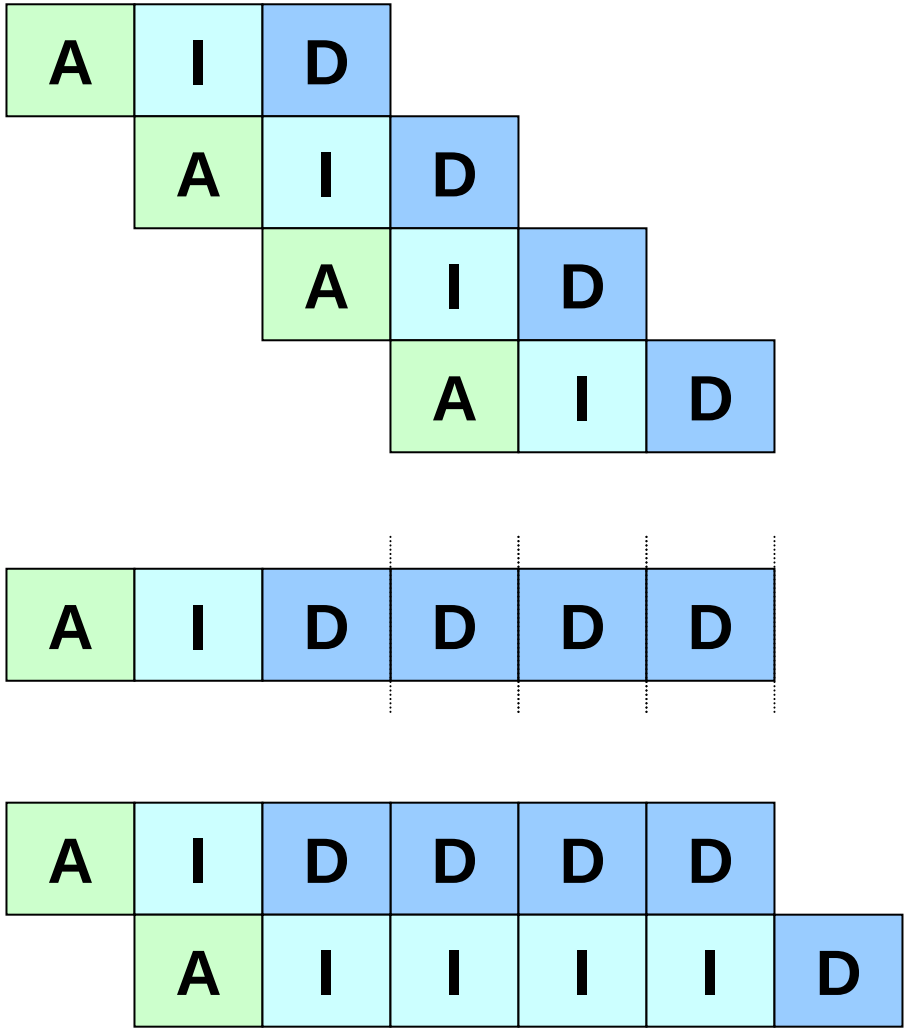
AMBA Bus

- Introduction
  - On Chip Bus
    - Can be synchronous, semi-synchronous, asynchronous
    - Typically contains Address, Data, Control (chip select, read and write enables, byte enables, …)
  - Bus cycle phases
    - Arbitration (A)
    - Addressing (or indexing, I)
    - Transmission (or data, D)
    - Acknowledge (K) (optional)
  - Without a pipeline
    - All phases in sequence
    - Latency = 3 T_{ck}
    - Throughput = $1/(3T_{ck})$



AMBA Bus

- Introduction
  - How to improve performance
  - Pipelined Bus
    - All phases concurrently
    - But applied to different transactions
    - Latency = 3 T_{ck}
    - Throughput = $1/T_{ck}$
    - Stalls might decrease the final throughput
  - Burst transfers
    - Reduced overhead
    - Good for modern memory and cache hierarchies
    - Pipelining less useful
      - But still used in many cases



AMBA Bus

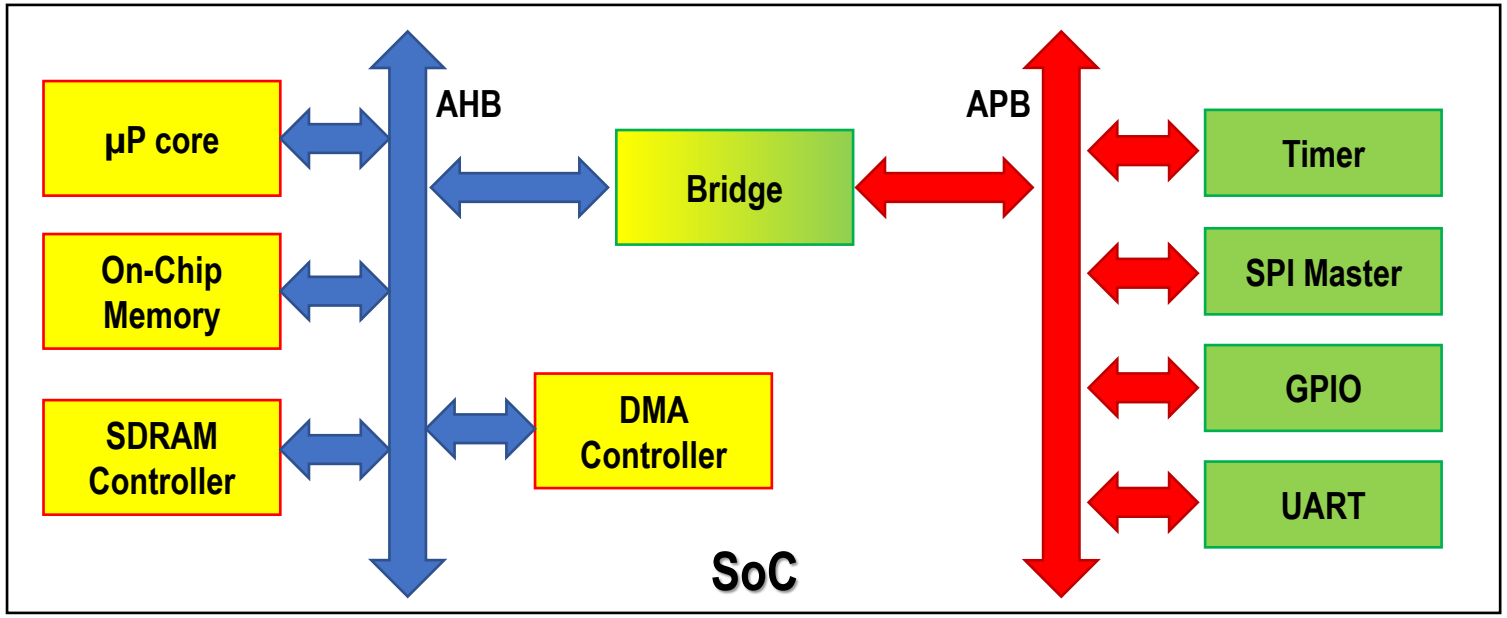
- Created by ARM
  - Dedicated to transmissions within a System on Chip (SoC)
  - **A**dvanced **M**icrocontroller **B**us **A**rchitecture
- Three versions (AMBA 2.0):
  - AHB: Advanced High-Performance Bus
    - Semisynchronous, pipelined, burst, multimaster
  - ASB: Advanced System Bus
  - APB: Advanced Peripheral Bus
    - Synchronous, no pipeline, no burst, single master

AMBA Bus

- Technology independent
  - Specified only at the level of bus cycle
  - Timing depends on the adopted technology
  - Electrical (physical) layer depends on the adopted technology
  - Good for full-custom, standard cell and gate array (FPGA)
- Devices designed at a high level to be compatible with AMBA can be reused and synthesized depending on the technology
  - Processor cores
  - Peripherals

AMBA Bus

- Typical scheme
  - AHB used as local bus of the processor
    - Connects to
      - Internal On-Chip Memories
      - Memory controllers (to connect external memories)
      - DMA controllers
  - APB used to connect to slow peripherals
    - Bridge to connect AHB with APB
  - Arbiter on AHB for multi-master
  - Centralized address decoder



AMBA AHB Bus

- AHB signals
  - General signals
  - Driven by masters
  - Driven by the Address Decoder

HCLK	Clock, rising edge
HRESETn	Reset (active low)

HADDR[31:0]	Address
HTRANS[1:0]	Transfer type (SEQ, NONSEQ, IDLE, BUSY)
HWRITE	Write signal (active high)
HSIZE[2:0]	Transfer size (8, 16, 32, …, 128)
HBURST[2:0]	Burst size (4, 8, 16, incr., wrapping)
HWDATA[31:0]	Write data bus (can be extended to more bits)
HPROT[3:0]	Protection (we will not see it)

HSELx	Chip selects

AMBA AHB Bus

- AHB signals
  - Driven by slaves
  - Used for arbitration

HRDATA[31:0]	Read data bus (can be extended to more bits)
HREADY	Acknowledge
HRESP[1:0]	Transfer response

HBUSREQx	Bus request
HGRANTx	Bus grant
HLOCKx	Bus lock request
HMASTER[3:0]	Highest priority master
HMASTLOCK	Bus lock
HSPLITx[15:0]	Split transactions

AMBA AHB Bus

- AHB signals
  - Encodings

HTRANS[1:0]	
00	IDLE
01	BUSY
10	NONSEQ
11	SEQ

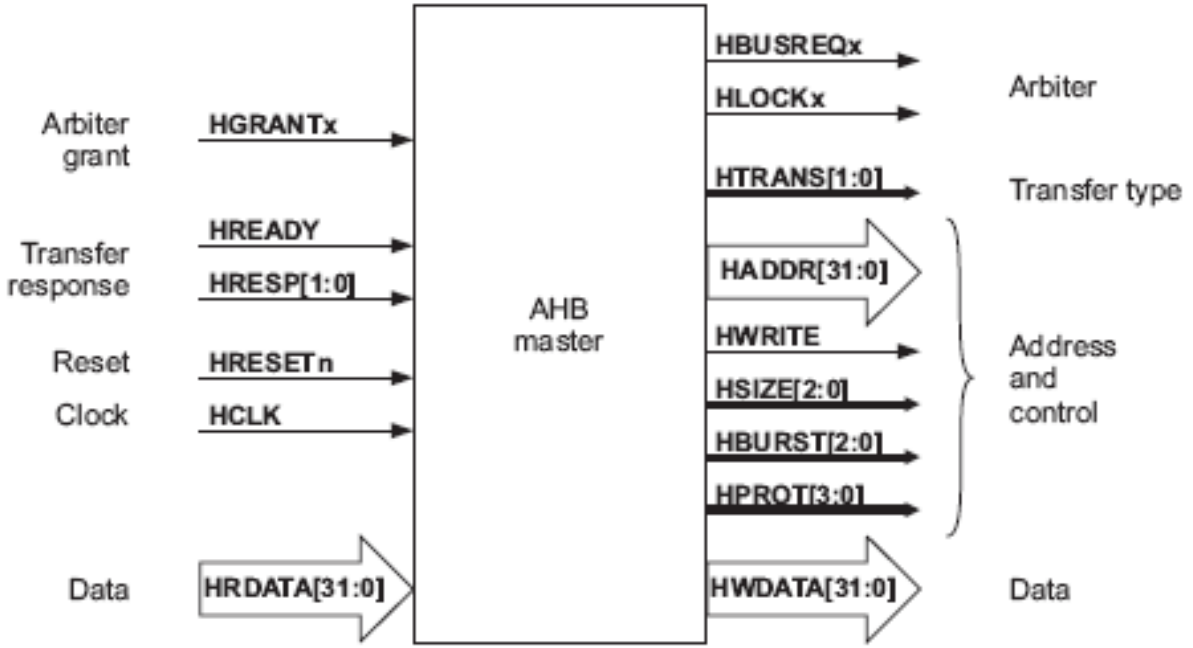
HBURST[2:0]	
000	SINGLE
001	INCR
010	WRAP4
011	INCR4
100	WRAP8
101	INCR8
110	WRAP16
111	INCR16

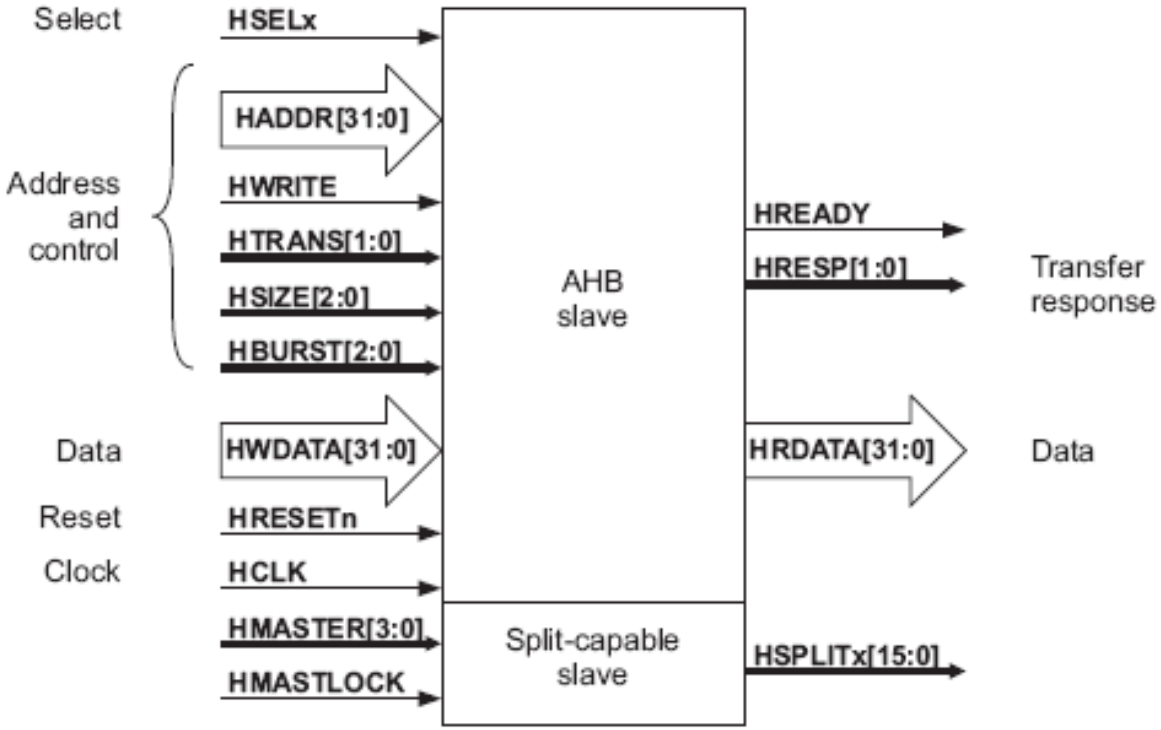
HSIZE[2:0]	
000	8 bit (byte)
001	16 bit (halfword)
010	32 bit (word)
011	64 bit (8 byte)
100	128 bit (16 byte)
101	256 bit (32 byte)
110	512 bit (64 byte)
111	1024 bit (128 byte)

HRESP[1:0]	
00	OKAY
01	ERROR
10	RETRY
11	SPLIT

AMBA AHB Bus

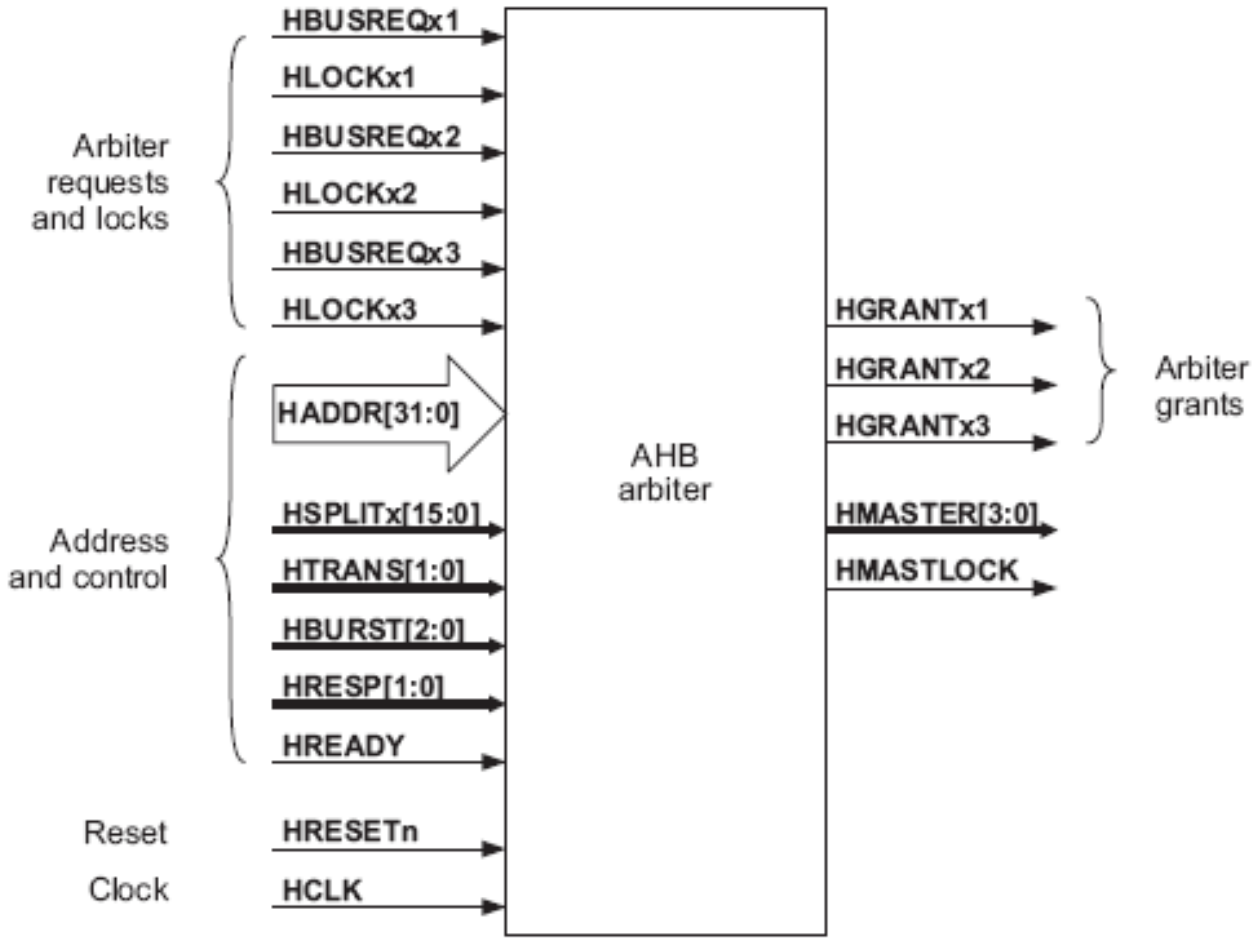
- Bus Elements
  - Bus Master
  - Bus Slave

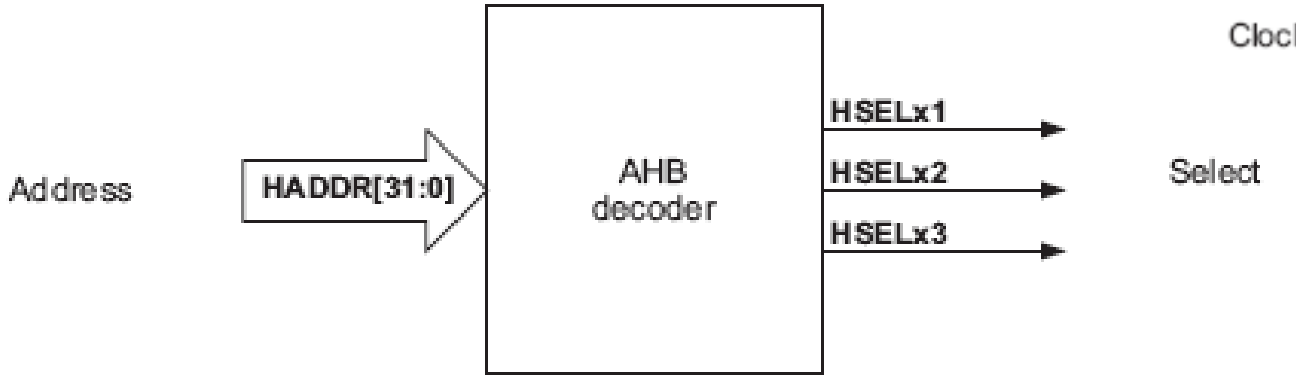




AMBA AHB Bus

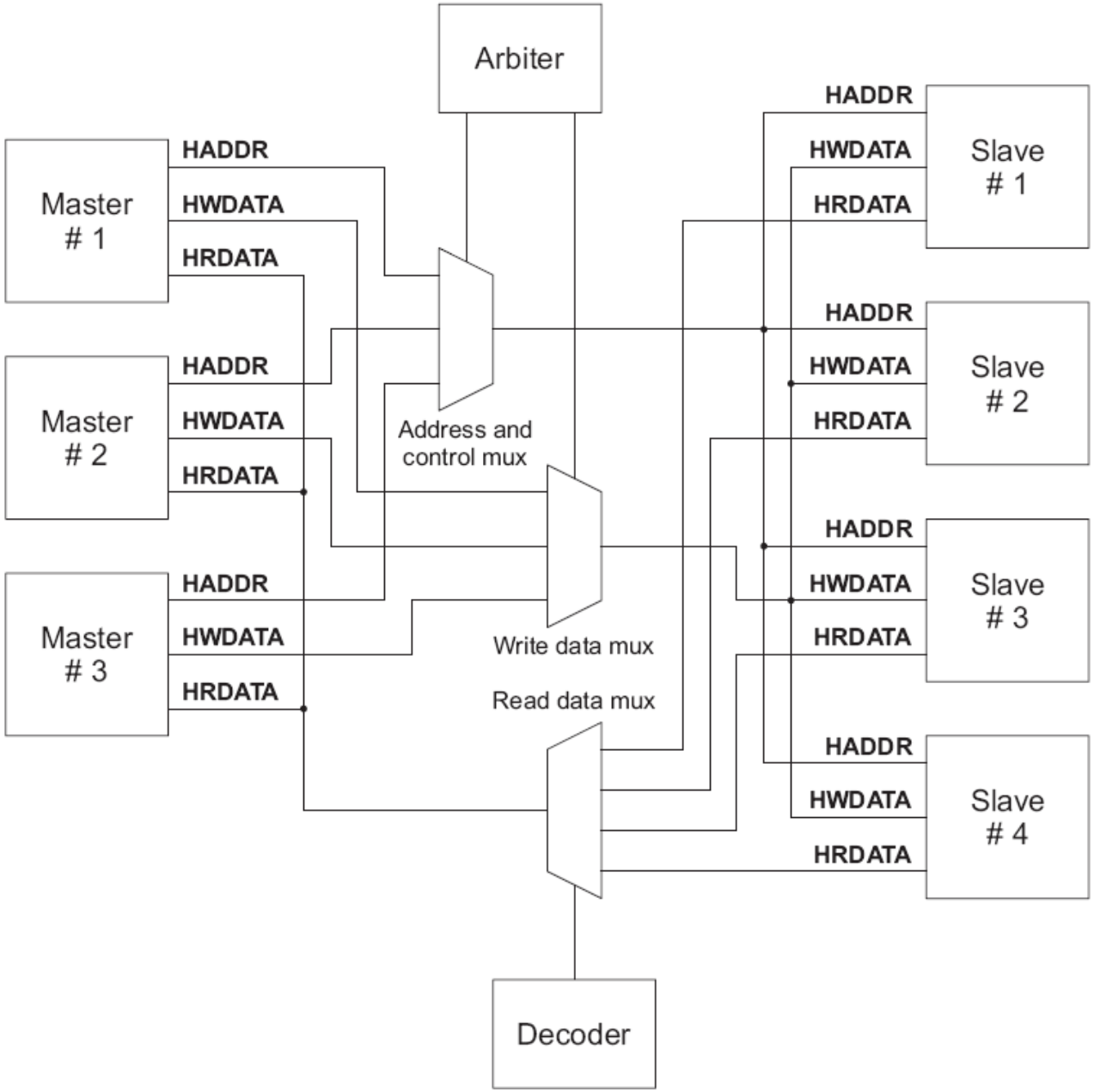
- Bus Elements
  - Centralized Address Decoder
  - Centralized Arbiter





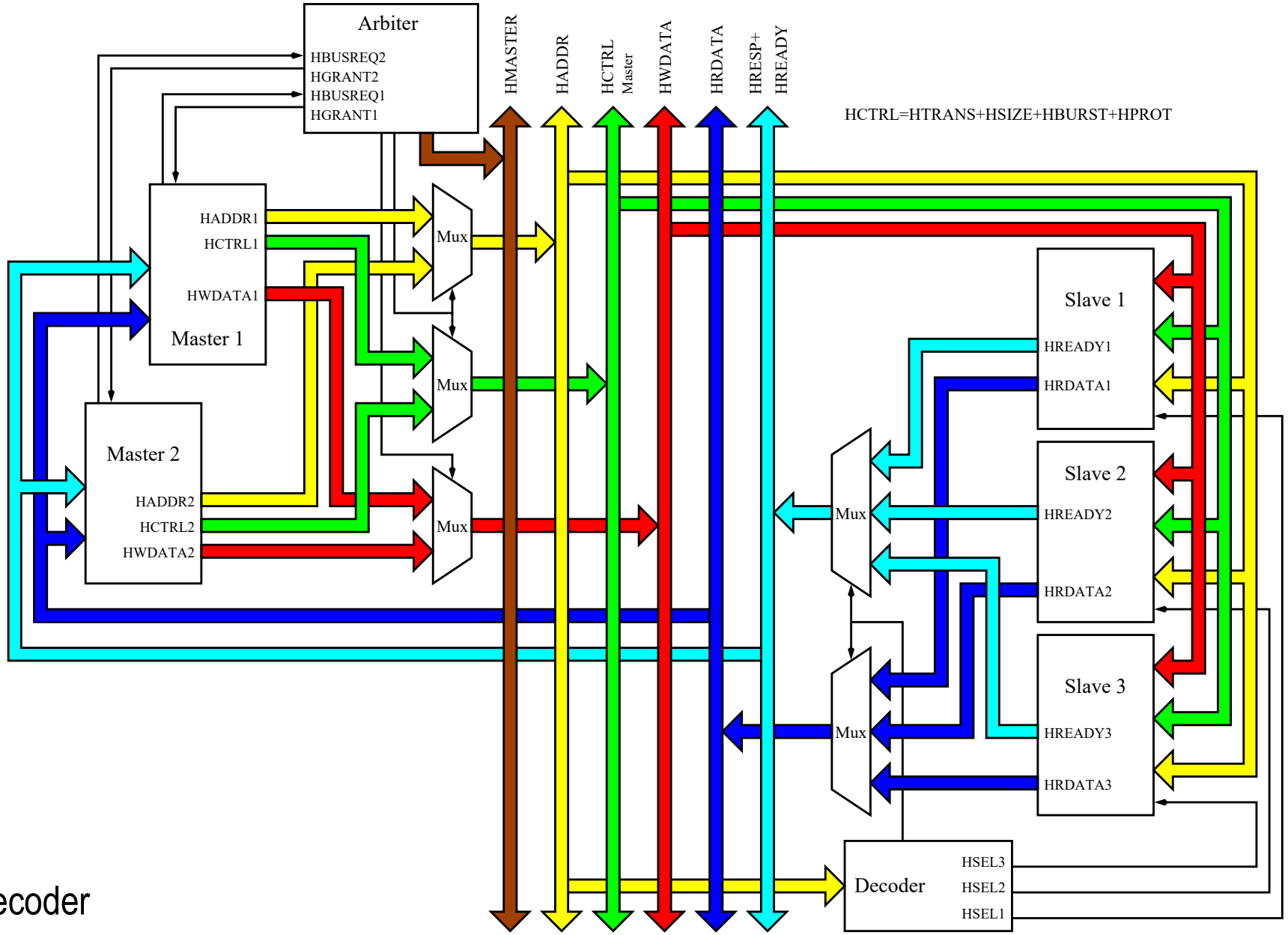
AMBA AHB Bus

- Bus connections
  - Arbiter and address decoder drive a number of multiplexers
    - Avoid conflicts on the wires of the bus
    - Selection signals change based on the bus phase
  - Masters drive
    - Address, Control and Write Data
  - Slaves drive
    - Read Data, Response and Ready Signal



AMBA AHB Bus

- Multiplexers
  - Conflicts on the bus solved using muxes
    - Controlled by Arbiter and Address Decoder
  - Address and control mux
    - Values from masters
    - Selection from arbiter
  - Write data mux
    - Values from masters
    - Selection from arbiter
  - Read data mux
    - Values from slaves
    - Selection from address decoder

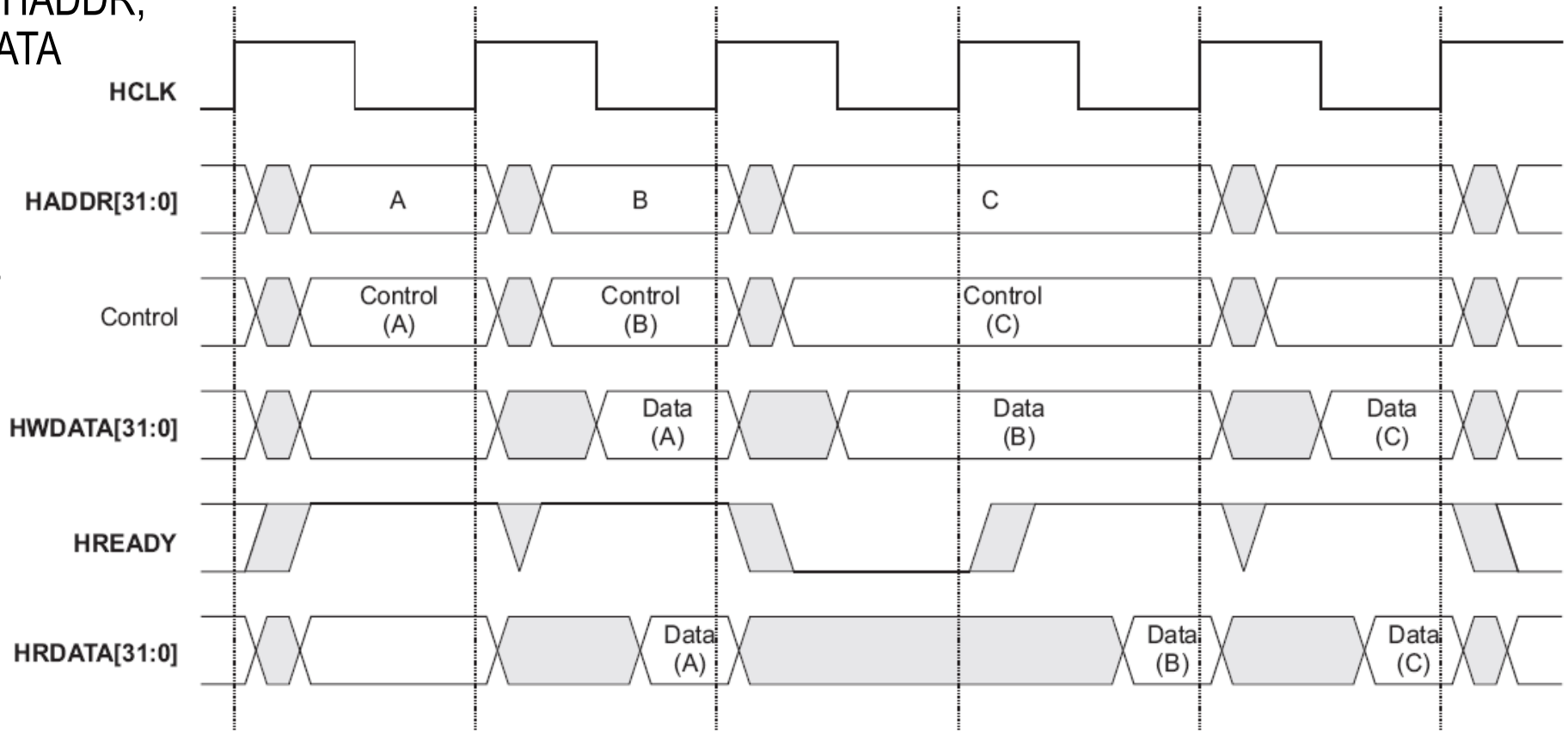


AMBA AHB Bus

- Simple read or write cycle
  - Pipelined
  - Two phases (assume the bus is already arbitrated)
    - Address phase
      - Provides indications on transfer direction, address, width, burst size and type (if any)
      - Always lasts one clock cycle
    - Data phase
      - May be longer than one clock cycle
  - 2 clock cycles, or insert wait state (stalls)
    - Slaves use the HREADY signal

AMBA AHB Bus

- Simple read or write cycle timing diagram
  - Transactions between a master and a slave
    - Master generates HADDR, Control and HWDATA
    - Slave generates HREADY and HRDATA
    - When HREADY is low, the pipeline stalls
    - Clock common to all blocks



AMBA AHB Bus

- Burst read or write cycle
  - Number of beats
    - 4, 8, 16
    - Incrementing or wrapping
  - Number of transferred bytes in a burst
    - $n_{bytes} = n_{beats} \cdot \text{size}$
  - Wrapping
    - Used for cache transfers
    - Wraps at $n_{beats} \cdot \text{size}$ boundaries
      - 4 beats wrapping burst of 32-bit word size wraps at 16 bytes boundaries
        - First address: 56 (aligned to transfer size of 4 bytes)
        - Second address: 60
        - Third address: 48 (wrapped)
        - Fourth address: 52

1st	0	0	1	1	1	0	0	0	56
2nd	0	0	1	1	1	1	0	0	60
3rd	0	1	0	0	0	0	0	0	64
4th	0	1	0	0	1	0	0	0	68

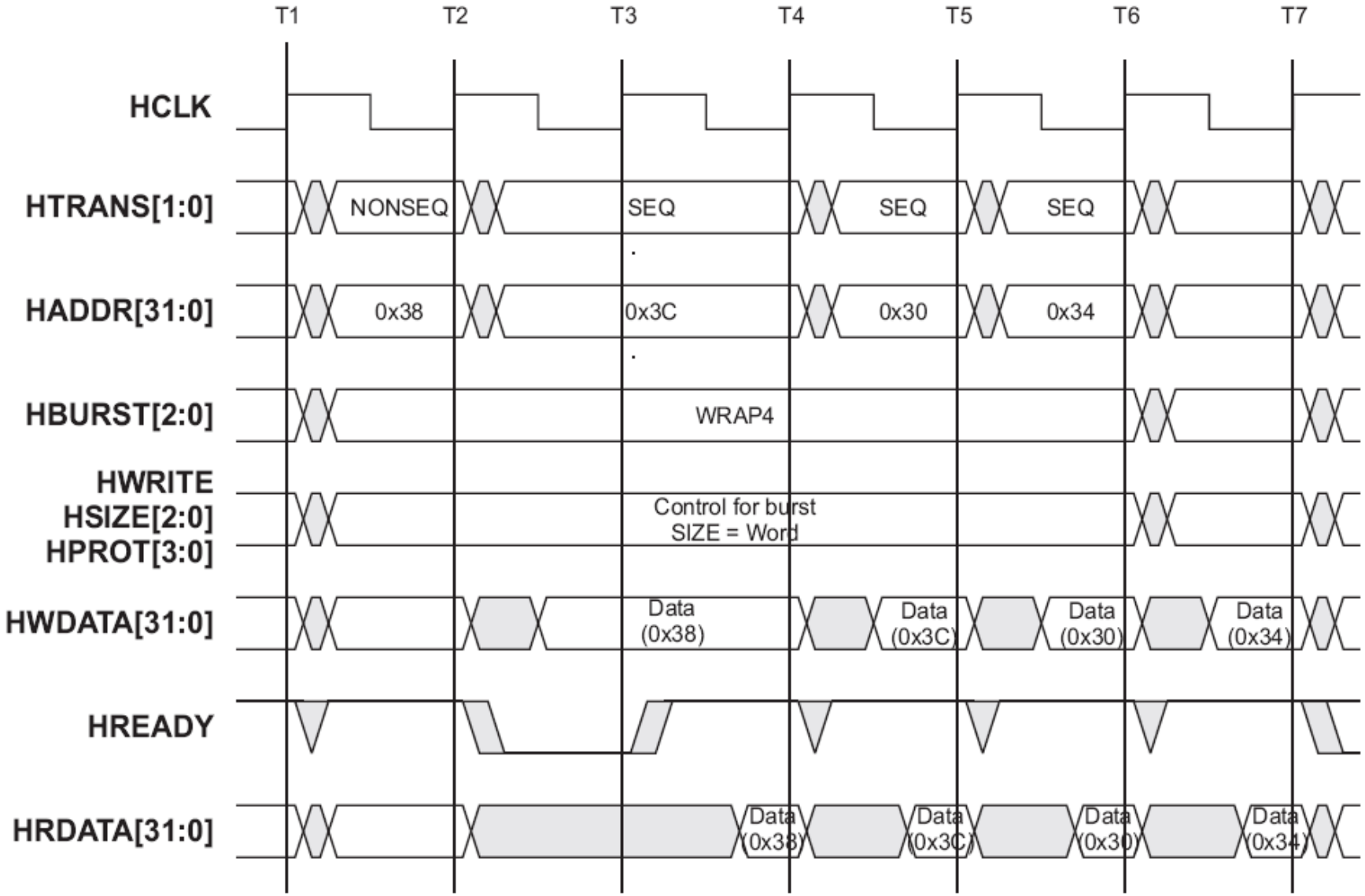
4 beats, 4 bytes size, incrementing

1st	0	0	1	1	1	0	0	0	56
2nd	0	0	1	1	1	1	0	0	60
3rd	0	0	1	1	0	0	0	0	48
4th	0	0	1	1	0	1	0	0	52

4 beats, 4 bytes size, wrapping

AMBA AHB Bus

- Burst read or write cycle
  - Timing diagram

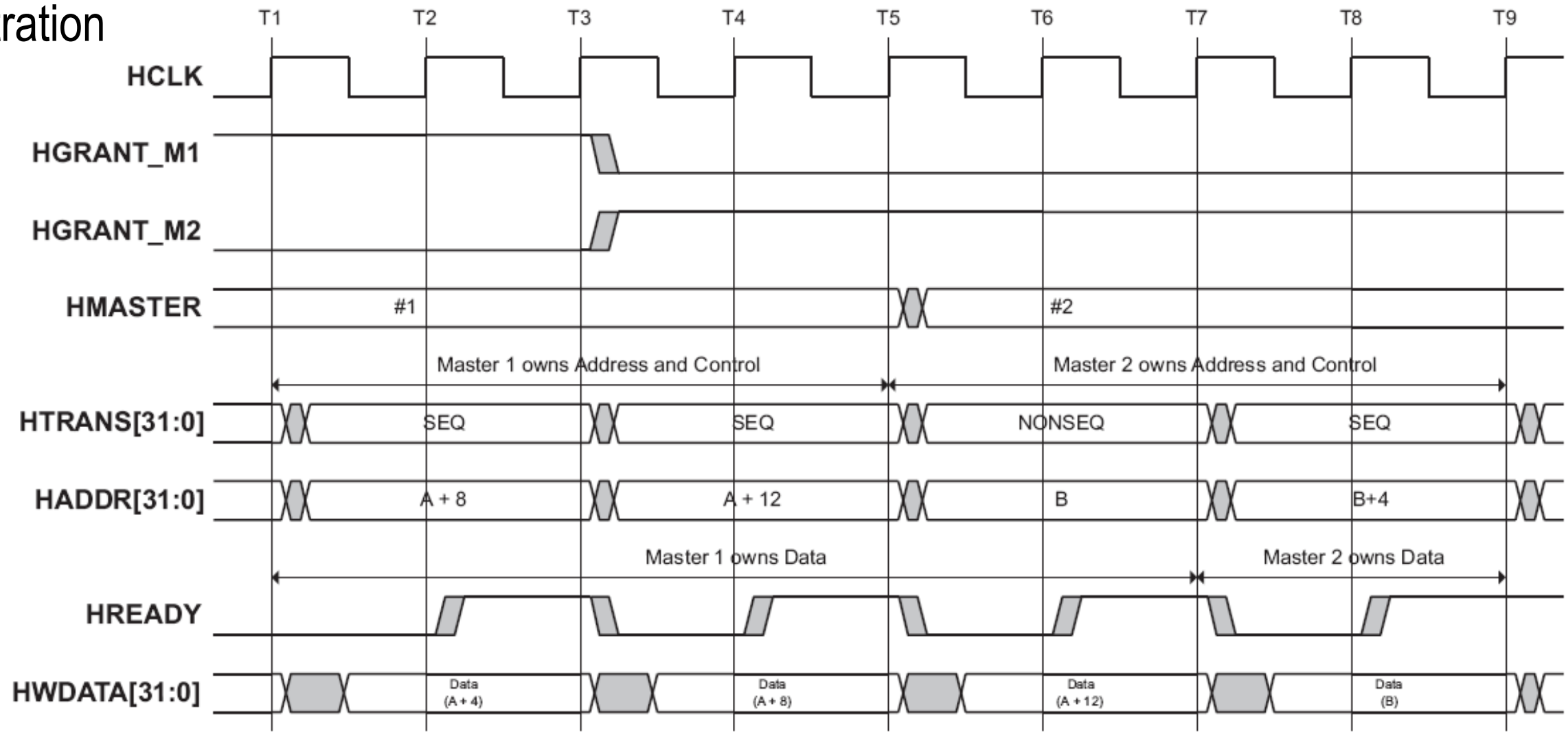


AMBA AHB Bus

- Arbitration
  - Each master requests the bus with the HBUSREQx signal
  - When HGRANTx is obtained, the master waits for HREADY to become high to get the bus
  - Usually the end of a burst is waited
    - A burst can be interrupted if a higher priority master requires the bus
  - The HMASTER[3:0] signal shows which master has the control of the bus
    - Address, Control and Data multiplexer selection signals change at different times
      - Address and Control multiplexer selection signals change first
      - Data multiplexer selection signals change one cycle later
        - Or more in case of pipeline stalls

AMBA AHB Bus

- Arbitration



AMBA AHB Bus

- Split and Retry
  - A slow slave may request to suspend a bus operation
  - The interrupted master is put in a low priority state until the slave allows to continue the transmission (HSPLITx signal)
  - When a slave requires a split, it keeps HREADY low
    - An IDLE cycle is needed to change the bus owner
  - RETRY: like a split, but the master keeps its priority

AMBA APB Bus

- Peripheral bus
  - Lower size an complexity
    - Lower number of wires
  - Lower clock frequency
- Synchronous bus
- Single master
  - Bridge from AHB if used in a large system
  - No need of arbitration
- No pipeline
  - 2 sequential phases
- No burst

AMBA AHB	AMBA APB
Semi-synchronous bus	Synchronous bus
Pipelined	No pipeline
Multiple masters (with arbitration)	Single master (AHB to APB bridge)
Burst transactions	No burst transactions

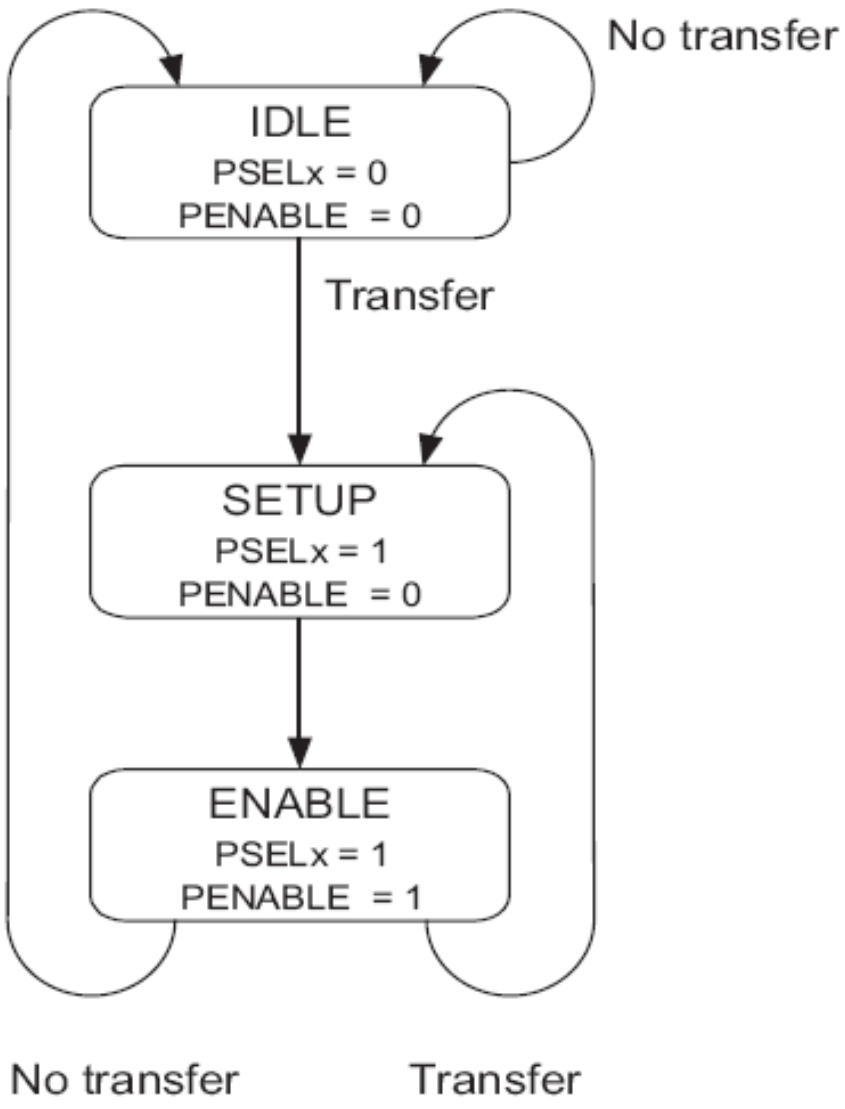
AMBA APB Bus

- APB signals
  - Global Clock and Reset signals
    - Clock is lower than AHB
  - Master (Bridge) drives
    - PADDR
    - PWRITE
    - PENABLE
    - PWDATA
  - Slave drives
    - PRDATA
  - Address Decoder drives
    - PSELx

PCLK	Clock
PRESET	Reset
PADDR	Address (should be stable in the 2 phases)
PWRITE	Read/Write (control)
PSELx	Slave selection (control)
PENABLE	Transfer enable, strobe (control)
PWDATA	Write Data
PRDATA	Read Data

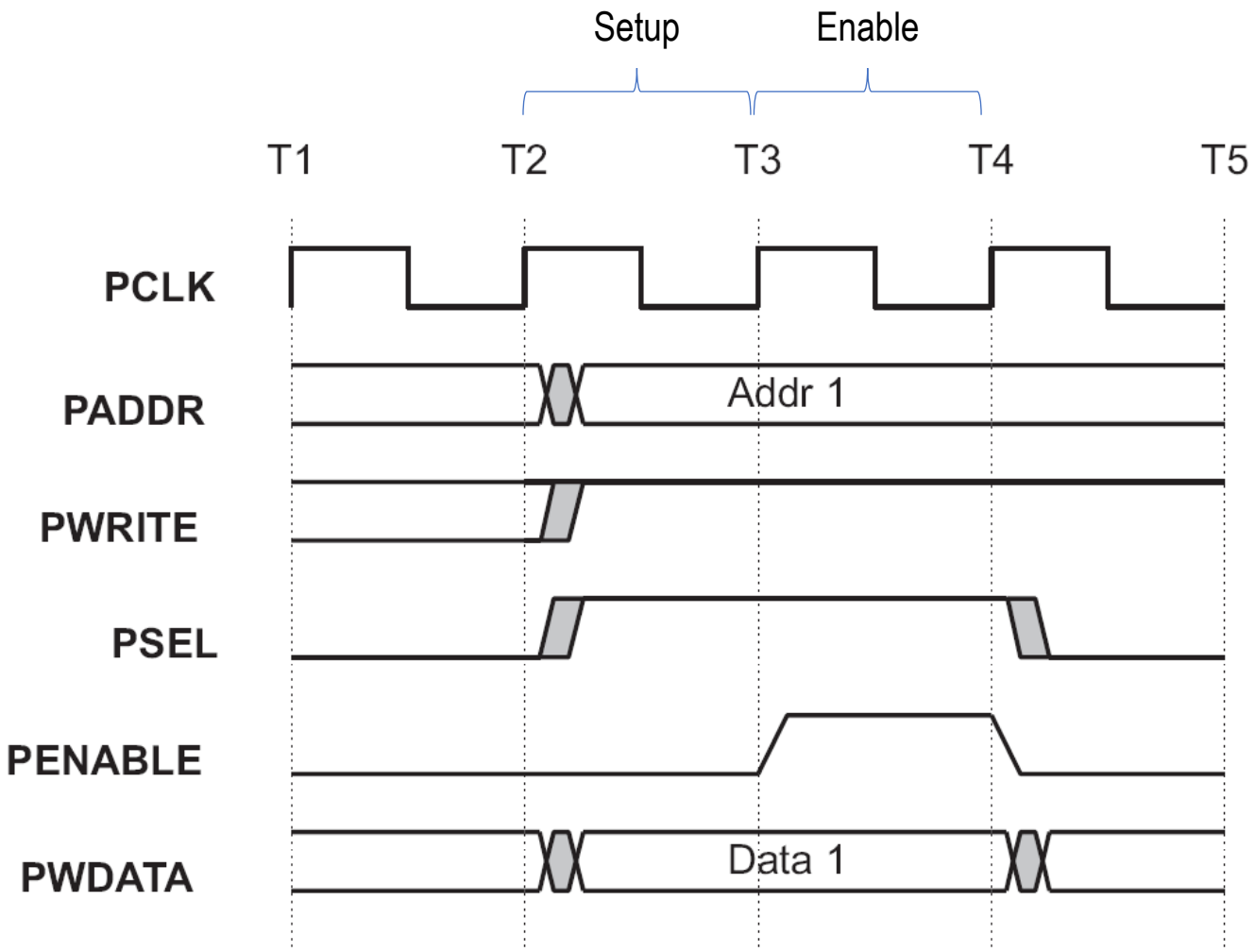
AMBA APB Bus

- APB signals
  - All transmissions last 2 clock cycles
  - When there is no transaction going on
    - All PSELx are at 0
    - PENABLE is set to 0
  - In the first clock cycle of a transaction
    - PADDR contains the slave register address
    - PSELx of the target slave is set to 1, the other are at 0
    - PENABLE is kept at 0
    - PWRITE is set according to the operation
  - In the second clock cycle of a transaction
    - PADDR and PSELx keep their previous value
    - PENABLE is set to 1
    - PWDATA or PRDATA are driven



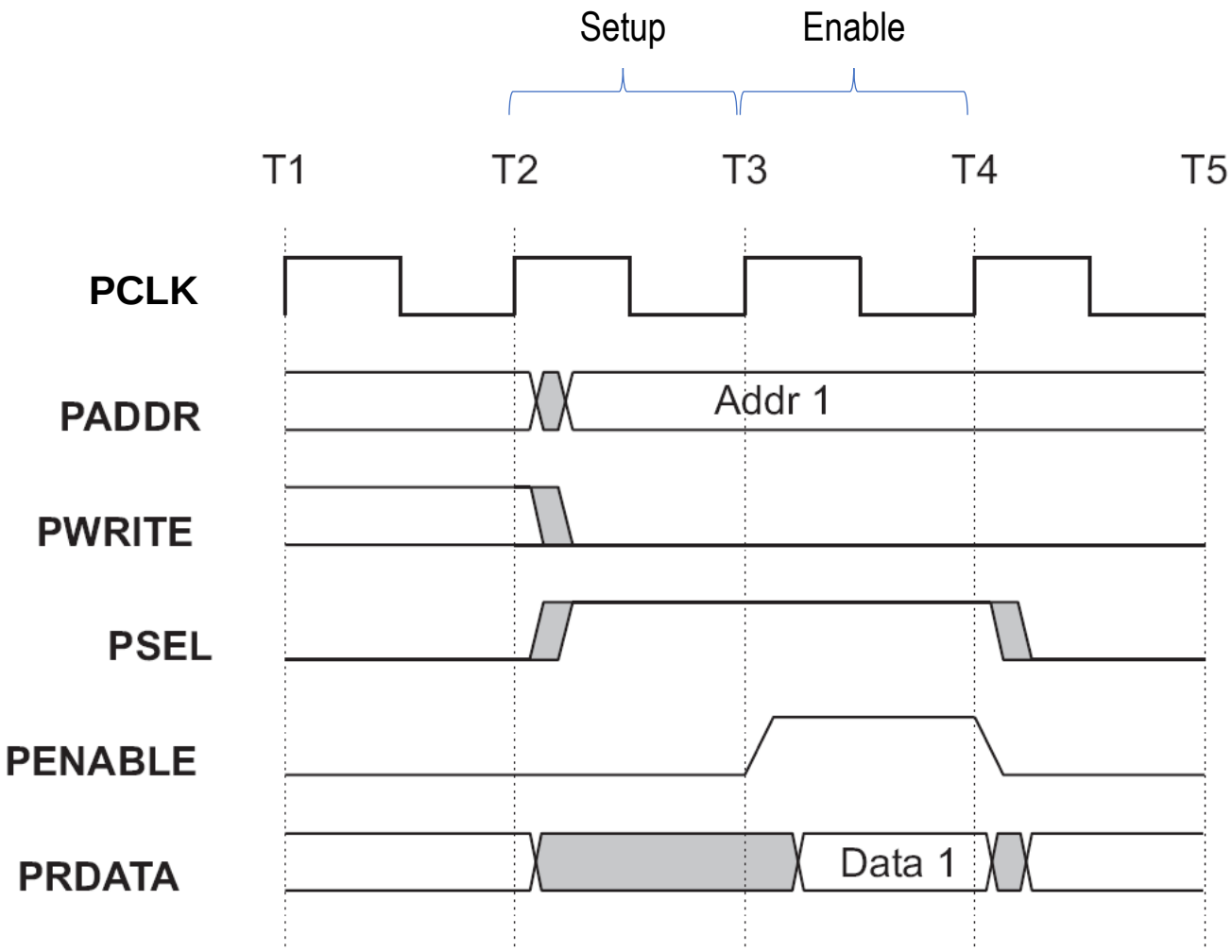
AMBA APB Bus

- APB write cycle
  - Write starts at T2
  - Write terminates in T3
  - PSEL is relative to the target slave
    - All other PSEL remain at 0
  - Cannot stop the operation if the slave is not ready



AMBA APB Bus

- APB read cycle
  - Read starts at T2
  - Read terminates in T3
  - PSEL is relative to the target slave
    - All other PSEL remain at 0
  - Cannot stop the operation if the slave is not ready



AMBA AHB vs. APB Bus

- Comparison of the AHB and APB busses
  - AHB: High Performance
  - APB: Peripheral

AHB	APB
Semi-synchronous	Synchronous
Support for multiple masters	Single Master (typically the Bridge)
Pipelined	Not pipelined
Support for burst transfers	Does not support burst transfer
Complex, expensive	Simple, cheap

AMBA AXI Bus

- AXI Bus (AMBA 3.0 and 4.0)
  - Advanced eXtensible Interface
  - Burst based
    - Issue only the first address of the burst
  - Independent transaction channels
    - Channel
      - Set of information signals and VALID and READY signals that provide a two-way handshake mechanism
    - Read address, write address, read data, write data, write response channels
  - Multiple outstanding transactions
  - Out of order transaction completion
  - Exclusive access
    - For semaphore operations

AMBA AXI Bus

- AXI Handshake
  - Two-way handshake mechanism
  - The information source uses the **VALID** signal to show when valid address, data or control information is available on the channel
  - The destination uses the **READY** signal to show when it can accept the information
  - Both the read data channel and the write data channel also include a **LAST** signal to indicate the transfer of the final data item in a transaction

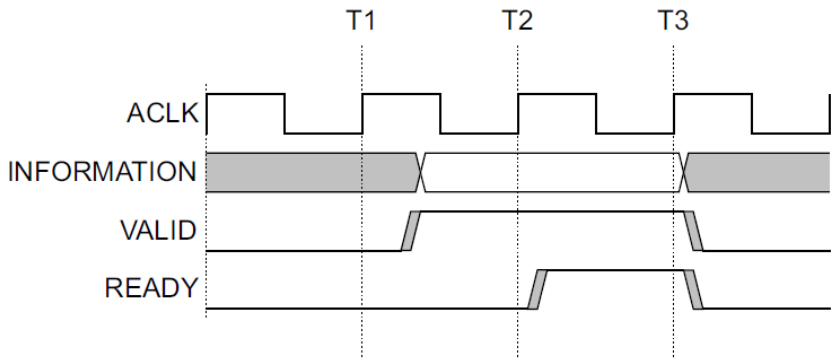


Figure A3-2 VALID before READY handshake

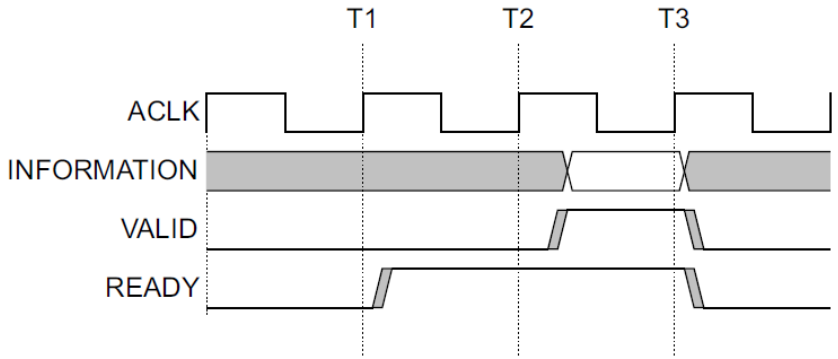


Figure A3-3 READY before VALID handshake

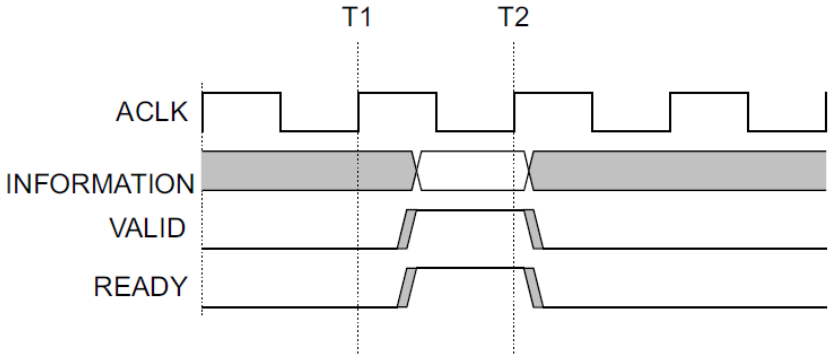


Figure A3-4 VALID with READY handshake

AMBA AXI Bus

- AXI Read
  - Data from slave to master
    - The read address channel carries all of the required address and control information for the transaction
    - The read data channel carries both the read data and the read response information from the slave to the master

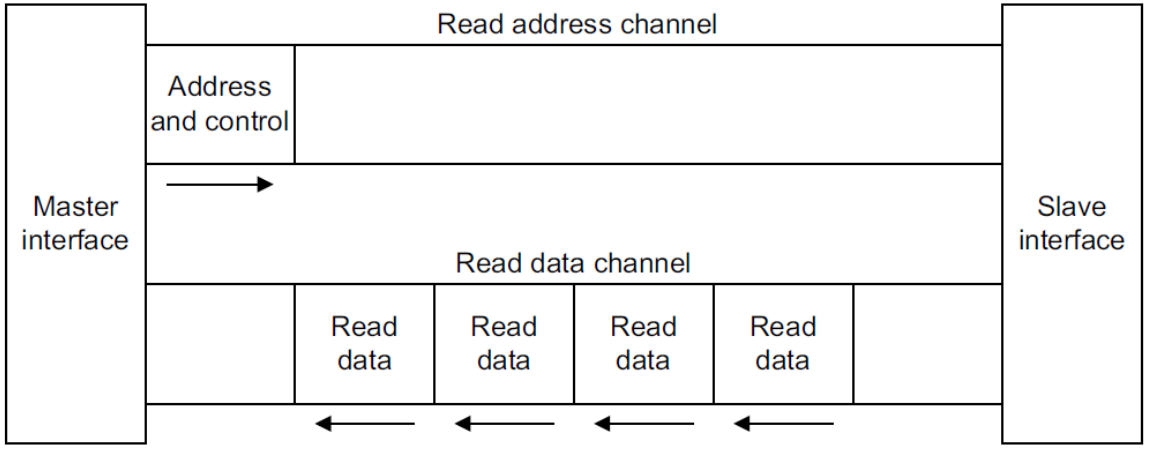
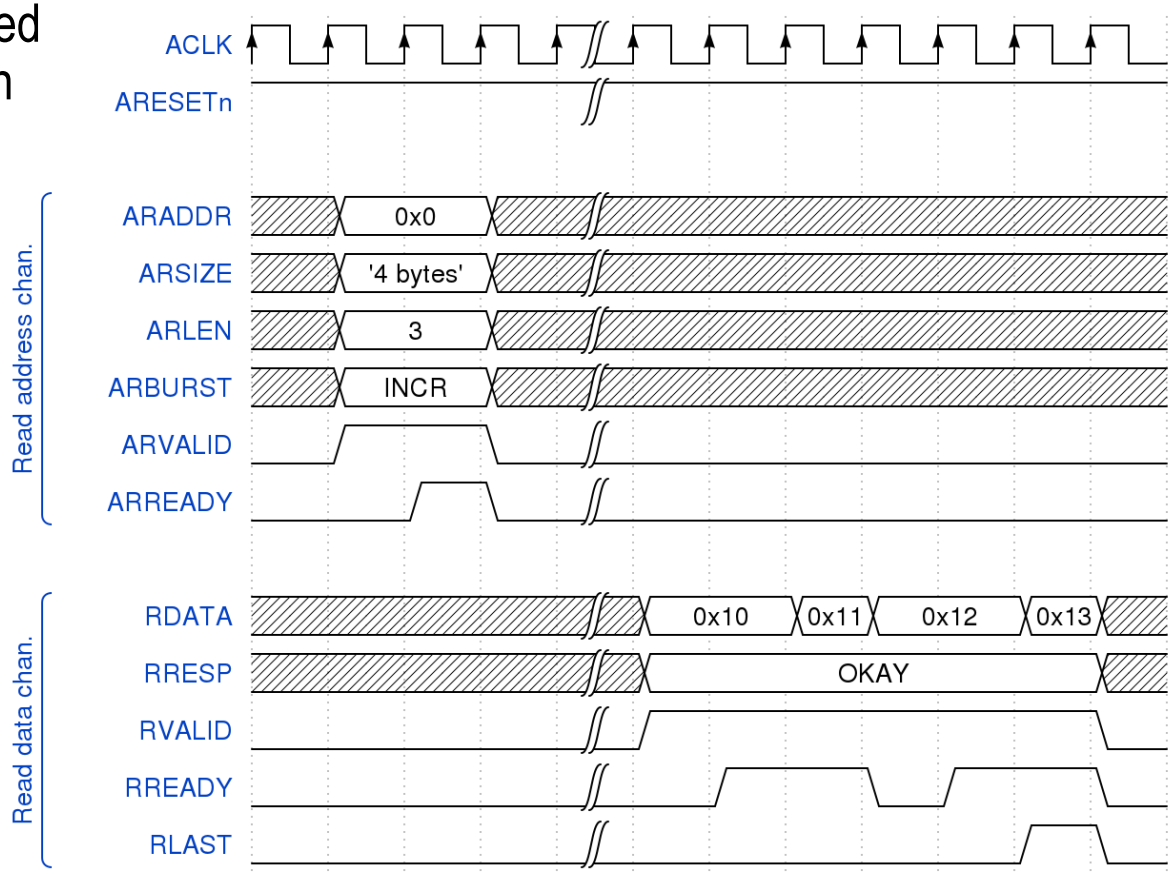


Figure A1-1 Channel architecture of reads



Example from Wikipedia

AMBA AXI Bus

- AXI Write
  - Data from master to slave
    - The write address channel carries all of the required address and control information for the transaction
    - The write data channel carries the write data from the master to the slave
    - A slave uses the write response channel to respond to write transactions
      - All write transactions require completion signaling on the write response channel
      - Completion is signaled only for a complete transaction, not for each data transfer in a transaction

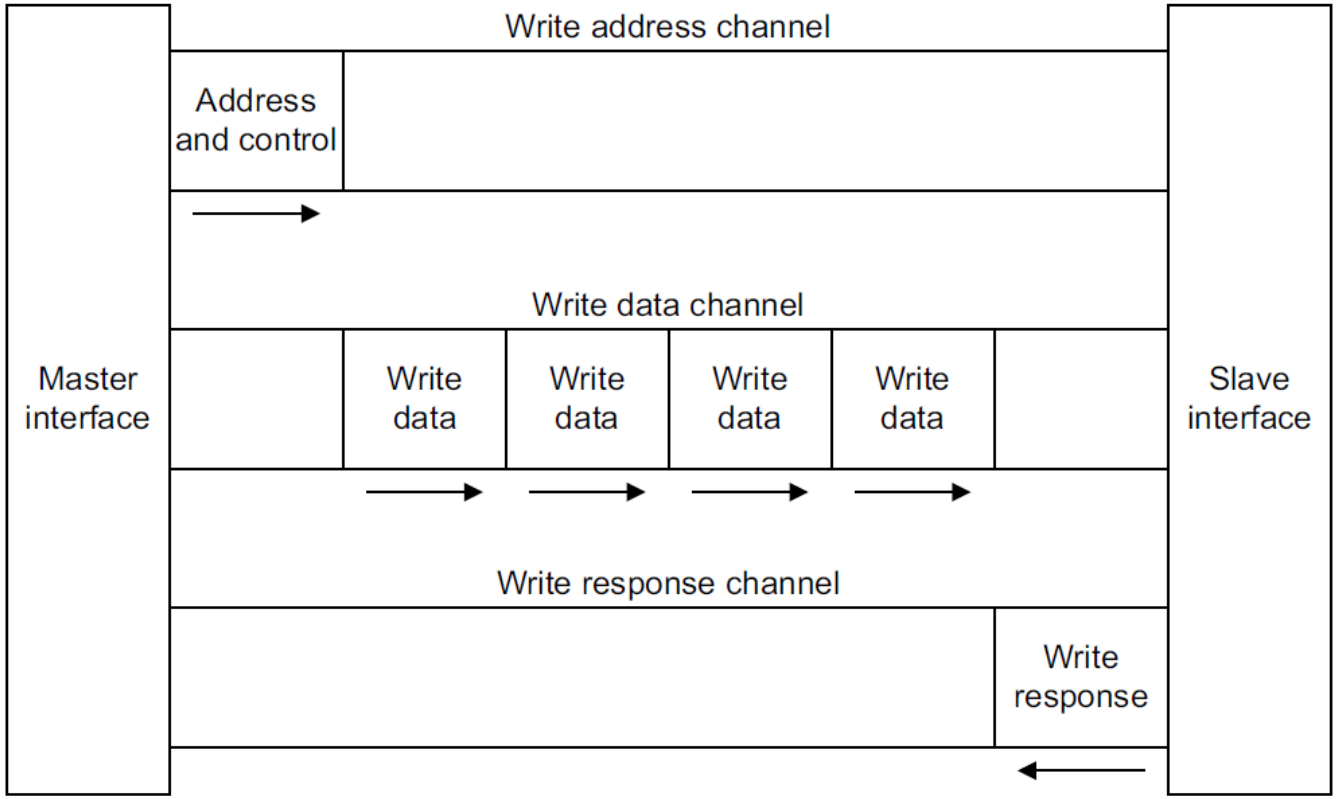
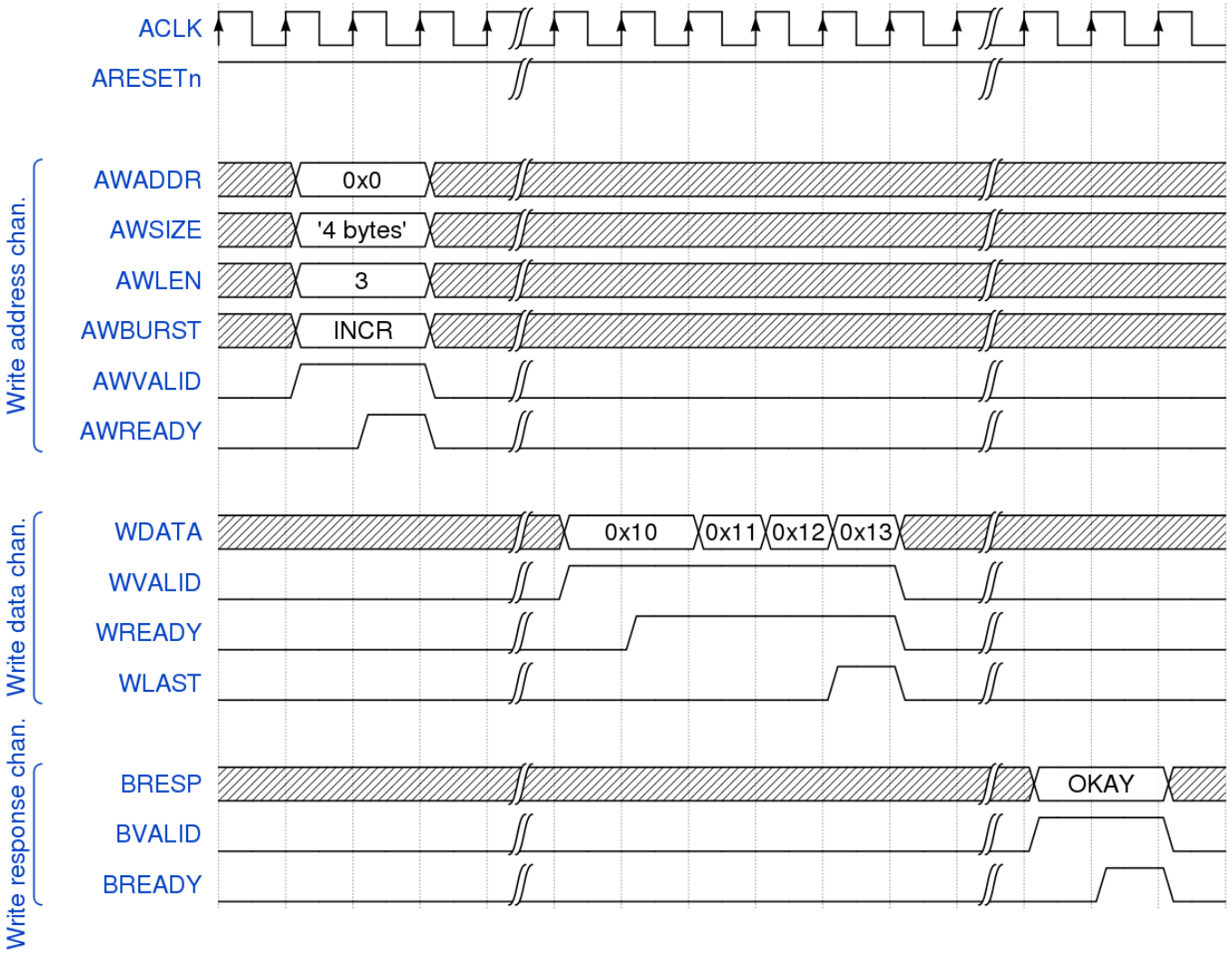


Figure A1-2 Channel architecture of writes

AMBA AXI Bus

- AXI Write
  - Data from master to slave
    - The write address channel carries all of the required address and control information for the transaction
    - The write data channel carries the write data from the master to the slave
    - A slave uses the write response channel to respond to write transactions
      - All write transactions require completion signaling on the write response channel
      - Completion is signaled only for a complete transaction, not for each data transfer in a transaction



Example from Wikipedia

AMBA Bus

- AHB and AXI comparison

AMBA AHB (2.0)	AMBA AXI (3.0)
Single shared address bus and separate read and write data buses	Five separate channels for read address, read data, write address, write data and write response
No simultaneous read and write transactions	Simultaneous read and write transactions
Transmit address of every data, also in burst mode	Transmit address of only first data in burst mode
SPLIT transactions for outstanding transactions	Out-of-Order transactions
No exclusive access support	Exclusive data access support (semaphore operations)
No low power interface	Native low power clock control interface